Capitolo 1 — Introduzione al Web

Es. **1.** **(2 punti)** Qual è il significato degli status code della famiglia `400` di HTTP?

- **A** Errore nella richiesta.
- B Risposta informativa: la richiesta è stata ricevuta e l'elaborazione continua.
- C La richiesta è stata ricevuta, compresa e soddisfatta con successo.
- D Reindirizzamento: per completare la richiesta servono ulteriori azioni del client.
- E Errore nel processamento della richiesta.
- F Errore nella trasmissione dei dati.
- G Errore nella connessione.
- H Errore nella risoluzione del nome di dominio.
- I La risorsa è stata trovata nella cache del browser.

Risposta: A. La prima cifra del codice identifica la classe: 1xx informativi, 2xx successo, 3xx reindirizzamento, 4xx errore del client (la richiesta è sbagliata: ripeterla identica non serve), 5xx errore del server (la richiesta era plausibile ma il server non è riuscito a elaborarla). Problemi di connessione o di trasmissione non producono alcun codice: la risposta non arriva proprio. *(Dispensa: cap. 1, «Codici di stato della risposta»)*

Es. **2.** **(2 punti)** Che cosa indica il codice di stato `403` in una risposta HTTP?

- **A** Il client è identificato ma non ha i permessi (`Forbidden`).
- B La richiesta è malformata (`Bad Request`).
- C Manca l'autenticazione, o non è valida (`Unauthorized`).
- D La risorsa richiesta non esiste (`Not Found`).
- E Il metodo non è ammesso per quella risorsa (`Method Not Allowed`).
- F È stata creata una nuova risorsa (`Created`).
- G Si è verificato un errore interno del server (`Internal Server Error`).
- H La copia in cache del client è ancora valida (`Not Modified`).
- I La risorsa si è spostata in modo permanente a un altro indirizzo.

Risposta: A. `403` appartiene alla classe 4xx. Promemoria: 200 OK, 201 Created, 204 No Content; 301 spostamento permanente, 302 temporaneo, 304 «la tua copia in cache va ancora bene»; 400 richiesta malformata, 401 «non so chi sei», 403 «so chi sei, ma non puoi», 404 risorsa inesistente, 405 metodo non ammesso; 500 errore interno, 503 server non disponibile. *(Dispensa: cap. 1, «Codici di stato della risposta»; cap. 9, «La risposta»)*

Es. **3.** **(2 punti)** Qual è la **stringa di query** della URL `http://www.example.com/cerca?q=css+grid&pagina=2`?

- **A** `q=css+grid&pagina=2`
- B `3000`
- C `8080`
- D `/`
- E `443`
- F (nessuna)

Risposta: A. Scomposta, la URL ha schema `http`, host `www.example.com`, porta 80, percorso `/cerca`, query `q=css+grid&pagina=2` e ancora (nessuna). La query è ciò che segue `?`, fino a `#`: coppie `chiave=valore` separate da `&`. *(Dispensa: cap. 1, «URL: individuare una risorsa»)*

RACCOLTA

Es. **4.** **(2 punti)** Quali delle seguenti affermazioni su Internet e il World Wide Web sono **vere**?

- [x] **A** Il Web è un servizio che funziona *sopra* Internet, insieme ad altri (posta elettronica, trasferimento di file, ...).
- [x] **B** Internet è la rete di reti che collega i dispositivi; il Web è un insieme di documenti ipertestuali collegati fra loro.
- [x] **C** Le risorse del Web sono identificate da URL e trasferite con il protocollo HTTP.
- [ ] D Internet e il World Wide Web sono due nomi della stessa cosa.
- [ ] E Il Web esisteva prima di Internet, che ne è un'estensione.
- [ ] F La posta elettronica è una parte del World Wide Web.
- [ ] G Il Web usa un protocollo di rete alternativo a TCP/IP.
- [ ] H Un documento è nel Web solo se è scritto in HTML5.

Risposte: A, B, C. Internet è l'infrastruttura (la rete di reti basata su TCP/IP); il World Wide Web è uno dei servizi che la usano: documenti ipertestuali identificati da URL e trasferiti con HTTP. Anche la posta elettronica usa Internet, ma non fa parte del Web. *(Dispensa: cap. 1, «Internet e il World Wide Web»)*

Es. **5.** **(2 punti)** Che cos'è un **ipertesto**?

- [x] **A** Un testo con collegamenti ad altri testi, che si può leggere in ordine non lineare.
- [ ] B Un documento HTML che contiene almeno un'immagine o un video.
- [ ] C Un testo che viaggia cifrato durante il trasferimento con HTTPS.
- [ ] D Un documento troppo grande per essere inviato in una sola risposta.
- [ ] E Un file di testo che il server genera dinamicamente a ogni richiesta.
- [ ] F Il corpo di una risposta HTTP, separato dalle intestazioni.

Risposta: A. L'ipertesto è un testo con collegamenti ad altri testi: il lettore non è costretto a una lettura sequenziale ma segue i riferimenti. Il Web è un enorme ipertesto, in cui i collegamenti sono URL. *(Dispensa: cap. 1, «Documenti ipertestuali»)*

Es. **6.** **(2 punti)** Nel protocollo HTTP, come si svolge la comunicazione fra client e server?

- [x] **A** Il client invia una richiesta e il server risponde; il server non inizia mai.
- [ ] B Il server invia periodicamente le pagine aggiornate ai client collegati.
- [ ] C Client e server possono iniziare la comunicazione indifferentemente.
- [ ] D Il server apre la comunicazione inviando l'elenco delle risorse disponibili.
- [ ] E Il client si collega e attende che il server scelga quale pagina inviare.
- [ ] F Il server risponde solo ai client che hanno già una sessione aperta.

Risposta: A. HTTP è un protocollo di tipo richiesta-risposta: il client parla per primo, sempre. Il server si limita a rispondere alle richieste che riceve. *(Dispensa: cap. 1, «HTTP: il protocollo del Web»)*

Es. **7.** **(2 punti)** Che cosa significa che HTTP è un protocollo *stateless*?

- [x] **A** Il server non conserva memoria delle richieste precedenti di un client.
- [ ] B Il server non può salvare in modo permanente i dati in un database.
- [ ] C Le risposte del server non possono mai essere conservate in una cache.
- [ ] D Il client non può inviare dati al server, ma solo riceverne.
- [ ] E La connessione viene chiusa e riaperta dopo ogni pacchetto trasmesso.
- [ ] F Le pagine restituite dal server sono sempre documenti statici.

Risposta: A. Senza stato vuol dire che il protocollo non ricorda nulla fra una richiesta e l'altra. Il vantaggio è la scalabilità (due richieste dello stesso utente possono essere servite da macchine diverse); lo svantaggio è che il concetto di «utente collegato» va costruito sopra HTTP, per esempio con i cookie. *(Dispensa: cap. 1, «HTTP è un protocollo stateless»)*

RACCOLTA

Es. **8.** **(2 punti)** Qual è il principale **vantaggio** del fatto che HTTP sia stateless?

- **A** La scalabilità: ogni richiesta può essere servita da un server diverso.
- B La sicurezza: senza stato le richieste non si possono intercettare.
- C Il login degli utenti è gestito automaticamente dal protocollo.
- D La velocità: le richieste non hanno bisogno di intestazioni.
- E Il server può inviare dati al client senza attendere una richiesta.
- F Il browser non ha più bisogno di conservare le pagine in cache.

Risposta: A. Se il server non deve ricordare nulla, qualunque macchina può rispondere a qualunque richiesta: è ciò che permette di scalare aggiungendo server. Il prezzo è che lo stato (per esempio il login) va gestito con meccanismi costruiti sopra HTTP. *(Dispensa: cap. 1, «HTTP è un protocollo stateless»)*

Es. **9.** **(2 punti)** Quali elementi compaiono nella **prima riga** di una richiesta HTTP/1.1?

- **A** Il metodo, il percorso della risorsa e la versione del protocollo (es. `GET /index.html HTTP/1.1`).
- B Il codice di stato e la sua descrizione (es. `200 OK`).
- C L'indirizzo IP del client e la porta sorgente.
- D Il nome di dominio e il tipo di contenuto richiesto.
- E Il metodo e il corpo della richiesta.
- F La versione del protocollo e la data della richiesta.

Risposta: A. La riga di richiesta contiene metodo, percorso e versione; seguono le intestazioni (fra cui `Host`), una riga vuota e l'eventuale corpo. Il codice di stato compare nella prima riga della *risposta*. *(Dispensa: cap. 1, «I messaggi HTTP»)*

Es. **10.** **(2 punti)** Quale fra le seguenti è una prima riga valida di una **risposta** HTTP/1.1?

- **A** `HTTP/1.1 404 Not Found`
- B `GET /index.html HTTP/1.1`
- C `404 Not Found HTTP/1.1`
- D `HTTP/1.1 GET 200`
- E `Status: 404 /index.html`
- F `Host: www.example.org`

Risposta: A. La riga di stato della risposta è formata da versione del protocollo, codice di stato e frase descrittiva. `GET /index.html HTTP/1.1` è la prima riga di una richiesta; `Host:` è un'intestazione. *(Dispensa: cap. 1, «Esempio di risposta»)*

Es. **11.** **(2 punti)** Quale intestazione di una richiesta HTTP/1.1 indica il nome di dominio del server a cui la richiesta è destinata?

- **A** `Host`
- B `Server`
- C `Origin`
- D `Location`
- E `Domain`
- F `Referer`

Risposta: A. `Host` è obbligatoria in HTTP/1.1: permette a un solo server (un solo indirizzo IP) di ospitare più siti. `Location` compare nelle risposte di reindirizzamento; `Server` descrive il software del server. *(Dispensa: cap. 1, «Header»)*

Es. **12.** **(2 punti)** In una risposta HTTP, quale intestazione indica il **tipo** di dati contenuto nel corpo (per esempio una pagina HTML o un oggetto JSON)?

- **A** `Content-Type`
- B `Content-Length`
- C `Accept`
- D `Media-Type`
- E `Content-Format`
- F `User-Agent`

Risposta: A. `Content-Type` dichiara il tipo del corpo (es. `text/html; charset=utf-8`); `Content-Length` ne indica la lunghezza in byte. `Accept` è un'intestazione di *richiesta*, con cui il client dice quali tipi preferisce ricevere. *(Dispensa: cap. 1, «Header»)*

FACCIATA

Es. **13.** **(2 punti)** Un server risponde `503` a una richiesta. Che cosa conviene fare al client?

- **A** Riprovare più tardi: il problema è del server, non della richiesta.
- B Correggere la richiesta: è stata inviata in un formato non valido.
- C Autenticarsi e ripetere la richiesta con le credenziali.
- D Seguire l'indirizzo indicato nell'intestazione `Location`.
- E Nulla: la richiesta è andata a buon fine.
- F Cancellare la cache, che contiene una copia non più valida.

Risposta: A. I codici 5xx segnalano un problema del server: riprovare più tardi ha senso (`503` indica proprio un'indisponibilità momentanea). I 4xx dicono invece che la colpa è della richiesta, e ripeterla identica non serve. *(Dispensa: cap. 1, «Codici di stato della risposta»)*

Es. **14.** **(2 punti)** Qual è il compito del **DNS**?

- **A** Tradurre un nome di dominio (es. `www.unina.it`) nell'indirizzo IP del server.
- B Instradare i pacchetti IP fino al server di destinazione.
- C Cifrare le richieste HTTP prima che lascino il browser.
- D Assegnare un indirizzo IP al computer quando si collega alla rete.
- E Memorizzare in cache le pagine visitate di recente.
- F Trasformare una URL relativa in una URL assoluta.

Risposta: A. Il DNS è una «rubrica»: dato un nome restituisce un indirizzo. Una volta ottenuto l'indirizzo IP, il browser apre la connessione e invia la richiesta HTTP; il DNS non interviene nel trasferimento dei dati. *(Dispensa: cap. 1, «Cosa succede quando si apre una pagina»)*

Es. **15.** **(2 punti)** L'utente digita `https://www.example.org/` nella barra degli indirizzi e preme Invio. In quale ordine avvengono le operazioni?

- **A** Risoluzione DNS, connessione al server, richiesta HTTP, risposta, visualizzazione.
- B Richiesta HTTP al DNS, che la inoltra al server e restituisce la pagina.
- C Connessione al server, risoluzione DNS, richiesta HTTP, risposta, visualizzazione.
- D Richiesta HTTP, risoluzione DNS della risposta, visualizzazione della pagina.
- E Download delle immagini, poi richiesta del documento HTML che le contiene.
- F Risoluzione DNS, poi visualizzazione della pagina conservata dal server DNS.

Risposta: A. Prima serve l'indirizzo IP (DNS), poi la connessione (TCP, e TLS per HTTPS), poi la richiesta. Le risorse incluse nella pagina (immagini, fogli di stile, script) vengono richieste dopo, man mano che il browser analizza l'HTML. *(Dispensa: cap. 1, «Cosa succede quando si apre una pagina»)*

Es. **16.** **(2 punti)** Nel nome di dominio `www.docenti.unina.it`, qual è il dominio di **primo livello** (TLD)?

- **A** `it`
- B `www`
- C `unina`
- D `docenti`
- E `unina.it`
- F `www.docenti`

Risposta: A. I nomi di dominio si leggono da destra verso sinistra: `it` è il dominio di primo livello, `unina` il secondo livello, `docenti` e `www` sono sottodomini. *(Dispensa: cap. 1, «Nome di dominio»)*

RACCOLTA

Es. **17.** **(2 punti)** Quali delle seguenti affermazioni sulle porte nelle URL sono **vere**?

- **A** Se la URL non indica la porta, per `http` si usa la porta 80.
- **B** Se la URL non indica la porta, per `https` si usa la porta 443.
- **C** In `http://localhost:3000/`, il server è in ascolto sulla porta 3000.
- **D** Una porta scritta esplicitamente segue i due punti dopo il nome dell'host.
- E La porta è obbligatoria in tutte le URL.
- F La porta 80 è riservata ai server HTTPS.
- G La porta viene indicata dopo il percorso, come `/index.html:8080`.
- H La porta identifica la scheda del browser che ha inviato la richiesta.
- I Il numero di porta fa parte del nome di dominio e viene risolto dal DNS.

Risposte: A, B, C, D. La porta identifica il processo server sulla macchina di destinazione. Si può omettere quando è quella predefinita dello schema (80 per HTTP, 443 per HTTPS). In sviluppo si usano spesso porte come 3000 perché le porte sotto la 1024 richiedono privilegi di amministratore. *(Dispensa: cap. 1, «Porta»)*

Es. **18.** **(2 punti)** Un server statico mappa i percorsi delle URL sui file di una cartella. Perché deve rifiutare una richiesta come `GET /../../etc/passwd`?

- **A** Perché `..` farebbe uscire dalla cartella pubblicata (*path traversal*).
- B Perché i percorsi non possono contenere più di due barre.
- C Perché i file senza estensione non possono essere inviati con HTTP.
- D Perché la sequenza `..` non è ammessa nelle URL e la rende malformata.
- E Perché le richieste `GET` non possono riferirsi a file di sistema.
- F Perché il browser non sa visualizzare un file di testo semplice.

Risposta: A. È l'attacco di *path traversal*: risalendo con `..` si arriva fuori dalla radice dei documenti. Un server deve servire solo ciò che sta nella *document root. (Dispensa: cap. 1, «Percorso»; cap. 2, «Dentro un server HTTP»)*

Es. **19.** **(2 punti)** Che cosa aggiunge HTTPS rispetto a HTTP?

- **A** La cifratura della comunicazione e l'autenticazione del server, grazie a TLS.
- B Un formato diverso per i messaggi, non compatibile con HTTP.
- C La garanzia che il sito sia affidabile e non malevolo.
- D La compressione obbligatoria delle risposte.
- E La possibilità di inviare richieste senza intestazioni.
- F Il supporto ai metodi `PUT` e `DELETE`, che HTTP non prevede.

Risposta: A. HTTPS è HTTP trasportato dentro un canale TLS: i messaggi sono gli stessi, ma viaggiano cifrati e il certificato garantisce l'identità del server. Il lucchetto non dice nulla sulle intenzioni di chi gestisce il sito. *(Dispensa: cap. 1, «Schema»; cap. 9, «HTTPS: HTTP dentro un canale TLS»)*

Es. **20.** **(2 punti)** Quale fra le seguenti affermazioni sui metodi (verbi) HTTP è corretta?

- **A** Il metodo dichiara un'intenzione, ma è il server a decidere come gestirla.
- B Il protocollo impedisce a una `GET` di modificare dati sul server.
- C Il metodo `POST` può essere usato solo per caricare file sul server.
- D Il metodo viene scelto dal server e comunicato al client nella risposta.
- E Una richiesta `GET` non può contenere alcun parametro.
- F I metodi definiti da HTTP sono soltanto due: `GET` e `POST`.

Risposta: A. Il verbo è una promessa, non un obbligo: nulla impedisce tecnicamente a un server di modificare dati in risposta a una `GET`, ma violare la semantica dei metodi rende l'applicazione imprevedibile (browser, proxy e motori di ricerca ripetono le `GET` liberamente). *(Dispensa: cap. 1, «Metodi (o verbi) di richiesta»)*

RACCOLTA

Es. **21.** **(5 punti)** Si spieghi che cosa significa che HTTP è un protocollo **stateless**, quali vantaggi e svantaggi comporta, e come si può comunque realizzare un'interazione con stato (per esempio il login).

Traccia della risposta.

- Ogni richiesta è indipendente: il server non conserva memoria delle richieste precedenti dello stesso client.
- Vantaggio: scalabilità. Qualunque server può rispondere a qualunque richiesta, quindi basta aggiungere macchine.
- Svantaggio: non esiste il concetto di utente collegato; dopo il login la richiesta successiva è di uno sconosciuto.
- Lo stato si costruisce sopra HTTP: cookie inviati a ogni richiesta, che contengono i dati (approccio ingenuo, manipolabile) o, meglio, un identificatore di sessione con i dati conservati sul server.

(Dispensa: cap. 1, «HTTP è un protocollo stateless»; cap. 13)

Es. **22.** **(5 punti)** Si descriva la struttura di una **richiesta** e di una **risposta** HTTP, indicando le parti che le compongono e facendo un esempio per ciascuna.

Traccia della risposta.

- Richiesta: riga di richiesta (metodo, percorso, versione), intestazioni (`Host`, `User-Agent`, `Accept`, ...), riga vuota, corpo facoltativo (es. dati di un form inviato con `POST`).
- Risposta: riga di stato (versione, codice, frase), intestazioni (`Content-Type`, `Content-Length`, ...), riga vuota, corpo (es. il documento HTML).
- Esempio: `GET /index.html HTTP/1.1` con `Host: www.example.org`; risposta `HTTP/1.1 200 OK` con `Content-Type: text/html`.
- I codici di stato si raggruppano in classi (2xx successo, 3xx reindirizzamento, 4xx errore del client, 5xx errore del server).

(Dispensa: cap. 1, «I messaggi HTTP»)

Es. **23.** **(5 punti)** Si descrivano le parti di una URL, usando come esempio `https://www.example.org:8443/corsi/web?anno=2025#esami`.

Traccia della risposta.

- Schema `https`: il protocollo (HTTP dentro TLS).
- Host `www.example.org`: il nome di dominio, tradotto dal DNS in un indirizzo IP.
- Porta `8443`: il processo server sulla macchina; se omessa vale 443 per https e 80 per http.
- Percorso `/corsi/web`: la risorsa sul server.
- Query `anno=2025`: parametri `chiave=valore` separati da `&`; ancora `esami`: usata solo dal browser, non viene inviata al server.

(Dispensa: cap. 1, «URL: individuare una risorsa»)

Es. **24.** **(5 punti)** Si descriva che cosa accade, dal momento in cui l'utente digita una URL e preme Invio, fino alla visualizzazione completa della pagina.

Traccia della risposta.

- Il browser estrae l'host dalla URL e lo risolve in un indirizzo IP con il DNS.
- Apre una connessione con il server sulla porta indicata (TCP, più TLS per HTTPS).
- Invia la richiesta HTTP (es. `GET /`) e riceve la risposta con il documento HTML.
- Analizza l'HTML, costruisce il DOM e invia nuove richieste per le risorse incluse (CSS, script, immagini), poi visualizza la pagina.

(Dispensa: cap. 1, «Cosa succede quando si apre una pagina»; cap. 9, «Dal clic alla pagina»)

Es. **25.** **(5 punti)** Si spieghi la differenza fra i codici di stato della famiglia `4xx` e quelli della famiglia `5xx`, con almeno due esempi per ciascuna.

Traccia della risposta.

- 4xx: errore del client. La richiesta è sbagliata e ripeterla identica non serve.
- Esempi 4xx: 400 richiesta malformata, 401 manca l'autenticazione, 403 permessi insufficienti, 404 risorsa inesistente, 405 metodo non ammesso.
- 5xx: errore del server. La richiesta era plausibile ma il server non è riuscito a elaborarla; riprovare più tardi può avere senso.
- Esempi 5xx: 500 errore interno (es. un'eccezione non gestita), 502 risposta non valida da un server a monte, 503 servizio momentaneamente non disponibile.

(Dispensa: cap. 1, «Codici di stato della risposta»)

Capitolo 2 — HTML

Es. **26.** **(2 punti)** Una pagina all'indirizzo `http://localhost:3000/corsi/web/lezioni/index.html` contiene il collegamento `<a href="#esami">`. A quale URL porta il collegamento?

- **A** `http://localhost:3000/corsi/web/lezioni/index.html#esami`
- B `http://localhost:3000/corsi/#esami`
- C `http://localhost:3000/corsi/web/lezioni/#esami`
- D `http://localhost:3000/corsi/web/esami`
- E `http://localhost:3000/corsi/esami`
- F `http://localhost:3000/esami`

Risposta: A. Una URL relativa si risolve rispetto alla directory della pagina corrente (`http://localhost:3000/corsi/web/lezioni/`). Se comincia con `/` si parte invece dalla radice del sito; `.` indica la directory corrente e `..` la directory padre; `?` e `#` sostituiscono solo la query o l'ancora della pagina corrente. *(Dispensa: cap. 2, «URL relative e assolute»)*

Es. **27.** **(2 punti)** Si consideri il seguente form, inviato senza che l'utente abbia modificato alcun campo. Quale URL viene richiesta all'invio?

```
<form action="/iscrizione" method="GET">
  <input type="text" name="nome" id="n" value="Mario Rossi">
  <input type="checkbox" name="esami" value="web" checked>
  <input type="checkbox" name="esami" value="pl2" checked>
  <input type="checkbox" name="esami" value="net" checked>
  <input type="radio" name="anno" value="1">
  <input type="radio" name="anno" value="2" checked>
  <input type="radio" name="anno" value="3">
  <select name="corso">
    <option value="tw">Tecnologie Web</option>
    <option value="igsw">Ingegneria del Software</option>
    <option value="rpd" selected>Reti</option>
  </select>
  <input type="text" id="note" value="nessuna">
  <button type="submit">Invia</button>
</form>
```

- **A** `/iscrizione?nome=Mario+Rossi&esami=web&esami=pl2&esami=net&anno=2&corso=rpd`
- B `/iscrizione?nome=Mario Rossi&esami=web&esami=pl2&esami=net&anno=2&corso=rpd`
- C `/iscrizione?nome=Mario+Rossi&esami=web&esami=pl2&esami=net&anno=2`
- D `/iscrizione?nome=Mario+Rossi&esami=web,pl2,net&anno=2&corso=rpd`
- E `/iscrizione?n=Mario+Rossi&esami=web&esami=pl2&esami=net&anno=2&corso=rpd`
- F `/iscrizione?nome=Mario+Rossi&esami=web&esami=pl2&esami=net&anno=2&corso=rpd&note=nessuna`

Risposta: A. Con `GET` i dati diventano la query della URL indicata da `action`, come coppie `name=valore` nell'ordine del documento. Si inviano solo i campi con un attributo `name` (l'`id` non conta): le checkbox spuntate ripetono il nome, quelle non spuntate non compaiono; un gruppo di radio senza scelta non compare; la `select` invia l'opzione `selected`, o la prima. Gli spazi diventano `+` (codifica delle URL). *(Dispensa: cap. 2, «Form»)*

Es. **28.** **(2 punti)** Quali dei seguenti elementi HTML sono **elementi vuoti** (*void*), cioè non hanno contenuto né tag di chiusura?

- **A** `<br>`
- **B** `<img>`
- **C** `<input>`
- **D** `<meta>`
- **E** `<link>`
- F `<p>`
- G `<script>`
- H `<a>`
- I `<li>`
- J `<title>`
- K `<span>`

Risposte: A, B, C, D, E. Gli elementi vuoti hanno solo il tag di apertura (con eventuali attributi): `br`, `img`, `input`, `meta`, `link`, `hr`, … `<script>` invece va sempre chiuso con `</script>`, anche quando usa l'attributo `src`. *(Dispensa: cap. 2, «Tag, elementi e attributi»)*

RACCOLTA

Es. **29.** **(2 punti)** Quali dei seguenti elementi si collocano normalmente nell'elemento `<head>` di un documento HTML?

- **A** `<title>`
- **B** `<meta charset="utf-8">`
- **C** `<link rel="stylesheet" href="s.css">`
- **D** `<link rel="icon" href="favicon.ico">`
- E `<h1>`
- F `<main>`
- G `<footer>`
- H `<p>`
- I `<table>`

Risposte: A, B, C, D. L'`head` contiene i metadati del documento: titolo (mostrato nella scheda), codifica dei caratteri, fogli di stile, icona, script. Il contenuto visibile (titoli, paragrafi, tabelle, sezioni) va nel `body`. *(Dispensa: cap. 2, «L'elemento head»)*

Es. **30.** **(2 punti)** Una pagina scritta in UTF-8 non dichiara la codifica dei caratteri e il server non la indica nella risposta. Che cosa può succedere?

- **A** Le lettere accentate possono comparire come sequenze di simboli senza senso.
- B Il browser rifiuta di visualizzare la pagina e mostra un errore.
- C Il server risponde con il codice di stato `415 Unsupported Media Type`.
- D Il browser mostra il codice sorgente della pagina invece del contenuto.
- E Gli elementi della pagina vengono visualizzati senza alcuno stile.
- F Le immagini non vengono caricate perché il percorso non è valido.

Risposta: A. Senza `<meta charset="utf-8">` (o l'intestazione `Content-Type` con il charset) il browser deve indovinare la codifica: se sbaglia, i caratteri non ASCII come «è» diventano sequenze di simboli (*mojibake*). *(Dispensa: cap. 2, «L'elemento head»)*

Es. **31.** **(2 punti)** Quale fra le seguenti affermazioni sugli elementi `<h1>`...`<h6>` è corretta?

- **A** Indicano titoli di sezione e la loro gerarchia, non la dimensione del testo.
- B Servono a ingrandire il testo: `<h1>` è il carattere più piccolo.
- C Ogni pagina può contenere al massimo un elemento per ciascun livello.
- D Devono comparire nell'`head`, prima del `title`.
- E Sono elementi in linea e vanno sempre racchiusi in un paragrafo.
- F Si possono usare solo all'interno di una tabella.

Risposta: A. Le intestazioni esprimono la struttura del documento (titolo, sottosezioni, ...): motori di ricerca e lettori di schermo le usano per capirne l'organizzazione. L'aspetto si controlla con il CSS. *(Dispensa: cap. 2, «Elementi fondamentali»)*

Es. **32.** **(2 punti)** Che differenza c'è fra `<strong>` e `<b>`?

- **A** `<strong>` indica un contenuto importante; `<b>` lo evidenzia senza dargli importanza.
- B Nessuna: sono sinonimi e i browser li trattano allo stesso modo in ogni contesto.
- C `<strong>` è deprecato in HTML5 e va sostituito con `<b>`.
- D `<b>` è un elemento di blocco, `<strong>` un elemento in linea.
- E `<strong>` rende il testo rosso, `<b>` lo rende in grassetto.
- F `<b>` si può usare solo nei titoli, `<strong>` solo nei paragrafi.

Risposta: A. HTML descrive il significato, non l'aspetto: `<strong>` ed `<em>` portano semantica (importanza, enfasi), mentre `<b>` e `<i>` sono semplici evidenziazioni tipografiche. Che entrambi appaiano in grassetto è solo lo stile predefinito del browser. *(Dispensa: cap. 2, «Semantica del testo»)*

RACCOLTA

Es. **33.** **(2 punti)** Uno sviluppatore commenta in una pagina HTML la password del database di prova, `<!-- password: s3cr3t -->`. Che cosa succede?

- **A** Il commento non è visibile nella pagina, ma chiunque può leggerlo nel sorgente.
- B Il server rimuove i commenti prima di inviare la pagina, quindi è al sicuro.
- C Il browser cifra i commenti, che sono leggibili solo dallo sviluppatore.
- D La pagina non viene visualizzata perché i commenti non possono contenere `:`.
- E Il commento viene mostrato in un riquadro grigio nella pagina.
- F Il commento è leggibile solo nei DevTools, non nel sorgente della pagina.

Risposta: A. I commenti HTML viaggiano nella risposta come il resto del documento: non vengono visualizzati, ma basta «Visualizza sorgente» o l'ispettore per leggerli. Non sono un nascondiglio. *(Dispensa: cap. 2, «Commenti»)*

Es. **34.** **(2 punti)** Che effetto ha l'attributo `target="_blank"` su un elemento `<a>`?

- **A** Il collegamento si apre in una nuova scheda (o finestra) del browser.
- B Il collegamento si apre nella stessa scheda, sostituendo la pagina.
- C Il collegamento viene disattivato e il testo appare grigio.
- D La risorsa collegata viene scaricata invece che visualizzata.
- E La pagina collegata viene aperta senza stili.
- F Il collegamento viene seguito solo con un doppio clic.

Risposta: A. `target` indica dove aprire la risorsa collegata: `_blank` in un nuovo contesto di navigazione (nuova scheda o finestra); il valore predefinito `_self` la apre nella scheda corrente. *(Dispensa: cap. 2, «L'attributo target»)*

Es. **35.** **(2 punti)** Perché per collegare le pagine **interne** di un sito conviene usare URL relative?

- **A** Perché non cambiano se il sito viene spostato su un altro host, per esempio dallo sviluppo alla produzione.
- B Perché le URL assolute non funzionano con il protocollo HTTPS.
- C Perché i browser caricano più velocemente le URL relative.
- D Perché le URL assolute non possono contenere parametri di query.
- E Perché le URL relative vengono risolte dal DNS e non dal browser.
- F Perché le URL relative non generano richieste HTTP.

Risposta: A. Una URL relativa viene completata dal browser a partire dalla pagina corrente: se cambia il nome dell'host, i collegamenti interni continuano a funzionare senza modifiche. Per le risorse esterne servono invece URL assolute. *(Dispensa: cap. 2, «Meglio relative o assolute?»)*

Es. **36.** **(2 punti)** Un browser richiede `http://localhost:8080/docs/`, che corrisponde a una directory. Che cosa restituisce di solito un server web statico?

- **A** Il file indice della directory (tipicamente `index.html`), se esiste.
- B Sempre l'elenco dei file contenuti nella directory.
- C Il codice di stato `400 Bad Request`, perché le URL non possono finire con `/`.
- D Un archivio compresso con tutti i file della directory.
- E Il primo file della directory in ordine alfabetico.
- F Il file `index.html` della radice del sito.

Risposta: A. Per convenzione il server cerca un file indice (`index.html`). Se non c'è, alcuni server possono generare l'elenco dei file (*autoindex*): comodo, ma pericoloso, perché espone file che non si volevano pubblicare. *(Dispensa: cap. 2, «URL che puntano a una directory»)*

RACCOLTA

Es. **37.** **(2 punti)** In una tabella HTML, come si ottiene una cella che occupa **due colonne**?

- **A** `<td colspan="2">`
- B `<td rowspan="2">`
- C `<td width="2">`
- D `<td span="2">`
- E `<tr colspan="2">`
- F `<th cols="2">`

Risposta: A. `colspan` estende la cella su più colonne, `rowspan` su più righe. Le righe sono `<tr>`, le celle di dati `<td>` e quelle di intestazione `<th>`. *(Dispensa: cap. 2, «Tabelle»)*

Es. **38.** **(2 punti)** Perché non si devono usare le tabelle HTML per impaginare una pagina (colonne, barre laterali, ...)?

- **A** Perché la tabella dichiara dati tabellari: usarla per l'aspetto ne falsa il significato.
- B Perché i browser moderni non visualizzano più le tabelle annidate.
- C Perché una pagina può contenere al massimo una tabella.
- D Perché le tabelle non possono contenere immagini o collegamenti.
- E Perché il contenuto delle tabelle non viene indicizzato in alcun caso.
- F Perché le tabelle si possono usare solo all'interno di un form.

Risposta: A. HTML descrive la struttura: una tabella dice «questi sono dati in righe e colonne». Usata per l'impaginazione confonde lettori di schermo e motori di ricerca, e rende il layout rigido; l'impaginazione si fa con il CSS (Flexbox, Grid). *(Dispensa: cap. 2, «Tabelle»)*

Es. **39.** **(2 punti)** Quali coppie di elementi sono corrette per una **lista di descrizione** (glossario)?

- **A** `<dl>` per la lista, `<dt>` per il termine e `<dd>` per la descrizione.
- B `<ul>` per la lista, `<li>` per il termine e `<p>` per la descrizione.
- C `<dl>` per la lista, `<li>` per il termine e `<dd>` per la descrizione.
- D `<ol>` per la lista, `<dt>` per il termine e `<dd>` per la descrizione.
- E `<list>` per la lista, `<term>` per il termine e `<desc>` per la descrizione.
- F `<dd>` per la lista, `<dl>` per il termine e `<dt>` per la descrizione.

Risposta: A. Le liste di descrizione associano termini e definizioni: `dl` (description list) contiene coppie `dt` (termine) e `dd` (descrizione). Le liste puntate sono `ul`, quelle numerate `ol`, entrambe con elementi `li`. *(Dispensa: cap. 2, «Liste di descrizione»)*

Es. **40.** **(2 punti)** Che testo visualizza il browser per il seguente paragrafo?

```
1 <p>Il tag &lt;b&gt; e &quot;x&quot; &amp; co.</p>
```

- **A** `Il tag <b> e "x" & co.`
- B `Il tag <b> e "x" & co.`
- C `Il tag e "x" & co.` (con il resto del testo in grassetto)
- D `Il tag <b> e "x" & co.`
- E `Il tag lt;b gt; e quot;x quot; amp; co.`
- F `Il tag e "x" & co.`

Risposta: A. I riferimenti a caratteri (entità) permettono di scrivere nel testo caratteri che altrimenti avrebbero un significato speciale: `<` per `<`, `>` per `>`, `&` per `&`, `"` per le virgolette. Il browser li sostituisce con il carattere corrispondente. *(Dispensa: cap. 2, «Riferimenti a caratteri (entità)»)*

RACCOLTA

Es. **41.** **(2 punti)** A che cosa serve il riferimento ` `?

- **A** Inserisce uno spazio che non viene compresso e su cui non si va a capo.
- B Inserisce un a capo, come l'elemento `<br>`.
- C Inserisce un carattere invisibile che separa due parole lunghe.
- D Indica la fine del contenuto di un paragrafo.
- E Inserisce una tabulazione larga quattro spazi.
- F Impedisce al browser di interpretare i tag che seguono.

Risposta: A. HTML comprime spazi e a capo consecutivi in un solo spazio. Lo spazio non divisibile (*non-breaking space*) non viene compresso e impedisce di andare a capo in quel punto (es. `10 km`). *(Dispensa: cap. 2, «Riferimenti a caratteri (entità)»)*

Es. **42.** **(2 punti)** Quali delle seguenti affermazioni sull'attributo `alt` di `<img>` sono **vere**?

- **A** Fornisce un testo alternativo letto dai lettori di schermo.
- **B** Viene mostrato al posto dell'immagine se questa non si carica.
- **C** Va sempre indicato (vuoto, `alt=""`, se l'immagine è solo decorativa).
- D Indica il percorso alternativo da cui scaricare l'immagine.
- E Specifica il testo del suggerimento mostrato al passaggio del mouse.
- F Imposta la larghezza alternativa dell'immagine sui dispositivi mobili.
- G È facoltativo e non ha alcun effetto sull'accessibilità.
- H Contiene la didascalia che il browser scrive sotto l'immagine.

Risposte: A, B, C. `alt` descrive l'immagine a chi non può vederla (lettori di schermo, immagine non caricata, motori di ricerca). Non è facoltativo: per le immagini decorative si usa la stringa vuota. *(Dispensa: cap. 2, «Immagini»)*

Es. **43.** **(2 punti)** Il browser riceve un documento HTML che include un foglio di stile, uno script e tre immagini, tutti con URL diverse e nessuno in cache. Quante richieste HTTP servono, in tutto, per caricare la pagina?

- **A** Sei: una per il documento e una per ciascuna risorsa inclusa.
- B Una sola: il server invia il documento con tutte le risorse incluse.
- C Due: una per il documento e una per tutte le risorse insieme.
- D Tre: una per il documento, una per lo stile e una per lo script.
- E Quattro: le immagini viaggiano in un'unica richiesta.
- F Cinque: il documento HTML non richiede una richiesta separata.

Risposta: A. Ogni risorsa ha la propria URL e viene richiesta separatamente: il browser legge l'HTML e, man mano che incontra `link`, `script` e `img`, invia nuove richieste. Si vede bene nella scheda Rete dei DevTools. *(Dispensa: cap. 2, «Dietro le quinte»)*

Es. **44.** **(2 punti)** Una pagina contiene `<h2 id="esami">Esami</h2>`. Come si scrive un collegamento che porti direttamente a quell'intestazione, nella stessa pagina?

- **A** `<a href="#esami">`
- B `<a href="esami">`
- C `<a name="#esami">`
- D `<a href="?id=esami">`
- E `<a target="esami">`
- F `<a href="/esami">`

Risposta: A. L'ancora (il frammento dopo `#`) fa scorrere la pagina fino all'elemento con quell'`id`. Il frammento non viene inviato al server. *(Dispensa: cap. 2, «Ancore nelle URL»)*

RACCOLTA

Es. **45.** **(2 punti)** Quali delle seguenti affermazioni sugli attributi `id` e `class` sono **vere**?

- **A** Un `id` deve essere unico nel documento.
- **B** Più elementi possono avere la stessa classe.
- **C** Un elemento può avere più classi, separate da spazi.
- D Un elemento può avere più `id`, separati da spazi.
- E `class` si può usare solo sugli elementi di blocco.
- F Due elementi con lo stesso `id` vengono uniti dal browser.
- G `id` e `class` sono attributi riservati agli elementi del form.
- H Il valore di `class` deve iniziare con un punto.

Risposte: A, B, C. `id` identifica un singolo elemento (unico nel documento, usato da ancore, CSS `#id` e `getElementById`); `class` raggruppa elementi (anche più classi per elemento, CSS `.classe`). Sono attributi globali, validi su qualunque elemento. *(Dispensa: cap. 2, «Attributi globali»)*

Es. **46.** **(2 punti)** In un form, quale attributo di un `<input>` determina la chiave con cui il valore viene inviato al server?

- **A** `name`
- B `id`
- C `value`
- D `for`
- E `label`
- F `key`

Risposta: A. I dati di un form viaggiano come coppie `name=valore`: un campo senza `name` non viene inviato. L'`id` serve a collegare l'etichetta (`<label for="...">`) e a trovare l'elemento con CSS o JavaScript. *(Dispensa: cap. 2, «Invio del form»)*

Es. **47.** **(2 punti)** Come si collega correttamente un'etichetta a un campo di testo?

- **A** `<label for="email">Email</label> <input id="email" name="email">`
- B `<label name="email">Email</label> <input name="email">`
- C `<label id="email">Email</label> <input for="email">`
- D `<label for="email">Email</label> <input name="mail" class="email">`
- E `<label>Email</label> <input label="Email">`
- F `<label href="#email">Email</label> <input id="email">`

Risposta: A. L'attributo `for` dell'etichetta contiene l'`id` del campo: così un clic sull'etichetta attiva il campo e i lettori di schermo leggono l'etichetta insieme al campo. In alternativa il campo si può mettere dentro la `label`. *(Dispensa: cap. 2, «Etichette»)*

Es. **48.** **(2 punti)** Quali delle seguenti affermazioni sull'invio di un form sono **vere**?

- **A** Con `GET` i dati compaiono nella URL, come stringa di query.
- **B** Con `POST` i dati viaggiano nel corpo della richiesta.
- **C** Una ricerca, che non modifica nulla, si presta all'invio con `GET`.
- D Con `POST` i dati sono cifrati anche senza HTTPS.
- E Con `GET` i dati viaggiano nelle intestazioni della richiesta.
- F Con `POST` non è possibile inviare caratteri accentati.
- G Il metodo predefinito di un form, se non indicato, è `POST`.
- H Una richiesta `GET` non può contenere più di un parametro.

Risposte: A, B, C. `GET` mette i dati nella query: comodo per ricerche (la URL si può salvare e condividere) ma i dati restano nella cronologia. `POST` li mette nel corpo, ed è adatto alle operazioni che modificano lo stato. Nessuno dei due cifra nulla: lo fa HTTPS. Il metodo predefinito è `GET`. *(Dispensa: cap. 2, «Invio con GET»)*

RACCOLTA

Es. **49.** **(2 punti)** Un campo `q` contiene `pane & vino`. Come compare nella query string della richiesta `GET`?

- **A** `q=pane+%26+vino`
- B `q=pane & vino`
- C `q=pane+&+vino`
- D `q=pane%20&%20vino`
- E `q="pane & vino"`
- F `q=pane&vino`

Risposta: A. Nella query alcuni caratteri hanno un significato (`&` separa i parametri, `=` separa chiave e valore), quindi i valori vengono codificati: lo spazio diventa `+` e `&` diventa `%26` (*percent encoding*). *(Dispensa: cap. 2, «Codifica delle URL»)*

Es. **50.** **(2 punti)** Un form ha tre checkbox con `name="esami"` e l'utente non ne spunta nessuna. Che cosa riceve il server per il parametro `esami`?

- **A** Nulla: il parametro non compare affatto nei dati inviati.
- B Il parametro con valore vuoto, `esami=`.
- C Il parametro con valore `off` per ciascuna casella.
- D Il parametro con valore `false`.
- E Il parametro con valore `null`.
- F Il form non viene inviato finché non si spunta almeno una casella.

Risposta: A. Le checkbox non spuntate non vengono inviate: il server non deve aspettarsi di trovare il parametro. Le caselle spuntate con lo stesso `name` ripetono invece la chiave (`esami=web&esami=pl2`). *(Dispensa: cap. 2, «Caselle di spunta»)*

Es. **51.** **(2 punti)** Che cosa rende mutuamente esclusivi tre pulsanti radio di un form?

- **A** Il fatto di avere lo stesso valore dell'attributo `name`.
- B Il fatto di avere lo stesso valore dell'attributo `id`.
- C Il fatto di essere dentro lo stesso elemento `<label>`.
- D L'attributo `exclusive` sul primo pulsante del gruppo.
- E Il fatto di avere valori diversi dell'attributo `value`.
- F L'attributo `type="group"` sul `<form>`.

Risposta: A. I radio con lo stesso `name` formano un gruppo in cui si può scegliere una sola opzione; all'invio si trasmette `name=value` dell'opzione scelta. Gli `id` devono anzi essere diversi. *(Dispensa: cap. 2, «Pulsanti radio»)*

Es. **52.** **(2 punti)** Qual è la differenza fra `<div>` e `<span>`?

- **A** `<div>` è un contenitore generico di blocco, `<span>` un contenitore generico in linea.
- B `<div>` è un contenitore in linea, `<span>` un contenitore di blocco.
- C `<span>` si può usare solo all'interno di una tabella.
- D `<div>` ha un significato semantico preciso, `<span>` no.
- E `<span>` ridimensiona il testo, `<div>` lo allinea.
- F Sono sinonimi: cambia solo il numero di caratteri da scrivere.

Risposta: A. Entrambi sono privi di significato e servono a raggruppare contenuto da stilizzare o manipolare: `div` raggruppa blocchi (va a capo), `span` porzioni di testo all'interno di una riga. Se esiste un tag semantico adatto, va preferito. *(Dispensa: cap. 2, «Organizzare il contenuto»)*

Es. **53.** **(2 punti)** Quali dei seguenti sono **tag semantici** di HTML5 per strutturare una pagina?

- **A** `<header>`
- **B** `<nav>`
- **C** `<main>`
- **D** `<article>`
- **E** `<footer>`
- **F** `<aside>`
- G `<div>`
- H `<span>`
- I `<b>`
- J `<center>`
- K `<container>`

Risposte: A, B, C, D, E, F. `header`, `nav`, `main`, `article`, `section`, `aside` e `footer` dicono che cosa contengono: aiutano lettori di schermo e motori di ricerca e rendono il codice più leggibile. `div` e `span` sono contenitori generici; `<container>` non esiste. *(Dispensa: cap. 2, «Tag semantici»)*

FACOLTÀ

Es. **54.** **(2 punti)** Quale dei seguenti frammenti HTML è annidato **correttamente**?

- **A** `<p>Un <em>testo <strong>molto</strong> importante</em>.</p>`
- B `<p>Un <em>testo <strong>molto</em> importante</strong>.</p>`
- C `<p>Un <em>testo</p> molto importante</em>.`
- D `<em><p>Un testo</em> molto importante.</p>`
- E `<p>Un <strong>testo <em>molto</strong></em> importante.</p>`

Risposta: A. Il documento è un albero: un elemento aperto dentro un altro deve chiudersi prima del suo contenitore. Tag «incrociati» non sono validi e il browser li corregge a modo suo. *(Dispensa: cap. 2, «Il documento come albero»)*

Es. **55.** **(2 punti)** Perché durante lo sviluppo conviene spuntare «Disattiva cache» nella scheda Rete dei DevTools?

- **A** Per essere sicuri che il browser scarichi la versione aggiornata di CSS e script.
- B Perché la cache impedisce di vedere le richieste nella scheda Rete.
- C Perché altrimenti il server rifiuta le richieste con codice 403.
- D Perché la cache rende la pagina vulnerabile ad attacchi XSS.
- E Perché la cache funziona solo con HTTPS e in sviluppo si usa HTTP.
- F Per poter modificare il codice HTML direttamente nel browser.

Risposta: A. La cache HTTP conserva copie delle risorse già scaricate per non richiederle di nuovo: ottima in produzione, ma in sviluppo può farci vedere una versione vecchia di un file appena modificato. *(Dispensa: cap. 2, «Analisi di rete»)*

Es. **56.** **(2 punti)** Che cosa significa che HTML è un *Living Standard*?

- **A** Che è una specifica aggiornata di continuo, senza numeri di versione.
- B Che ogni browser ne definisce una versione diversa.
- C Che la specifica viene riscritta da zero ogni anno.
- D Che si può usare solo nelle pagine generate dinamicamente.
- E Che il suo sviluppo si è fermato con HTML5.
- F Che solo i tag più usati fanno parte dello standard.

Risposta: A. Lo standard HTML, curato dal WHATWG, evolve in modo continuo: nuove funzionalità vengono aggiunte senza versioni numerate («HTML5» è ormai solo un nome). *(Dispensa: cap. 2, «Un po' di storia»)*

Es. **57.** **(5 punti)** Si descrivano le differenze fra l'invio di un form con il metodo `GET` e con il metodo `POST`, indicando in quali casi conviene usare l'uno o l'altro.

Traccia della risposta.

- `GET`: i dati diventano la query della URL (`?nome=valore&...`), visibile, salvabile nei segnalibri, registrata in cronologia e nei log.
- `POST`: i dati viaggiano nel corpo della richiesta, codificati allo stesso modo (`application/x-www-form-urlencoded`).
- `GET` per operazioni che leggono (ricerche, filtri): è sicura e ripetibile.
- `POST` per operazioni che modificano lo stato (registrazioni, acquisti) o per dati lunghi; nessuno dei due metodi cifra i dati: per questo serve HTTPS.

(Dispensa: cap. 2, «Invio con GET»)

Es. **58.** **(5 punti)** Si spieghi perché in HTML conviene usare i **tag semantici** (`header`, `nav`, `main`, `article`, `footer`, ...) invece di semplici `div`, con un esempio.

Traccia della risposta.

- I tag semantici dichiarano il ruolo del contenuto, non il suo aspetto.
- Accessibilità: i lettori di schermo permettono di saltare alla navigazione o al contenuto principale.
- Motori di ricerca e altri programmi capiscono meglio la struttura della pagina.
- Il codice è più leggibile e manutenibile; l'aspetto resta compito del CSS.
- Esempio: `<nav>` con i link del menu, `<main>` con il contenuto, `<footer>` con contatti e note legali.

(Dispensa: cap. 2, «Tag semantici»)

ACCOLTA

Es. **59.** **(5 punti)** Si spieghi la differenza fra URL assolute e URL relative (con e senza barra iniziale e con i segmenti `.` e `..`), indicando quando conviene usare le une o le altre.

Traccia della risposta.

- Assoluta: contiene schema e host (`https://example.org/a.html`).
- Relativa con `/` iniziale: si risolve a partire dalla radice del sito corrente.
- Relativa senza `/`: si risolve a partire dalla directory della pagina corrente; `.` è la directory corrente, `..` la directory padre.
- Le relative per le risorse interne (sopravvivono al cambio di host); le assolute per le risorse esterne.

(Dispensa: cap. 2, «URL relative e assolute»)

Es. **60.** **(5 punti)** Si spieghi che cos'è la **cache HTTP**, quali vantaggi offre e perché durante lo sviluppo di un sito può creare problemi.

Traccia della risposta.

- Il browser conserva copie delle risorse già scaricate e le riusa invece di richiederle di nuovo.
- Vantaggi: meno richieste e meno dati trasferiti, pagine più veloci, meno carico sul server.
- Il server può indicare per quanto tempo una risorsa è valida; per verificarla il client può chiedere se è cambiata e ricevere `304 Not Modified`.
- In sviluppo si rischia di vedere una versione vecchia di CSS o script: si disattiva la cache dai DevTools (scheda Rete).

(Dispensa: cap. 2, «Analisi di rete»)

Capitolo 3 — CSS

Es. **61.** **(2 punti)** Si consideri il seguente insieme di regole CSS incluse in un singolo foglio di stile importato nella pagina web il cui `<body>` è riportato di seguito. Qual è il colore finale dell'elemento `h1`?

```css
.h { color: gray !important; }
h1 { color: brown; }
h1#c { color: black; }
div.b h1.f { color: green; }
#c > .h { color: yellow !important; }
```

```html
<body><div id="c" class="b">
  <h1 class="f h" id="a" style="color: orange;">
    Hi!
  </h1>
</div></body>
```

- **A** yellow
- B gray
- C brown
- D black
- E green
- F orange

Risposta: A. Le dichiarazioni `!important` stanno in un bucket più prioritario di quello delle dichiarazioni normali, stile inline compreso: fra le regole rimaste vince la specificità più alta, (1, 1, 0) (`#c > .h`). Le regole: `.h` (0, 1, 0); `h1` (0, 0, 1); `h1#c` non seleziona l'`h1`; `div.b h1.f` (0, 2, 2); `#c > .h` (1, 1, 0). *(Dispensa: cap. 3, «La cascata»)*

Es. **62.** **(2 punti)** Quale dei seguenti selettori ha la **specificità** più alta?

- **A** `div#cont a.lnk`
- B `a[href]`
- C `#x a`
- D `ul.lst li a`

Risposta: A. La specificità si confronta componente per componente: prima il numero di id, poi quello di classi, attributi e pseudo-classi, poi quello di tipi di elemento; non è un numero a tre cifre (una classe batte qualunque numero di tipi, un id qualunque numero di classi). Combinatori e `*` non contano. Qui: `a[href]` (0, 1, 1), `#x a` (1, 0, 1), `ul.lst li a` (0, 1, 3), `div#cont a.lnk` (1, 1, 2). *(Dispensa: cap. 3, «Specificità»)*

RACCOLTA

Es. **63.** **(2 punti)** Data la lista seguente, quali elementi `li` seleziona il selettore `li:nth-child(2n+1)`?

```
<ul>
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>4</li>
  <li>5</li>
  <li>6</li>
  <li>7</li>
  <li>8</li>
</ul>
```

- **A** Gli elementi in posizione 1, 3, 5 e 7.
- B Gli elementi in posizione 2, 4, 6 e 8.
- C Gli elementi in posizione 3, 5 e 7.
- D Gli elementi in posizione 1, 2, 3, 4, 5, 6, 7 e 8.
- E Gli elementi in posizione 1 e 8.

Risposta: A. `:nth-child(an+b)` seleziona i figli in posizione `an+b` per n = 0, 1, 2, ...; le posizioni si contano **a partire da 1**. `odd` equivale a `2n+1` ed `even` a `2n`. Con `2n+1` si ottengono le posizioni 1, 3, 5, 7. *(Dispensa: cap. 3, «Relazioni di posizione»)*

Es. **64.** **(2 punti)** Che cosa seleziona `.art p:nth-child(5)` nel frammento seguente?

```
<div class="art">
  <h2>Titolo</h2>
  <p>Testo 2</p>
  <p>Testo 3</p>
  <h2>Titolo</h2>
  <p>Testo 5</p>
  <p>Testo 6</p>
  <h2>Titolo</h2>
</div>
```

- **A** Il 5° figlio del `div`, che è un paragrafo.
- B Tutti i paragrafi del `div` dal 5° in poi.
- C Il 5° figlio di ogni paragrafo.
- D Nessun elemento: `:nth-child` conta solo i paragrafi, e il 5° non esiste.

Risposta: A. `:nth-child(k)` conta **tutti** i figli del genitore, di qualunque tipo; poi il selettore di tipo `p` controlla che quello in posizione k sia un paragrafo. Qui il 5° figlio è un `p`. *(Dispensa: cap. 3, «Il conteggio senza selettore di tipo»)*

Es. **65.** **(2 punti)** Quale dei seguenti selettori CSS seleziona tutti i link contenuti in `<li>` di posto pari in una lista non ordinata con classe `lst`?

- **A** `ul.lst li:nth-child(even) a`
- B `ol.lst li:nth-child(even) a`
- C `ul.lst::nth-child(2) a`
- D `ol.lst li:even-child a`
- E `ul.lst::nth-child(even) a`
- F `a li:even-child ul .lst`

Risposta: A. Il selettore si legge da sinistra a destra, dal contenitore più esterno: `ul.lst` è una lista non ordinata **e** di classe `lst` (senza spazio: selettore composto); lo spazio è il combinatore discendente; `li:nth-child(even)` sono le voci di posto pari; `a` sono gli elementi cercati, a qualunque profondità dentro la voce. Nelle alternative: `ol.lst` è una lista ordinata; `::` introduce gli pseudo-elementi, e `::nth-child` non esiste; `:even-child` non esiste; l'ordine conta: a sinistra va il contenitore più esterno. *(Dispensa: cap. 3, «Combinatori»; «Relazioni di posizione») — Fonte: prima prova intercorso del 23 aprile 2024.*

RACCOLTA

Es. **66.** **(2 punti)** Si consideri il seguente insieme di regole CSS incluse in un singolo foglio di stile importato nella pagina web il cui `<body>` è riportato di seguito. Qual è il colore finale dell'elemento `h1`?

```css
.f { color: yellow; }
#g { color: blue; }
div h1 { color: pink !important; }
h1 .f { color: green !important; }
#cont h1.f { color: black; }
```

```html
<body><div id="cont">
  <h1 class="f" id="g" style="color: red;">
    Hi!
  </h1>
</div></body>
```

- **A** pink
- B yellow
- C blue
- D green
- E black
- F red

Risposta: A. Le dichiarazioni `!important` battono quelle normali, compreso lo stile inline. Fra le due marcate `!important`, `h1 .f` **non** seleziona l'`h1`: indica un elemento di classe `f` *dentro* un `h1` (combinatore discendente). Resta `div h1`, quindi il colore è `pink`. *(Dispensa: cap. 3, «La cascata») — Fonte: prima prova intercorso del 23 aprile 2024.*

Es. **67.** **(2 punti)** Si consideri il seguente insieme di regole CSS incluse in un singolo foglio di stile importato nella pagina web il cui `<body>` è riportato di seguito. Qual è il colore finale dell'elemento `h1`?

```css
div h1 { color: orange; }
#x .w { color: blue; }
.y #z { color: red; }
div.y h1.w { color: green; }
```

```html
<body><div id="x" class="y">
  <h1 class="w" id="z">
    Hi!
  </h1>
</div></body>
```

- **A** red
- B orange
- C blue
- D green

Risposta: A. Tutte e quattro le regole selezionano l'`h1`. Specificità: `div h1` (0, 0, 2), `#x .w` (1, 1, 0), `.y #z` (1, 1, 0), `div.y h1.w` (0, 2, 2). Vincono a pari merito le due con un id e una classe; a parità di specificità prevale l'ultima nell'ordine: `red`. *(Dispensa: cap. 3, «Specificità») — Fonte: facsimile mostrato in aula.*

Es. **68.** **(2 punti)** Si considerino le regole e il documento seguenti. Qual è il colore finale dell'elemento `h1`?

```css
#cont { color: red; }
.box { color: green !important; }
h1 { color: blue; }
```

```html
<body><div id="cont" class="box">
  <h1>Titolo</h1>
</div></body>
```

- **A** blue
- B red
- C green
- D black

Risposta: A. Le prime due regole selezionano il `div`, non l'`h1`: all'`h1` arriverebbe il colore solo per **ereditarietà**, che ha la priorità più bassa in assoluto. Basta una regola qualunque che selezioni direttamente l'`h1` (qui `h1`, con specificità minima) per vincere, indipendentemente da id e `!important` delle regole del genitore. *(Dispensa: cap. 3, «Ereditarietà»)*

Es. **69.** **(2 punti)** Si considerino le regole e il documento seguenti. Qual è il colore finale del paragrafo?

```css
#t { color: blue; }
p { color: green !important; }
#t.nota { color: orange; }
```

```html
<body>
  <p id="t" class="nota" style="color: red;">
    Testo
  </p>
</body>
```

- **A** green
- B blue
- C red
- D orange

Risposta: A. `!important` porta la dichiarazione nel bucket più prioritario: batte lo stile inline (che è una dichiarazione normale) e i selettori più specifici. La specificità conta solo fra dichiarazioni dello stesso bucket. *(Dispensa: cap. 3, «Origine e importanza»)*

RACCOLTA

Es. **70.** **(2 punti)** Nella regola `p.nota { color: gray; margin: 0; }`, che cosa sono `p.nota`, `color` e `gray`?

- **A** Il selettore, una proprietà e il suo valore.
- B La proprietà, il selettore e la dichiarazione.
- C Il blocco di dichiarazione, un valore e una proprietà.
- D Il selettore, un valore e la sua unità di misura.
- E Il nome della regola, un attributo HTML e il suo valore.
- F L'elemento, la classe e lo stile inline.

Risposta: A. Una regola è formata da un selettore e da un blocco di dichiarazioni fra graffe; ogni dichiarazione è una coppia `proprietà: valore`, terminata da `;`. *(Dispensa: cap. 3, «Anatomia di una regola»)*

Es. **71.** **(2 punti)** Quali dei seguenti sono modi validi per applicare regole CSS a una pagina HTML?

- **A** `<link rel="stylesheet" href="stile.css">` nell'`head`
- **B** Un elemento `<style>` con le regole al suo interno
- **C** L'attributo `style="..."` su un singolo elemento
- D `<script src="stile.css">` nell'`head`
- E `<css href="stile.css">` nell'`head`
- F `<style src="stile.css">` senza contenuto
- G `<link rel="css" src="stile.css">` nel `body`
- H Un commento HTML che contiene le regole

Risposte: A, B, C. Foglio esterno con `<link rel="stylesheet">`, elemento `<style>` nel documento e attributo `style` (stile inline). Il foglio esterno è da preferire: separa contenuto e presentazione e si condivide fra più pagine (e il browser lo tiene in cache). *(Dispensa: cap. 3, «Come includere il CSS in una pagina»)*

Es. **72.** **(2 punti)** Quale selettore seleziona tutti i collegamenti che puntano a un file **PDF** (cioè il cui `href` termina con `.pdf`)?

- **A** `a[href$=".pdf"]`
- B `a[href^=".pdf"]`
- C `a[href*=".pdf"]`
- D `a[href=".pdf"]`
- E `a.pdf`
- F `a[href~=".pdf"]`

Risposta: A. Nei selettori di attributo `^=` significa «inizia con», `$=` «termina con», `*=` «contiene», `=` «è uguale a». `a.pdf` selezionerebbe i link con classe `pdf`. *(Dispensa: cap. 3, «Corrispondenza parziale sugli attributi»)*

Es. **73.** **(2 punti)** Che differenza c'è fra i selettori `p.nota` e `p .nota`?

- **A** `p.nota`: paragrafi di classe `nota`; `p .nota`: elementi di classe `nota` dentro un paragrafo.
- B Nessuna: lo spazio nei selettori viene ignorato.
- C `p.nota`: elementi di classe `nota` dentro un paragrafo; `p .nota`: paragrafi di classe `nota`.
- D `p .nota` seleziona sia i paragrafi sia gli elementi di classe `nota`.
- E `p.nota` non è valido: classe e tipo vanno separati da uno spazio.
- F `p .nota` seleziona i paragrafi che seguono un elemento di classe `nota`.

Risposta: A. Senza spazio le condizioni si riferiscono allo stesso elemento (selettore composto: è un `p` **e** ha classe `nota`); lo spazio è il combinatore discendente. *(Dispensa: cap. 3, «Selettori composti»)*

RACCOLTA

Es. **74.** **(2 punti)** Quali paragrafi seleziona il selettore `div > p` nel frammento seguente?

```
<div id="d">
  <h2 id="t">Titolo</h2>
  <p id="p1">Uno</p>
  <section id="s">
    <p id="p2">Due</p>
  </section>
  <p id="p3">Tre</p>
</div>
```

- **A** p1 e p3
- B p1, p2 e p3
- C Solo p1
- D Solo p2
- E p2 e p3

Risposta: A. Il combinatore figlio `>` seleziona solo i figli diretti: `p2` è dentro la `section`, quindi è un discendente del `div` ma non un suo figlio. *(Dispensa: cap. 3, «Combinatori»)*

Es. **75.** **(2 punti)** Quali paragrafi seleziona il selettore `div p` nel frammento seguente?

```
<div id="d">
  <h2 id="t">Titolo</h2>
  <p id="p1">Uno</p>
  <section id="s">
    <p id="p2">Due</p>
  </section>
  <p id="p3">Tre</p>
</div>
```

- **A** p1, p2 e p3
- B p1 e p3
- C Solo p2
- D Solo p1
- E Nessuno

Risposta: A. Il combinatore discendente (lo spazio) seleziona gli elementi a qualunque profondità dentro il `div`, compreso `p2`, che sta nella `section`. *(Dispensa: cap. 3, «Combinatori»)*

Es. **76.** **(2 punti)** Quali paragrafi seleziona il selettore `h2 + p` nel frammento seguente?

```
<div id="d">
  <h2 id="t">Titolo</h2>
  <p id="p1">Uno</p>
  <section id="s">
    <p id="p2">Due</p>
  </section>
  <p id="p3">Tre</p>
</div>
```

- **A** Solo p1
- B p1 e p3
- C p1, p2 e p3
- D Solo p3
- E Nessuno

Risposta: A. `+` seleziona il fratello **immediatamente** successivo: solo `p1` segue direttamente l'`h2`. *(Dispensa: cap. 3, «Combinatori»)*

RACCOLTA

Es. **77.** **(2 punti)** Quali paragrafi seleziona il selettore `h2 ~ p` nel frammento seguente?

```
<div id="d">
  <h2 id="t">Titolo</h2>
  <p id="p1">Uno</p>
  <section id="s">
    <p id="p2">Due</p>
  </section>
  <p id="p3">Tre</p>
</div>
```

- **A** p1 e p3
- B Solo p1
- C p1, p2 e p3
- D Solo p3
- E p2 e p3

Risposta: A. `~` seleziona tutti i fratelli successivi di quel tipo: `p1` e `p3` sono fratelli dell'`h2` che lo seguono; `p2` non è un fratello (sta nella `section`). *(Dispensa: cap. 3, «Combinatori»)*

Es. **78.** **(2 punti)** Quali paragrafi seleziona il selettore `section + p` nel frammento seguente?

```
<div id="d">
  <h2 id="t">Titolo</h2>
  <p id="p1">Uno</p>
  <section id="s">
    <p id="p2">Due</p>
  </section>
  <p id="p3">Tre</p>
</div>
```

- **A** Solo p3
- B p1 e p3
- C Solo p2
- D p2 e p3
- E Nessuno

Risposta: A. `p3` è il fratello immediatamente successivo della `section`. *(Dispensa: cap. 3, «Combinatori»)*

Es. **79.** **(2 punti)** Quali delle seguenti sono **pseudo-classi** CSS?

- **A** `:hover`
- **B** `:focus`
- **C** `:first-child`
- **D** `:visited`
- **E** `:invalid`
- F `::before`
- G `::first-line`
- H `::selection`
- I `.hover`
- J `[hover]`

Risposte: A, B, C, D, E. Le pseudo-classi (un solo `:`) selezionano elementi in base a uno stato (`:hover`, `:focus`, `:visited`, `:invalid`) o alla posizione (`:first-child`, `:nth-child`). Gli pseudo-elementi (`::`) selezionano invece *parti* di un elemento o contenuto generato. *(Dispensa: cap. 3, «Pseudo-classi» e «Pseudo-elementi»)*

Es. **80.** **(2 punti)** Perché i browser limitano le proprietà che si possono applicare con `:visited` (sostanzialmente ai soli colori)?

- **A** Per impedire a una pagina di scoprire, misurando lo stile, quali siti ha visitato l'utente.
- B Perché i link visitati non fanno parte del DOM.
- C Perché `:visited` funziona solo sui link con `target="_blank"`.
- D Perché la cronologia è salvata sul server e non è accessibile al CSS.
- E Perché le altre proprietà richiederebbero di ricaricare la pagina.
- F Perché `:visited` è stata deprecata ed esiste solo per compatibilità.

Risposta: A. Se `:visited` potesse cambiare dimensioni o posizione, uno script potrebbe misurare i link e ricostruire la cronologia dell'utente: è un problema di privacy, e i browser lo evitano consentendo solo modifiche di colore, che restano invisibili a JavaScript. *(Dispensa: cap. 3, «Stati storici»)*

RACCOLTA

Es. **81.** **(2 punti)** Quali delle seguenti affermazioni sul contenuto generato con `::before` e `::after` sono **vere**?

- **A** Richiede la proprietà `content` per comparire.
- **B** Non fa parte del DOM: non si trova, per esempio, con `querySelector`.
- **C** Non dovrebbe contenere informazioni essenziali, perché non è vero contenuto.
- D Viene aggiunto al codice HTML della pagina inviata dal server.
- E Si può selezionare con `document.querySelector('::before')`.
- F Si inserisce prima o dopo l'elemento, fuori dal suo riquadro.
- G Funziona solo sugli elementi vuoti come `<img>` e `<input>`.
- H Sostituisce il contenuto dell'elemento a cui si applica.

Risposte: A, B, C. `::before` e `::after` creano un figlio fittizio all'inizio o alla fine del contenuto dell'elemento, solo se `content` è impostato. Non esiste nel DOM e non è affidabile per lettori di schermo e copia-incolla: va usato per decorazioni, non per informazioni. *(Dispensa: cap. 3, «Pseudo-elementi»)*

Es. **82.** **(2 punti)** Qual è l'ordine dei bucket di origine e importanza, dal **meno** al **più** prioritario?

- **A** User agent, utente, autore, autore `!important`, utente `!important`, user agent `!important`.
- B User agent, utente, autore, user agent `!important`, utente `!important`, autore `!important`.
- C Autore, utente, user agent, autore `!important`, utente `!important`, user agent `!important`.
- D User agent `!important`, utente `!important`, autore `!important`, autore, utente, user agent.
- E User agent, autore, utente, utente `!important`, autore `!important`, user agent `!important`.

Risposta: A. Fra le dichiarazioni normali vince l'autore (che sovrascrive gli stili predefiniti del browser); fra le `!important` l'ordine si **inverte**: le esigenze dell'utente (per esempio l'alto contrasto per accessibilità) devono poter battere quelle del sito. *(Dispensa: cap. 3, «Origine e importanza»)*

Es. **83.** **(2 punti)** Un selettore con **undici** classi e un selettore con **un** id selezionano lo stesso elemento e impostano proprietà in conflitto. Quale prevale?

- **A** Quello con l'id, perché la specificità si confronta per componenti.
- B Quello con le undici classi, perché 11 è maggiore di 1.
- C Il più recente nell'ordine del foglio di stile, perché la specificità è pari.
- D Nessuno dei due: in caso di conflitto si applica il valore predefinito.
- E Quello con le undici classi, ma solo se è scritto dopo.
- F Quello con l'id, ma solo se è marcato `!important`.

Risposta: A. La specificità è una terna (id, classi, tipi) confrontata da sinistra: (1, 0, 0) batte (0, 11, 0). Non è un numero a tre cifre in cui le classi «riportano» negli id. *(Dispensa: cap. 3, «Specificità»)*

Es. **84.** **(2 punti)** Quali delle seguenti proprietà vengono **ereditate** per impostazione predefinita dagli elementi figli?

- **A** `color`
- **B** `font-family`
- **C** `font-size`
- **D** `text-align`
- E `border`
- F `margin`
- G `padding`
- H `background-color`
- I `width`

Risposte: A, B, C, D. Si ereditano soprattutto le proprietà legate al testo (colore, font, allineamento); quelle del riquadro (bordi, margini, padding, sfondo, dimensioni) no. Con il valore `inherit` si può forzare l'ereditarietà di qualunque proprietà. *(Dispensa: cap. 3, «Ereditarietà»)*

RACCOLTA

Es. **85.** **(2 punti)** Una pagina senza alcun foglio di stile mostra comunque i titoli `h1` grandi e in grassetto e i link blu e sottolineati. Perché?

- [A] Per gli stili predefiniti del browser (*user agent styles*).
- [B] Perché HTML stabilisce l'aspetto di ogni elemento, senza bisogno di CSS.
- [C] Perché il server aggiunge un foglio di stile predefinito alla risposta.
- [D] Perché il sistema operativo applica il proprio tema alle pagine web.
- [E] Per gli stili inline che il browser aggiunge al codice HTML.
- [F] Perché i tag `h1` e `a` contengono già gli attributi di stile.

Risposta: A. Ogni browser applica un proprio foglio di stile di base (origine *user agent*), con la priorità più bassa fra le dichiarazioni normali. Proprio perché differisce fra browser, molti progetti partono da un «CSS reset». *(Dispensa: cap. 3, «Gli stili predefiniti del browser»)*

Es. **86.** **(5 punti)** Quali sono le possibili origini di regole CSS nello step di analisi dei bucket di origine-importanza nell'algoritmo *cascade* di risoluzione dei conflitti?

Traccia della risposta.

- Tre origini: stili dello *user agent* (il browser), stili locali dell'utente (estensioni, impostazioni di accessibilità), stili dell'autore della pagina.
- Combinate con l'importanza (`!important` o no) danno sei bucket.
- Dal meno al più prioritario: user agent, utente, autore, autore `!important`, utente `!important`, user agent `!important`.
- L'inversione fra le `!important` protegge le esigenze dell'utente (accessibilità); a parità di bucket decidono livelli (stili inline in cima), specificità e ordine.

(Dispensa: cap. 3, «Origine e importanza») — Fonte: prima prova intercorso del 23 aprile 2024.

Es. **87.** **(5 punti)** Si spieghi come si calcola e come si confronta la **specificità** di un selettore CSS, con almeno tre esempi, e in quale punto dell'algoritmo della cascata interviene.

Traccia della risposta.

- Terna (a, b, c): a = id, b = classi, attributi e pseudo-classi, c = tipi di elemento e pseudo-elementi; `*` e combinatori non contano.
- Si confronta componente per componente da sinistra: non è un numero a tre cifre.
- Esempi: `h1` (0, 0, 1), `.nota p` (0, 1, 1), `#menu a:hover` (1, 1, 1).
- Interviene dopo origine/importanza e livelli (lo stile inline vince sui fogli di stile); a parità di specificità vince la dichiarazione che compare per ultima.

(Dispensa: cap. 3, «Specificità»)

Es. **88.** **(5 punti)** Si spieghi che cos'è l'**ereditarietà** in CSS: quali proprietà si ereditano, che priorità ha rispetto alla cascata e come la si può controllare.

Traccia della risposta.

- Alcune proprietà, se non impostate su un elemento, prendono il valore calcolato del genitore.
- Si ereditano soprattutto le proprietà del testo (`color`, `font-*`, `text-align`), non quelle del riquadro (`margin`, `border`, `background`).
- Ha la priorità più bassa in assoluto: qualunque regola che selezioni l'elemento batte il valore ereditato, anche se il genitore ha selettori più specifici o `!important`.
- Si controlla con le parole chiave `inherit` (forza l'ereditarietà) e `initial` (valore iniziale).

(Dispensa: cap. 3, «Ereditarietà»)

Es. **89.** **(5 punti)** Si descrivano i tre modi per applicare regole CSS a un documento HTML, discutendone vantaggi e svantaggi.

Traccia della risposta.

- Foglio esterno con `<link rel="stylesheet" href="...">`: separa contenuto e stile, è condiviso fra le pagine e messo in cache; richiede una richiesta in più.
- Elemento `<style>` nella pagina: utile per stili di una sola pagina, ma non riutilizzabile.
- Attributo `style`: vale per un solo elemento, mescola contenuto e presentazione ed è difficile da mantenere; ha la priorità più alta fra le dichiarazioni normali.
- Preferibile il foglio esterno.

(Dispensa: cap. 3, «Come includere il CSS in una pagina»)

Es. **90.** **(5 punti)** Si spieghi la differenza fra **pseudo-classi** e **pseudo-elementi**, con due esempi per ciascuno.

Traccia della risposta.

- Pseudo-classe (`:`): seleziona elementi in un certo stato o posizione, come se avessero una classe in più.
- Esempi: `a:hover`, `input:focus`, `li:nth-child(2n)`, `input:invalid`.
- Pseudo-elemento (`::`): seleziona una parte di un elemento o crea contenuto generato.
- Esempi: `p::first-line`, `p::first-letter`, `.nota::before` con `content`, `::selection`; il contenuto generato non fa parte del DOM.

(Dispensa: cap. 3, «Pseudo-classi» e «Pseudo-elementi»)

Capitolo 4 — Layout e Responsive Design

Es. **91.** **(2 punti)** Si consideri la regola seguente, applicata a un `div` di classe `box`. Quanto è largo il **content box** dell'elemento?

```
.box {
  box-sizing: border-box;
  width: 200px;
  padding: 10px;
  border: 2px solid black;
  margin: 20px;
}
```

- **A** 176px
- B 212px
- C 264px
- D 220px
- E 240px
- F 200px

Risposta: A. Con `box-sizing: content-box` (il valore predefinito) `width` indica la larghezza del solo contenuto: padding e bordo si aggiungono ($200 + 2{\cdot}10 + 2{\cdot}2 = 224$). Con `border-box` `width` arriva fino al bordo e il contenuto si restringe ($200 - 2{\cdot}10 - 2{\cdot}2 = 176$). I margini stanno sempre fuori dal bordo. *(Dispensa: cap. 4, «Il box model»)*

Es. **92.** **(2 punti)** Si considerino le regole CSS e il frammento HTML seguenti. Qual è la dimensione del carattere calcolata per lo `span` di classe `c`?

```
.a { font-size: 12px; }
.b { font-size: 1.5em; }
.c { font-size: 2rem; }
```

```
<div class="a">A <div class="b">B <span class="c">C</span></div></div>
```

- **A** 32px
- B 48px
- C 24px
- D 36px
- E 18px
- F 12px

Risposta: A. `em` (e `%` in `font-size`) si riferiscono alla dimensione del carattere del **genitore**, `rem` a quella dell'elemento radice (16px, il valore predefinito). Qui `.b` vale $1.5{\cdot}12 = 18$px, e `.c` (2rem) vale 32px. *(Dispensa: cap. 4, «Unità relative al carattere»)*

Es. **93.** **(2 punti)** Con la regola seguente, quanto sono larghe le tre colonne della griglia?

```
.griglia {
  display: grid;
  width: 700px;
  grid-template-columns: 150px 1fr 2fr;
}
```

- **A** 150px, 183.333px e 366.667px
- B 150px, 233.33px e 466.67px
- C 150px, 275px e 275px
- D 233.33px, 233.33px e 233.33px
- E 150px, 100px e 200px
- F 150px, 366.67px e 183.33px

Risposta: A. L'unità `fr` divide lo spazio **rimasto** dopo le colonne a dimensione fissa: $700 - 150 = 550$px, ripartiti in 3 parti (183.333px ciascuna). *(Dispensa: cap. 4, «L'unità fr»)*

RACCOLTA

Es. **94.** **(2 punti)** Un paragrafo è stilizzato con le regole seguenti. Di che colore appare in una finestra con viewport largo **380px**?

```
p { color: red; }
@media (min-width: 480px) {
  p { color: black; }
}
@media (min-width: 1200px) {
  p { color: orange; }
}
```

- **A** red
- B black
- C orange
- D gray

Risposta: A. `min-width: X` vale quando il viewport è largo **almeno** X pixel, `max-width: X` quando è largo **al più** X (gli estremi sono inclusi). Le regole dentro una media query hanno la stessa specificità delle altre: fra quelle che valgono, vince l'ultima nell'ordine. *(Dispensa: cap. 4, «Media query»)*

Es. **95.** **(2 punti)** Quale fra le seguenti affermazioni sul pixel CSS (`px`) è corretta?

- **A** È un'unità di riferimento: su schermi ad alta densità corrisponde a più pixel fisici.
- B Corrisponde sempre esattamente a un pixel fisico dello schermo.
- C È un'unità relativa alla dimensione del carattere dell'elemento.
- D Equivale all'1% della larghezza del viewport.
- E Non si può usare per le dimensioni dei caratteri.
- F Vale un millimetro sullo schermo, qualunque sia il dispositivo.

Risposta: A. Il `px` CSS è un'unità «logica»: su uno schermo con rapporto di pixel 2 un pixel CSS occupa 2×2 pixel fisici. Per questo le pagine hanno dimensioni simili su schermi di densità diversa. *(Dispensa: cap. 4, «Unità assolute»)*

Es. **96.** **(2 punti)** Che cosa indica `50vw`?

- **A** Metà della larghezza del viewport.
- B Metà della larghezza dell'elemento genitore.
- C Metà della larghezza dello schermo fisico del dispositivo.
- D 50 volte la dimensione del carattere corrente.
- E Metà della larghezza del documento, compresa la parte non visibile.
- F 50 pixel, su qualunque dispositivo.

Risposta: A. `vw` e `vh` valgono l'1% della larghezza e dell'altezza del viewport (l'area della finestra in cui si vede la pagina). Le percentuali **%** si riferiscono invece, di solito, al blocco contenitore. *(Dispensa: cap. 4, «Unità relative al viewport»)*

Es. **97.** **(2 punti)** Quale fra le seguenti affermazioni su `padding` e `margin` è corretta?

- **A** Il padding sta dentro il bordo e prende lo sfondo; il margine sta fuori, trasparente.
- B Il margine sta dentro il bordo e prende lo sfondo; il padding sta fuori, trasparente.
- C Sono sinonimi: l'uno o l'altro si sceglie in base al browser.
- D Il padding si può applicare solo agli elementi in linea.
- E Il margine è sempre incluso nella larghezza indicata da `width`.
- F Il padding non può avere valori diversi sui quattro lati.

Risposta: A. Dall'interno verso l'esterno: contenuto, padding, bordo, margine. Lo sfondo dell'elemento si estende sotto contenuto e padding (fino al bordo); il margine è spazio vuoto che separa l'elemento dagli altri. *(Dispensa: cap. 4, «Il box model»)*

RACCOLTA

Es. **98.** **(2 punti)** Che differenza c'è fra `display: none` e `visibility: hidden`?

- **A** Con `display: none` l'elemento non occupa spazio; con `visibility: hidden` sì.
- B Con `visibility: hidden` l'elemento non occupa spazio; con `display: none` sì.
- C `display: none` rimuove l'elemento dal DOM, `visibility: hidden` no.
- D Nessuna: sono due modi equivalenti di nascondere un elemento.
- E `visibility: hidden` nasconde anche il contenuto dei figli, `display: none` no.
- F `display: none` vale solo per gli elementi di blocco.

Risposta: A. Entrambi nascondono l'elemento, che resta nel DOM. Con `display: none` non viene generato alcun riquadro, quindi gli altri elementi ne occupano il posto; con `visibility: hidden` il riquadro c'è ma è invisibile, e lo spazio resta vuoto. *(Dispensa: cap. 4, «La proprietà display»)*

Es. **99.** **(2 punti)** Quali delle seguenti affermazioni sugli elementi con `display: inline` (come `<span>` o `<a>`) sono **vere**?

- **A** Non vanno a capo: si dispongono uno dopo l'altro sulla riga.
- **B** Ignorano `width` e `height`.
- C Occupano tutta la larghezza disponibile del contenitore.
- D Iniziano sempre su una nuova riga.
- E Non possono contenere testo.
- F Non possono avere padding né bordi.
- G Diventano elementi di blocco se hanno un `id`.

Risposte: A, B. Gli elementi in linea scorrono nel testo e la loro dimensione dipende dal contenuto: `width` e `height` non hanno effetto. `inline-block` li lascia sulla riga ma accetta le dimensioni; `block` li fa andare a capo e occupare tutta la larghezza. *(Dispensa: cap. 4, «La proprietà display»)*

Es. **100.** **(2 punti)** A che cosa serve la proprietà `clear: both`?

- **A** A far scendere l'elemento sotto gli elementi flottanti che lo precedono.
- B A rimuovere tutti gli stili applicati all'elemento.
- C A far flottare l'elemento sia a destra sia a sinistra.
- D A centrare l'elemento orizzontalmente nel contenitore.
- E A nascondere gli elementi flottanti che precedono l'elemento.
- F A eliminare margini e padding dell'elemento.

Risposta: A. `float` toglie un elemento dal flusso normale e lo sposta a sinistra o a destra, con il testo che gli scorre attorno. `clear` impedisce a un elemento di affiancarsi ai float precedenti: con `both` scende sotto quelli di entrambi i lati. *(Dispensa: cap. 4, «Float e clear»)*

Es. **101.** **(2 punti)** Un elemento ha `position: relative; top: 20px; left: 10px`. Che cosa succede?

- **A** Si sposta rispetto alla sua posizione normale e il suo spazio originale resta occupato.
- B Si sposta rispetto all'angolo del viewport e non occupa più spazio.
- C Si sposta rispetto al primo antenato posizionato e lascia libero il suo spazio.
- D Resta fermo: `top` e `left` non hanno effetto con `position: relative`.
- E Si sposta di 20px verso l'alto e di 10px verso destra.
- F Resta fisso nello schermo anche quando si scorre la pagina.

Risposta: A. Con `relative` l'elemento resta nel flusso: gli altri elementi si dispongono come se fosse nella posizione normale, e lui viene solo disegnato spostato (qui 20px più in basso e 10px più a destra). *(Dispensa: cap. 4, «relative»)*

RACCOLTA

Es. **102.** **(2 punti)** Rispetto a quale elemento viene posizionato il `div` di classe `badge`?

```html
<div class="card" style="position: relative;">
  <div class="wrap">
    <div class="badge" style="position: absolute; top: 0; right: 0;">
      Nuovo
    </div>
  </div>
</div>
```

- **A** Rispetto al `div` di classe `card`, il primo antenato posizionato.
- B Rispetto al `div` di classe `wrap`, il genitore diretto.
- C Rispetto al `body`, perché `absolute` usa sempre il documento.
- D Rispetto al viewport, perché `absolute` equivale a `fixed`.
- E Rispetto alla sua posizione normale nel flusso.
- F Rispetto al primo antenato con una larghezza impostata.

Risposta: A. Un elemento con `position: absolute` esce dal flusso e si posiziona rispetto al **primo antenato posizionato** (con `position` diversa da `static`): qui la `card`, perché la `wrap` non è posizionata. Se non c'è alcun antenato posizionato si usa il blocco iniziale (la pagina). *(Dispensa: cap. 4, «absolute»)*

Es. **103.** **(2 punti)** Quali delle seguenti affermazioni sono **vere**?

- **A** Un elemento `fixed` resta fermo nel viewport quando si scorre la pagina.
- **B** Un elemento `sticky` scorre con la pagina finché non raggiunge la soglia indicata (es. `top: 0`), poi vi resta attaccato.
- **C** Un elemento `absolute` non occupa spazio nel flusso del documento.
- D Un elemento `fixed` si posiziona rispetto al suo genitore diretto.
- E Un elemento `sticky` esce dal flusso come un elemento `absolute`.
- F `static` è il valore che rende efficaci `top` e `left`.
- G Un elemento `relative` non occupa più spazio nel flusso.
- H `fixed` e `sticky` funzionano solo sugli elementi in linea.

Risposte: A, B, C. `static` è il valore predefinito (nessun posizionamento). `relative` sposta senza togliere dal flusso; `absolute` toglie dal flusso e usa l'antenato posizionato; `fixed` usa il viewport; `sticky` si comporta come `relative` finché non raggiunge la soglia, poi come `fixed` dentro il contenitore. *(Dispensa: cap. 4, «Posizionamento»)*

Es. **104.** **(2 punti)** In un contenitore con `display: flex` e `flex-direction: column`, quale proprietà allinea gli elementi lungo l'asse **verticale**?

- **A** `justify-content`
- B `align-items`
- C `vertical-align`
- D `align-content`
- E `text-align`
- F `flex-wrap`

Risposta: A. `justify-content` agisce sull'asse principale, `align-items` su quello trasversale. Con `flex-direction: column` l'asse principale diventa verticale, quindi è `justify-content` a distribuire gli elementi dall'alto in basso. *(Dispensa: cap. 4, «justify-content» e «align-content e align-items»)*

Es. **105.** **(2 punti)** Quali elementi diventano *flex item* con la regola `.menu { display: flex; }`?

- **A** I figli diretti dell'elemento di classe `menu`.
- B Tutti i discendenti dell'elemento di classe `menu`.
- C L'elemento di classe `menu` stesso.
- D Gli elementi fratelli dell'elemento di classe `menu`.
- E Solo i figli che hanno a loro volta `display: flex`.
- F Solo gli elementi in linea contenuti nel menu.

Risposta: A. `display: flex` rende l'elemento un contenitore flex e i suoi **figli diretti** flex item. I discendenti più interni seguono il normale flusso dentro il proprio flex item. *(Dispensa: cap. 4, «Contenitori flex»)*

RACCOLTA

Es. **106.** **(2 punti)** Come si distribuiscono tre elementi in un contenitore flex in riga con `justify-content: space-between`?

- **A** Il primo all'inizio, l'ultimo alla fine, lo spazio libero diviso fra loro.
- B Tutti e tre al centro, uno accanto all'altro.
- C Con lo stesso spazio prima del primo, fra gli elementi e dopo l'ultimo.
- D Tutti e tre all'inizio, con lo spazio libero dopo l'ultimo.
- E Uno sotto l'altro, distanziati in modo uniforme.
- F Allargati fino a occupare tutto lo spazio disponibile.

Risposta: A. `space-between` mette il primo e l'ultimo elemento ai bordi e divide lo spazio rimanente in parti uguali **fra** gli elementi; `space-around` e `space-evenly` lasciano spazio anche ai lati; `center` li raggruppa al centro. *(Dispensa: cap. 4, «justify-content»)*

Es. **107.** **(2 punti)** Che cosa fa la proprietà `grid-template-areas` di un contenitore Grid?

- **A** Dà un nome alle aree della griglia, che i figli occupano con `grid-area`.
- B Stabilisce quante righe deve avere la griglia, in base al contenuto.
- C Imposta lo spazio fra le righe e le colonne della griglia.
- D Assegna automaticamente un nome di classe a ogni cella.
- E Disegna i bordi delle aree della griglia.
- F Converte i figli in flex item disposti in colonna.

Risposta: A. Con `grid-template-areas` si disegna la griglia «a parole» (es. `"header header" "nav main"`), poi ogni figlio si piazza in un'area con `grid-area: main`. Rende il layout leggibile e facile da cambiare con le media query. *(Dispensa: cap. 4, «Aree con nome»)*

Es. **108.** **(2 punti)** Che effetto ha l'assenza di `<meta name="viewport" content="width=device-width, initial-scale=1">` in una pagina responsive vista su uno smartphone?

- **A** Il browser impagina la pagina su un viewport largo come un desktop e la rimpicciolisce.
- B La pagina non viene visualizzata e il browser mostra un errore.
- C Le media query vengono ignorate solo sui browser desktop.
- D Il browser carica automaticamente la versione mobile del sito.
- E Le immagini vengono scaricate in bassa risoluzione.
- F Nessuno: il tag serve solo ai motori di ricerca.

Risposta: A. Senza il meta tag i browser mobili simulano uno schermo largo circa 980px e poi riducono tutto: le media query pensate per gli schermi stretti non scattano e il testo risulta minuscolo. `width=device-width` fa coincidere il viewport con la larghezza reale del dispositivo. *(Dispensa: cap. 4, «Il meta tag viewport»)*

Es. **109.** **(2 punti)** Che cosa distingue un layout **responsive** da uno **adattivo**?

- **A** Il responsive si adatta con continuità a ogni larghezza; l'adattivo usa alcune larghezze fisse.
- B L'adattivo si adatta con continuità; il responsive usa alcune larghezze fisse.
- C Il responsive richiede un sito separato per i dispositivi mobili.
- D L'adattivo funziona solo con JavaScript.
- E Il responsive usa solo unità assolute come `px`.
- F Nessuna: sono due nomi della stessa tecnica.

Risposta: A. Un layout adattivo prevede alcuni layout a larghezza fissa e sceglie il più adatto; un layout responsive combina dimensioni fluide (percentuali, `fr`, Flexbox) e media query per adattarsi a qualunque larghezza. Esistono poi i layout fissi, quelli liquidi e i siti mobili separati. *(Dispensa: cap. 4, «Responsive design»)*

RACCOLTA

Es. **110.** **(2 punti)** Un `div` alto 100px contiene un testo che richiederebbe 300px. Quale valore di `overflow` mostra una barra di scorrimento **solo se serve**?

- [A] `auto`
- [B] `visible`
- [C] `hidden`
- [D] `scroll`
- [E] `clip-auto`
- [F] `wrap`

Risposta: A. `visible` (predefinito) lascia uscire il contenuto dal riquadro, `hidden` lo taglia, `scroll` mostra sempre le barre, `auto` le mostra solo quando il contenuto non ci sta. *(Dispensa: cap. 4, «Il box model»)*

Es. **111.** **(5 punti)** Si descriva il **box model** del CSS e si spieghi l'effetto della proprietà `box-sizing`, con un esempio numerico.

Traccia della risposta.

- Ogni elemento è un riquadro: contenuto, padding, bordo, margine (dall'interno).
- Il padding prende lo sfondo dell'elemento, il margine è spazio trasparente esterno.
- Con `content-box` (predefinito) `width` è la larghezza del contenuto: `width: 200px; padding: 20px; border: 5px` occupa 250px fino al bordo.
- Con `border-box` `width` arriva fino al bordo: gli stessi valori danno 200px, con un contenuto di 150px. Spesso si imposta `* { box-sizing: border-box; }`.

(Dispensa: cap. 4, «Il box model»)

Es. **112.** **(5 punti)** Si confrontino **Flexbox** e **Grid**: a che cosa servono, quali sono i concetti principali di ciascuno e quando conviene usare l'uno o l'altro.

Traccia della risposta.

- Flexbox dispone gli elementi lungo **una** dimensione (riga o colonna): asse principale e trasversale, `justify-content`, `align-items`, `flex-wrap`.
- Grid dispone gli elementi in **due** dimensioni: `grid-template-columns`/`rows`, unità `fr`, aree con nome (`grid-template-areas`, `grid-area`).
- In entrambi i casi gli stili si applicano al contenitore e i figli diretti diventano item.
- Flexbox per barre, menu, allineamenti di componenti; Grid per l'impaginazione complessiva della pagina. Si possono combinare.

(Dispensa: cap. 4, «Flexbox» e «Grid»)

Es. **113.** **(5 punti)** Che cos'è il **responsive design**? Si descrivano gli strumenti con cui si realizza (media query, unità relative, meta tag viewport) con un breve esempio.

Traccia della risposta.

- Un'unica pagina che si adatta a schermi di ogni dimensione, invece di un sito mobile separato o di layout fissi.
- Media query: `@media (max-width: 600px) { ... }` applica regole solo in certe condizioni (larghezza, orientamento, ...); approccio *mobile first* con `min-width`.
- Dimensioni fluide: percentuali, `fr`, Flexbox e Grid, unità relative come `rem`.
- Meta tag `<meta name="viewport" content="width=device-width, initial-scale=1">`: senza, i browser mobili simulano uno schermo da desktop e rimpiccioliscono la pagina.

(Dispensa: cap. 4, «Responsive design»)

Es. **114.** **(5 punti)** Si descrivano i valori della proprietà `position` (`static`, `relative`, `absolute`, `fixed`, `sticky`), indicando per ciascuno rispetto a che cosa si posiziona l'elemento e se resta nel flusso.

Traccia della risposta.

- `static`: valore predefinito, nel flusso; `top`/`left` non hanno effetto.
- `relative`: spostato rispetto alla posizione normale, il suo spazio resta occupato.
- `absolute`: fuori dal flusso, rispetto al primo antenato posizionato (o alla pagina).
- `fixed`: fuori dal flusso, rispetto al viewport (resta fermo durante lo scorrimento).
- `sticky`: come `relative` finché non raggiunge la soglia, poi resta attaccato.

(Dispensa: cap. 4, «Posizionamento»)

Capitolo 5 — JavaScript

Es. **115.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
1 console.log(typeof 'hello');
2 console.log(typeof ({ a: 1 }));
3 console.log(typeof typeof 42);
```

- **A** `string`, `object`, `string`.
- B `string`, `bigint`, `object`.
- C `string`, `object`, `number`.
- D `symbol`, `object`, `string`.
- E `string`, `NaN`, `string`.
- F `string`, `boolean`, `string`.

Risposta: A. `typeof` restituisce una stringa: i tipi primitivi `number` (anche `NaN` e `Infinity`), `string`, `boolean`, `undefined`, `bigint`, `symbol`; per gli oggetti (array e `null` compresi, una storica stranezza) `object`, e `function` per le funzioni. *(Dispensa: cap. 5, «Tipi di dato primitivi»)*

Es. **116.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
1 console.log(true === 1, true == "1", "1" === 1);
```

- **A** `false true false`.
- B `true true true`.
- C `false false false`.
- D `true true false`.
- E `false true true`.
- F `false false true`.

Risposta: A. `===` confronta valore **e** tipo: con tipi diversi è sempre `false`. `==` invece converte gli operandi: stringhe e booleani diventano numeri (`"1" == 1`, `true == 1`, `0 == false` sono veri; `true == 42` è falso perché `true` diventa 1). `null == undefined` è vero, mentre `NaN` non è uguale a niente, nemmeno a se stesso. Per evitare sorprese si usa sempre `===`. *(Dispensa: cap. 5, «Uguaglianza debole e uguaglianza stretta»)*

Es. **117.** **(2 punti)** Si consideri il seguente frammento di codice JavaScript (si assuma la presenza di `"use strict";`).

```
1 console.log(x);
2 if (true) {
3   console.log(x);
4   let x;
5   console.log(x); }
```

Con riferimento al ciclo di vita di una variabile dichiarata con `let`, indicare quali delle seguenti affermazioni sono **vere**.

- **A** In Linea 3, `x` è dichiarata ma non inizializzata.
- **B** In Linea 5, `x` ha valore `undefined`.
- C In Linea 3, `x` è inizializzata a `undefined`.
- D In Linea 5, `x` ha valore `null`.
- E Il codice esegue senza sollevare errori.
- F In Linea 1, `x` è inizializzata ma non dichiarata.

Risposte: A, B. Una variabile `let` viene dichiarata (sollevata) all'inizio del suo blocco, ma resta non inizializzata fino alla riga della dichiarazione: dalla linea 2 alla 4 è nella *temporal dead zone*, e usarla solleva un `ReferenceError`. Con `let x;` viene inizializzata a `undefined`. In Linea 1, fuori dal blocco, `x` non esiste affatto: l'esecuzione si ferma proprio lì con un `ReferenceError`. *(Dispensa: cap. 5, «Il ciclo di vita di una variabile») — Fonte: prima prova intercorso del 23 aprile 2024.*

Es. **118.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
1 try { console.log("A"); x = 1; console.log(x); }
2 catch(e) { console.log("X"); } finally { console.log("Z"); }
```

- **A** `A`, `X`, `Z`.
- B Viene sollevato un `ReferenceError`.
- C `X`, `Z`.
- D `A`, `1`, `Z`.
- E `A`, `1`, `X`, `Z`.
- F `A`, poi viene sollevato un `ReferenceError`.

Risposta: A. In modalità strict assegnare a una variabile mai dichiarata solleva un `ReferenceError` (senza strict creerebbe una variabile globale). L'errore viene catturato dal `catch`, e il blocco `finally` viene eseguito in ogni caso. *(Dispensa: cap. 5, «Prima di ECMAScript 6: var e dichiarazioni implicite») — Fonte: prima prova intercorso del 23 aprile 2024.*

FACCIATA

Es. **119.** **(2 punti)** Quali dei seguenti **non** sono tipi di dato primitivi in JavaScript?

- **A** `array`
- **B** `Promise`
- **C** `float`
- **D** `object`
- **E** `integer`
- F `boolean`
- G `number`
- H `string`
- I `undefined`
- J `bigint`
- K `symbol`

Risposte: A, B, C, D, E. I tipi primitivi sono `number`, `string`, `boolean`, `undefined`, `null`, `bigint` e `symbol`. Esiste un solo tipo numerico (`number`, a virgola mobile): `float` e `integer` non esistono. Array, Promise e funzioni sono oggetti. *(Dispensa: cap. 5, «Tipi di dato primitivi»)* — *Fonte: prima prova intercorso del 23 aprile 2024.*

Es. **120.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
1 if (true) {
2   var a = 1;
3   let b = 2;
4 }
5 console.log(typeof a, typeof b);
```

- **A** `number undefined`.
- B `number number`.
- C `undefined undefined`.
- D `undefined number`.
- E Viene sollevato un `ReferenceError`.
- F `number`, poi viene sollevato un `ReferenceError`.

Risposta: A. `var` ha visibilità di funzione (qui globale): `a` esiste anche fuori dal blocco. `let` ha visibilità di blocco: fuori dall'`if` `b` non esiste. `typeof` applicato a un nome non dichiarato restituisce `"undefined"` invece di sollevare un errore. *(Dispensa: cap. 5, «Gli scope»)*

Es. **121.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
1 console.log(n);
2 var n = 5;
3 console.log(n);
```

- **A** `undefined`, 5.
- B Viene sollevato un `ReferenceError`.
- C 5, 5.
- D Viene sollevato un `TypeError`.
- E `null`, 5.
- F `undefined`, poi viene sollevato un `ReferenceError`.

Risposta: A. Le dichiarazioni `var` vengono sollevate (*hoisting*) all'inizio dello scope e inizializzate subito a `undefined`; l'assegnamento resta al suo posto. Con `let` la prima riga solleverebbe invece un `ReferenceError` (TDZ). *(Dispensa: cap. 5, «Hoisting»)*

Es. **122.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
1 let x = "esterna";
2 function f() {
3   console.log(x);
4   let x = "interna";
5 }
6 f();
```

- **A** Viene sollevato un `ReferenceError`.
- B `esterna`.
- C `undefined`.
- D `interna`.
- E `esterna`, `interna`.

Risposta: A. Dentro `f` la dichiarazione `let x` viene sollevata all'inizio della funzione e **nasconde** la `x` esterna, ma fino alla sua riga è nella *temporal dead zone*: leggerla solleva un `ReferenceError`, non stampa la variabile esterna. *(Dispensa: cap. 5, «Hoisting»)*

RACCOLTA

Es. **123.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
1 const pet = { nome: "Fido" };
2 pet.nome = "Rex";
3 console.log(pet.nome);
4 pet = { nome: "Fufi" };
5 console.log(pet.nome);
```

- [A] `Rex`, poi viene sollevato un `TypeError`.
- [B] `Rex`, `Fufi`.
- [C] Viene sollevato un `TypeError`.
- [D] `Fido`, `Fufi`.
- [E] `Rex`, `Rex`.
- [F] `Fido`, poi viene sollevato un `TypeError`.

Risposta: A. `const` impedisce di **riassegnare la variabile**, non di modificare l'oggetto a cui si riferisce: cambiare `pet.nome` è lecito, assegnare a `pet` un altro oggetto solleva un `TypeError` (*Assignment to constant variable*). *(Dispensa: cap. 5, «let e const»)*

Es. **124.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
1 function risposta() {
2   return
3     42;
4 }
5 console.log(risposta());
```

- [A] `undefined`.
- [B] `42`.
- [C] Viene sollevato un `SyntaxError`.
- [D] `null`.
- [E] `NaN`.

Risposta: A. Per l'inserimento automatico dei punti e virgola, un `return` seguito da un a capo diventa `return;`: la funzione restituisce `undefined` e `42;` è codice irraggiungibile. Il valore va scritto sulla stessa riga del `return`. *(Dispensa: cap. 5, «Il linguaggio»)*

Es. **125.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
1 function saluta(nome = "ospite") {
2   console.log("Ciao " + nome);
3 }
4 saluta();
5 saluta(undefined);
6 saluta(null);
```

- [A] `Ciao ospite`, `Ciao ospite`, `Ciao null`.
- [B] `Ciao ospite`, `Ciao undefined`, `Ciao null`.
- [C] `Ciao ospite`, `Ciao ospite`, `Ciao ospite`.
- [D] `Ciao undefined`, `Ciao undefined`, `Ciao null`.
- [E] `Ciao ospite`, `Ciao undefined`, `Ciao ospite`.

Risposta: A. Il valore predefinito di un parametro si usa quando l'argomento manca o vale `undefined`; `null` è un valore a tutti gli effetti, quindi viene usato così com'è. *(Dispensa: cap. 5, «Argomenti predefiniti»)*

RACCOLTA

Es. **126.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
1 console.log(quadrato(3));
2 console.log(cubo(2));
3 function quadrato(n) { return n * n; }
4 var cubo = function (n) { return n * n * n; };
```

- **A** 9, poi viene sollevato un `TypeError`.
- B 9, 8.
- C 9, poi viene sollevato un `ReferenceError`.
- D Viene sollevato un `TypeError`.
- E 9, `undefined`.
- F Viene sollevato un `ReferenceError`.

Risposta: A. Le dichiarazioni di funzione vengono sollevate per intero: `quadrato` si può chiamare prima della sua definizione. `cubo` invece è una variabile `var` a cui viene assegnata un'espressione di funzione: durante l'hoisting vale `undefined`, e chiamarla solleva un `TypeError` (*cubo is not a function*). Con `let` o `const` si avrebbe un `ReferenceError`. *(Dispensa: cap. 5, «Hoisting delle funzioni»)*

Es. **127.** **(2 punti)** Qual è l'output dell'esecuzione del seguente frammento di codice JavaScript (si assuma la presenza di `"use strict";`)?

```
1 foo();
2 function foo() { console.log("Hello!"); }
3 foo();
```

- **A** `Hello!`, `Hello!`.
- B `undefined`, `Hello!`, `Hello!`.
- C Viene sollevato un `ReferenceError`.
- D `Hello!`, `Hello!`, `Hello!`.

Risposta: A. Le dichiarazioni di funzione vengono sollevate per intero (nome e corpo) all'inizio dello scope: la prima chiamata, che precede la definizione, funziona già. La definizione da sola non esegue nulla: le chiamate sono due, quindi `Hello!` compare due volte. Non compare `undefined`, perché il valore restituito dalle chiamate non viene stampato. *(Dispensa: cap. 5, «Hoisting delle funzioni») — Fonte: facsimile mostrato in aula.*

Es. **128.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
1 function contatore() {
2   let n = 0;
3   return () => ++n;
4 }
5 const a = contatore();
6 const b = contatore();
7 a(); a();
8 console.log(a(), b());
```

- **A** 3 1.
- B 3 3.
- C 1 1.
- D 2 1.
- E 3 4.
- F Viene sollevato un `ReferenceError`.

Risposta: A. Ogni chiamata di `contatore` crea un nuovo ambiente con la propria variabile `n`, e la funzione restituita lo ricorda (*closure*). `a` e `b` hanno contatori indipendenti: `a` arriva a 3, `b` a 1. *(Dispensa: cap. 5, «Funzioni annidate restituite»)*

RACCOLTA

Es. **129.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
let a = { x: 1, dati: { y: 1 } };
let b = a;
let c = { ...a };
b.x = 2;
c.dati.y = 3;
console.log(a.x, a.dati.y);
```

- **A** 2 3.
- B 1 1.
- C 2 1.
- D 1 3.
- E 2 undefined.
- F Viene sollevato un TypeError.

Risposta: A. Gli oggetti si assegnano **per riferimento**: `b` è lo stesso oggetto di `a`. La sintassi `{ ...a }` crea una copia **superficiale**: `c.x` è indipendente, ma `c.dati` punta allo stesso oggetto interno di `a.dati`. *(Dispensa: cap. 5, «Clonare un oggetto»)*

Es. **130.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
const pet = {
  nome: "Fido",
  saluta() { console.log("Ciao, sono " + this.nome); }
};
pet.saluta();
const f = pet.saluta;
f();
```

- **A** Ciao, sono Fido, poi viene sollevato un TypeError.
- B Ciao, sono Fido, Ciao, sono Fido.
- C Ciao, sono Fido, Ciao, sono undefined.
- D Viene sollevato un TypeError.
- E Ciao, sono Fido, Ciao, sono .

Risposta: A. `this` dipende da **come** si chiama la funzione: in `pet.saluta()` vale `pet`. `f` è la stessa funzione chiamata da sola: in modalità strict `this` vale `undefined`, e leggere `this.nome` solleva un `TypeError`. *(Dispensa: cap. 5, «La parola chiave this»)*

Es. **131.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
function Timer() {
  this.secondi = 0;
  this.avanza = () => { this.secondi++; };
}
const t = new Timer();
const avanza = t.avanza;
avanza(); avanza();
console.log(t.secondi);
```

- **A** 2.
- B 0.
- C Viene sollevato un TypeError.
- D NaN.
- E undefined.

Risposta: A. Le arrow function non hanno un proprio `this`: usano quello dello scope in cui sono definite, qui l'oggetto creato da `new Timer()`. Per questo `avanza` funziona anche chiamata da sola. Con una funzione normale `this` sarebbe `undefined` e si avrebbe un `TypeError`. *(Dispensa: cap. 5, «La parola chiave this»)*

RACCOLTA

Es. **132.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
function Persona(nome) {
  this.nome = nome;
}
const p = new Persona("Ada");
console.log(p.nome);
const q = Persona("Alan");
console.log(q);
```

- **A** Ada, poi viene sollevato un TypeError.
- B Ada, undefined.
- C Ada, Alan.
- D Viene sollevato un TypeError.
- E Ada, [object Object].

Risposta: A. Con `new` viene creato un nuovo oggetto e `this` si riferisce a esso. Senza `new` la funzione viene chiamata normalmente: in modalità strict `this` vale `undefined` e `this.nome = ...` solleva un `TypeError`. *(Dispensa: cap. 5, «Costruttori»)*

Es. **133.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
const pet = { nome: "Fido" };
console.log(pet.razza);
console.log(pet.padrone?.nome);
console.log(pet.padrone.nome);
```

- **A** undefined, undefined, poi viene sollevato un TypeError.
- B undefined, undefined, poi viene sollevato un ReferenceError.
- C undefined, undefined, undefined.
- D undefined, poi viene sollevato un TypeError.
- E Viene sollevato un TypeError.

Risposta: A. Leggere una proprietà inesistente restituisce `undefined`. L'optional chaining `?.` si ferma se ciò che sta a sinistra è `null` o `undefined` e restituisce `undefined`. Senza, leggere `.nome` da `undefined` solleva un `TypeError` (non un `ReferenceError`: la variabile `pet` esiste). *(Dispensa: cap. 5, «Optional chaining»)*

Es. **134.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
const conto = { saldo: 100 };
Object.defineProperty(conto, "saldo", { writable: false });
console.log(conto.saldo);
conto.saldo = 0;
console.log(conto.saldo);
```

- **A** 100, poi viene sollevato un TypeError.
- B 100, 0.
- C 100, 100.
- D Viene sollevato un TypeError.
- E 0, 0.

Risposta: A. Con `writable: false` la proprietà diventa di sola lettura. In modalità strict assegnarle un valore solleva un `TypeError`; senza strict l'assegnamento verrebbe ignorato in silenzio (e si stamperebbe di nuovo 100). *(Dispensa: cap. 5, «Configurare le proprietà»)*

RACCOLTA

Es. **135.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
const r = {
  base: 3, altezza: 4,
  get area() { return this.base * this.altezza; },
  set lato(v) { this.base = v; this.altezza = v; }
};
r.lato = 5;
console.log(r.area);
```

- **A** 25.
- B 12.
- C Viene sollevato un `TypeError`.
- D `undefined`.
- E `[Function: get area]`.

Risposta: A. Un getter si legge come una proprietà (`r.area`, senza parentesi) ma esegue la funzione; un setter si attiva con un assegnamento. `r.lato = 5` imposta base e altezza a 5, quindi l'area vale 25. *(Dispensa: cap. 5, «Getter e setter»)*

Es. **136.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
const nome = "Ada";
const anni = 36;
console.log(`${nome} ha ${anni + 1} anni`);
console.log('${nome} ha ${anni + 1} anni');
```

- **A** `Ada ha 37 anni, ${nome} ha ${anni + 1} anni.`
- B `Ada ha 37 anni, Ada ha 37 anni.`
- C `Ada ha 361 anni, ${nome} ha ${anni + 1} anni.`
- D `Ada ha 37 anni, Ada ha 361 anni.`
- E Viene sollevato un `SyntaxError`.

Risposta: A. Solo i *template literal*, delimitati dagli accenti gravi, interpretano le espressioni `${...}`; fra apici singoli o doppi il testo resta letterale. Dentro `${}` si valuta un'espressione: `anni + 1` è 37. *(Dispensa: cap. 5, «Funzioni»)*

Es. **137.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
const valori = [0, "0", "", [], null, NaN];
for (const v of valori) {
  if (v) console.log(JSON.stringify(v));
}
```

- **A** `"0"`, `[]`.
- B `"0"`.
- C 0, `"0"`, `[]`.
- D `[]`.
- E `"0"`, `""`, `[]`.
- F Non viene stampato nulla.

Risposta: A. Sono *falsy* `false`, `0`, `""`, `null`, `undefined` e `NaN`: tutto il resto è *truthy*, compresa la stringa `"0"` (non vuota) e l'array vuoto (un oggetto). *(Dispensa: cap. 5, «Operatori»)*

RACCOLTA

Es. **138.** **(2 punti)** Quali delle seguenti affermazioni su `var`, `let` e `const` sono **vere**?

- **A** `let` e `const` hanno visibilità di blocco.
- **B** `var` ha visibilità di funzione.
- **C** Una variabile `const` va inizializzata nella dichiarazione.
- **D** Le variabili `var` vengono sollevate e inizializzate a `undefined`.
- E `const` rende immutabile l'oggetto a cui la variabile si riferisce.
- F Le variabili `let` non vengono sollevate affatto.
- G Con `let` si può ridichiarare una variabile nello stesso blocco.
- H `var` ha visibilità di blocco, come `let`.
- I Leggere una variabile `let` prima della dichiarazione restituisce `undefined`.

Risposte: A, B, C, D. `let`/`const` sono visibili nel blocco e vengono sollevate ma restano non inizializzate fino alla dichiarazione (TDZ: leggerle prima solleva un `ReferenceError`). `var` è visibile nella funzione e viene inizializzata subito a `undefined`. `const` impedisce la riassegnazione, non la modifica dell'oggetto. *(Dispensa: cap. 5, «Variabili»)*

Es. **139.** **(2 punti)** Quale dei seguenti modi di includere uno script esterno è corretto?

- **A** `<script src="app.js"></script>`
- B `<script src="app.js" />`
- C `<link rel="script" href="app.js">`
- D `<script href="app.js"></script>`
- E `<js src="app.js"></js>`
- F `<script>app.js</script>`

Risposta: A. Il tag `script` non è mai auto-chiudente: anche con `src` va chiuso con `</script>`. `<script src="app.js" />` viene letto come un tag aperto, e il resto della pagina finisce dentro lo script. *(Dispensa: cap. 5, «Includere JavaScript in una pagina»)*

Es. **140.** **(2 punti)** Quali effetti ha la direttiva `"use strict";`?

- **A** Assegnare a una variabile non dichiarata solleva un `ReferenceError`.
- **B** In una funzione chiamata da sola `this` vale `undefined`.
- **C** Assegnare a una proprietà di sola lettura solleva un `TypeError`.
- D Rende obbligatorio dichiarare il tipo delle variabili.
- E Impedisce di usare le arrow function.
- F Trasforma `==` in `===` in tutto lo script.
- G Vieta l'uso di `var` e di `function`.
- H Fa eseguire lo script solo dopo il caricamento della pagina.

Risposte: A, B, C. La modalità strict trasforma in errori alcuni comportamenti silenziosi della versione «sciatta» del linguaggio: dichiarazioni implicite, `this` globale nelle funzioni chiamate da sole, scritture su proprietà non scrivibili. I moduli e le classi sono sempre in modalità strict. *(Dispensa: cap. 5, «JavaScript moderno»)*

Es. **141.** **(5 punti)** Si confrontino le dichiarazioni `var`, `let` e `const` in JavaScript rispetto a visibilità (scope), hoisting e possibilità di riassegnazione, con un esempio.

Traccia della risposta.

- `var`: visibilità di funzione; sollevata e inizializzata a `undefined` (leggerla prima dà `undefined`); ridichiarabile.
- `let`: visibilità di blocco; sollevata ma non inizializzata fino alla dichiarazione (TDZ, `ReferenceError`); riassegnabile.
- `const`: come `let`, ma va inizializzata e non si può riassegnare (`TypeError`); l'oggetto a cui punta resta modificabile.
- Esempio: `var` dentro un `if` è visibile fuori, `let` no.

(Dispensa: cap. 5, «Variabili»)

RACCOLTA

Es. **142.** **(5 punti)** Si spieghi la differenza fra gli operatori `==` e `===` in JavaScript, con almeno tre esempi in cui danno risultati diversi.

Traccia della risposta.

- `===` (uguaglianza stretta): vero solo se tipo e valore coincidono, senza conversioni.
- `==` (uguaglianza debole): se i tipi differiscono converte gli operandi (di solito in numeri) prima di confrontarli.
- Esempi: `"1" == 1` vero e `"1" === 1` falso; `0 == false` vero, `0 === false` falso; `null == undefined` vero, `null === undefined` falso.
- Le conversioni producono sorprese (es. `true == "1"` vero, `true == 42` falso): si usa sempre `===`.

(Dispensa: cap. 5, «Uguaglianza debole e uguaglianza stretta»)

Es. **143.** **(5 punti)** Si spieghi come viene determinato il valore di `this` in JavaScript, distinguendo chiamata come metodo, chiamata semplice, chiamata con `new` e arrow function.

Traccia della risposta.

- Come metodo (`obj.m()`): `this` è l'oggetto prima del punto.
- Chiamata semplice (`f()`): in modalità strict `this` è `undefined` (anche se `f` era un metodo estratto dall'oggetto).
- Con `new`: `this` è il nuovo oggetto in costruzione.
- Arrow function: non hanno un proprio `this`, usano quello dello scope in cui sono definite (utile nelle callback, inadatte come metodi).

(Dispensa: cap. 5, «La parola chiave this»)

Es. **144.** **(5 punti)** Che cos'è una **closure** in JavaScript? Si illustri il concetto con un esempio e si indichi a che cosa serve.

Traccia della risposta.

- Una funzione che ricorda le variabili dello scope in cui è stata creata, anche dopo che la funzione esterna è terminata.
- Esempio: `function contatore() { let n = 0; return () => ++n; }`: ogni contatore ha la propria `n`.
- Serve a mantenere uno stato privato (non accessibile dall'esterno) e a creare funzioni configurate (fabbriche di funzioni, gestori di eventi).

(Dispensa: cap. 5, «Funzioni annidate restituite»)

Es. **145.** **(5 punti)** Che cos'è l'**hoisting** in JavaScript? Si spieghi come si comportano le dichiarazioni `var`, `let`, le dichiarazioni di funzione e le espressioni di funzione.

Traccia della risposta.

- Le dichiarazioni vengono elaborate prima di eseguire il codice dello scope, come se fossero spostate in cima.
- `var`: sollevata e inizializzata a `undefined`; `let`/`const`: sollevate ma nella TDZ fino alla dichiarazione (`ReferenceError`).
- Dichiarazioni di funzione: sollevate per intero, si possono chiamare prima della definizione.
- Espressioni di funzione: si comportano come la variabile che le contiene (con `var` `TypeError` *is not a function*, con `let`/`const` `ReferenceError`).

(Dispensa: cap. 5, «Hoisting»)

Capitolo 6 — JavaScript avanzato

Es. **146.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
1 const a = [3, 7, 1, 2];
2 console.log(a.pop());
3 console.log(a.shift());
4 console.log(a.push(9));
5 console.log(a.join(","));
```

- [A] 2, 3, 3, 7,1,9.
- [B] 2, 2, 3, 7,1,9.
- [C] 2, 3, 3, 3,7,1,2.
- [D] 3, 3, 3, 7,1,9.
- [E] 2, 2, 9, 7,1,9.
- [F] 3, 3, 9, 7,1,9.

Risposta: A. `push` e `unshift` aggiungono in fondo e in testa e restituiscono la **nuova lunghezza**; `pop` e `shift` rimuovono dall'ultimo e dal primo posto e restituiscono l'**elemento rimosso**. `indexOf` restituisce la posizione (o -1), `includes` un booleano. Tutti modificano l'array, tranne `indexOf`, `includes` e `length`. *(Dispensa: cap. 6, «Metodi per aggiungere e rimuovere»)*

Es. **147.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
1 const animale = { zampe: 4, verso: "..." };
2 const gatto = { __proto__: animale, verso: "miao" };
3 gatto.zampe = 3;
4 console.log(gatto.zampe, animale.zampe, gatto.verso);
5 console.log(Object.keys(gatto).length);
```

- **A** 3 4 miao, 2.
- B 3 3 miao, 2.
- C 3 4 miao, 3.
- D 4 4 miao, 1.
- E 3 3 ..., 1.
- F 3 4 ..., 2.

Risposta: A. Leggendo una proprietà che l'oggetto non ha, JavaScript la cerca lungo la catena dei prototipi. Scrivendo, invece, la proprietà viene sempre creata (o modificata) **sull'oggetto stesso**: `gatto.zampe = 3` non tocca `animale`. Le proprietà proprie di `gatto` sono `verso` e `zampe`. *(Dispensa: cap. 6, «I prototipi si usano solo in lettura»)*

Es. **148.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
1 function Pet(nome) {
2   this.nome = nome;
3   this.corri = function () { return "corro"; };
4 }
5 Pet.prototype.saluta = function () { return "ciao " + this.nome; };
6 const a = new Pet("Fido");
7 const b = new Pet("Rex");
8 console.log(a.saluta === b.saluta, a.corri === b.corri);
9 console.log(b.saluta());
```

- **A** `true false`, ciao Rex.
- B `false false`, ciao Rex.
- C `true true`, ciao Rex.
- D `false true`, ciao Rex.
- E `true false`, ciao Fido.
- F `true false`, ciao `undefined`.

Risposta: A. I metodi definiti su `Pet.prototype` esistono una sola volta e sono condivisi da tutte le istanze tramite la catena dei prototipi. Una funzione assegnata a `this` nel costruttore viene invece ricreata per ogni oggetto. `this` dentro `saluta` è l'oggetto su cui la si chiama. *(Dispensa: cap. 6, «Due modi per aggiungere metodi»)*

Es. **149.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
1 const a = [10, 20, 30];
2 a[5] = 60;
3 console.log(a.length, a[4]);
4 a.length = 2;
5 console.log(a.join("-"));
```

- **A** 6 `undefined`, 10-20.
- B 4 `undefined`, 10-20.
- C 6 `null`, 10-20.
- D 6 `undefined`, 10-20-30--60.
- E Viene sollevato un `RangeError`.
- F 4 60, 10-20.

Risposta: A. Assegnare a un indice oltre la fine allunga l'array: `length` diventa l'ultimo indice più uno e le posizioni intermedie restano vuote (leggerle dà `undefined`). Ridurre `length` tronca l'array. *(Dispensa: cap. 6, «La proprietà length»)*

RACCOLTA

Es. **150.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
const a = [1, 2, 3, 4, 5];
const b = a.slice(1, 3);
const c = a.splice(1, 2);
console.log(b.join(","), c.join(","), a.join(","));
```

- **A** `2,3 2,3 1,4,5.`
- B `2,3 2,3 1,2,3,4,5.`
- C `2,3,4 2,3 1,4,5.`
- D `2,3 2,3,4 1,5.`
- E `2 2,3 1,4,5.`
- F `2,3 3,4 1,2,5.`

Risposta: A. `slice(inizio, fine)` restituisce una copia della porzione (fine esclusa) senza modificare l'array; `splice(inizio, quanti)` **rimuove** `quanti` elementi dall'array originale e li restituisce. *(Dispensa: cap. 6, «Metodi per aggiungere e rimuovere»)*

Es. **151.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
const voti = [18, 24, 30, 27, 21];
const r = voti
  .filter(v => v >= 24)
  .map(v => v + 1)
  .reduce((somma, v) => somma + v, 0);
console.log(r, voti.length);
```

- **A** `84 5.`
- B `81 5.`
- C `84 3.`
- D `125 5.`
- E `81 3.`
- F `[ 25, 31, 28 ] 5.`

Risposta: A. `filter` tiene 24, 30 e 27; `map` li trasforma in 25, 31 e 28; `reduce` li somma a partire da 0: 84. Questi metodi restituiscono nuovi valori senza modificare l'array originale, che resta di 5 elementi. *(Dispensa: cap. 6, «Iterazione»)*

Es. **152.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
const a = ["x", "y"];
for (const i in a) console.log(typeof i, i);
for (const v of a) console.log(v);
```

- **A** `string 0, string 1, x, y.`
- B `number 0, number 1, x, y.`
- C `string x, string y, 0, 1.`
- D `number 0, number 1, 0, 1.`
- E `string 0, string 1, 0, 1.`

Risposta: A. `for...in` scorre i **nomi** delle proprietà enumerabili, che sono stringhe anche per gli indici di un array (e includerebbe eventuali proprietà aggiunte): per gli array non va usato. `for...of` scorre i **valori** di un oggetto iterabile. *(Dispensa: cap. 6, «Iterazione»)*

Es. **153.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
const [a, , b = 5, ...resto] = [1, 2, undefined, 4, 6];
const { nome: n, anni = 18 } = { nome: "Ada", corso: "TW" };
console.log(a, b, resto.length, n, anni);
```

- **A** `1 5 2 Ada 18.`
- B `1 2 2 Ada 18.`
- C `1 5 3 Ada 18.`
- D `1 undefined 2 Ada 18.`
- E `1 5 2 undefined 18.`
- F Viene sollevato un `ReferenceError`.

Risposta: A. Nella destrutturazione di array si può saltare una posizione con una virgola, dare un valore predefinito (usato se l'elemento è `undefined`) e raccogliere il resto con `...`. In quella di oggetti `nome: n` copia la proprietà `nome` nella variabile `n`, e `anni = 18` vale 18 perché la proprietà manca. *(Dispensa: cap. 6, «Assegnamenti destrutturanti»)*

RACCOLTA

Es. **154.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
function conta(primo, ...altri) {
  return altri.length;
}
const v = [1, 2];
console.log(conta(...v, 3, ...v));
const o = { a: 1, b: 2 };
const p = { ...o, b: 3, c: 4 };
console.log(Object.values(p).join(","));
```

- **A** 4, 1,3,4.
- B 5, 1,3,4.
- C 4, 1,2,3,4.
- D 2, 1,3,4.
- E 4, 1,2,4.
- F 3, 1,3,4.

Risposta: A. Nella chiamata lo spread espande gli array: gli argomenti sono 1, 2, 3, 1, 2; il parametro *rest* `...altri` raccoglie tutti quelli dopo il primo (4). Negli oggetti lo spread copia le proprietà, e quelle scritte dopo sovrascrivono quelle con lo stesso nome. *(Dispensa: cap. 6, «Assegnamenti destrutturanti»)*

Es. **155.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
const o = {};
o[1] = "numero";
o["1"] = "stringa";
const m = new Map();
m.set(1, "numero");
m.set("1", "stringa");
console.log(Object.keys(o).length, m.size, m.get(1));
```

- **A** `1 2 numero`.
- B `2 2 numero`.
- C `1 1 stringa`.
- D `1 2 stringa`.
- E `2 1 numero`.
- F `1 2 undefined`.

Risposta: A. Le chiavi delle proprietà di un oggetto sono stringhe: `o[1]` e `o["1"]` sono la stessa proprietà. Una `Map` accetta chiavi di qualunque tipo e le distingue: il numero 1 e la stringa `"1"` sono due chiavi diverse. *(Dispensa: cap. 6, «Mappe»)*

Es. **156.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
const s = new Set([3, 1, 3, "3", 1]);
s.add(1);
console.log(s.size, [...s].join(" "));
```

- **A** `3 3 1 3`.
- B `2 1 3`.
- C `5 3 1 3 3 1`.
- D `4 3 1 3 1`.
- E `3 1 3 3`.
- F `2 3 1`.

Risposta: A. Un `Set` contiene valori senza ripetizioni (il numero 3 e la stringa `"3"` sono diversi) e ricorda l'ordine di inserimento; aggiungere un valore già presente non ha effetto. *(Dispensa: cap. 6, «Insiemi»)*

RACCOLTA

Es. **157.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
1 const voti = new Map([["web", 30], ["reti", 27]]);
2 voti.set("web", 28);
3 for (const [esame, voto] of voti) {
4   console.log(esame + ": " + voto);
5 }
```

- **A** `web: 28, reti: 27.`
- B `web: 30, reti: 27, web: 28.`
- C `reti: 27, web: 28.`
- D `web: 30, reti: 27.`
- E `0: web,28, 1: reti,27.`

Risposta: A. Iterando una `Map` con `for...of` si ottengono coppie `[chiave, valore]`, che si possono destrutturare. Le chiavi restano nell'ordine di inserimento; `set` su una chiave esistente ne aggiorna il valore senza spostarla. *(Dispensa: cap. 6, «Iterare su una mappa»)*

Es. **158.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
1 console.log(typeof Pet);
2 const p = new Pet("Fido");
3 class Pet {
4   constructor(nome) { this.nome = nome; }
5 }
```

- **A** Viene sollevato un `ReferenceError`.
- B `function`.
- C `class`.
- D `function`, poi viene sollevato un `ReferenceError`.
- E `undefined`, poi viene sollevato un `TypeError`.
- F `undefined`.

Risposta: A. Le dichiarazioni di classe vengono sollevate come `let`: fino alla dichiarazione la classe è nella *temporal dead zone*. Già `typeof Pet` solleva un `ReferenceError` (lo stesso `typeof` che, per un nome mai dichiarato, restituirebbe `"undefined"`). *(Dispensa: cap. 6, «Non solo zucchero sintattico»)*

Es. **159.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
1  class Pet {
2    constructor(nome) { this.nome = nome; }
3    verso() { return "..."; }
4    presentati() { return this.nome + " fa " + this.verso(); }
5  }
6  class Cane extends Pet {
7    verso() { return "bau"; }
8  }
9  const c = new Cane("Fido");
10 console.log(typeof Pet, c.presentati(), c instanceof Pet);
```

- **A** `function Fido fa bau true.`
- B `class Fido fa bau true.`
- C `function Fido fa ... true.`
- D `function Fido fa bau false.`
- E `object Fido fa bau true.`
- F `class Fido fa ... false.`

Risposta: A. Una classe è, sotto sotto, una funzione costruttrice (`typeof` dà `function`). `Cane` eredita costruttore e `presentati` da `Pet` e ridefinisce `verso`: `this.verso()` chiama il metodo dell'oggetto reale, quindi quello di `Cane`. Un `Cane` è anche un `Pet`. *(Dispensa: cap. 6, «Ereditarietà fra classi»)*

RACCOLTA

Es. **160.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
class Punto {
  constructor(x) { this.x = x; }
  static origine() { return new Punto(0); }
}
console.log(Punto.origine().x);
const p = Punto(5);
console.log(p.x);
```

- **A** 0, poi viene sollevato un `TypeError`.
- B 0, 5.
- C 0, `undefined`.
- D Viene sollevato un `TypeError`.
- E 0, poi viene sollevato un `ReferenceError`.

Risposta: A. I metodi `static` si chiamano sulla classe, non sulle istanze. A differenza delle vecchie funzioni costruttrici, una classe non si può chiamare senza `new`: `Punto(5)` solleva un `TypeError` (*Class constructor Punto cannot be invoked without 'new'*). *(Dispensa: cap. 6, «Non solo zucchero sintattico»)*

Es. **161.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
function f() {
  try {
    console.log("A");
    null.x;
    console.log("B");
  } catch (e) {
    console.log(e instanceof TypeError, e instanceof Error);
    return "C";
  } finally {
    console.log("D");
  }
}
console.log(f());
```

- **A** A, true true, D, C.
- B A, true true, C, D.
- C A, B, D, C.
- D A, true false, D, C.
- E A, true true, C.
- F A, false true, D, C.

Risposta: A. Leggere una proprietà di `null` solleva un `TypeError`, quindi `B` non viene stampata. Ogni errore predefinito eredita da `Error`. Il `return` nel `catch` viene eseguito, ma prima che la funzione restituisca davvero il valore viene eseguito il blocco `finally`. *(Dispensa: cap. 6, «try / catch / finally»)*

Es. **162.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
class VotoNonValido extends Error {}
function registra(v) {
  if (v < 18 || v > 30) throw new VotoNonValido("voto " + v);
  return "ok " + v;
}
for (const v of [28, 31, 18]) {
  try { console.log(registra(v)); }
  catch (e) { console.log(e.name, e.message, e instanceof VotoNonValido); }
}
```

- **A** `ok 28`, `Error voto 31 true`, `ok 18`.
- B `ok 28`, `VotoNonValido voto 31 true`, `ok 18`.
- C `ok 28`, `Error voto 31 true`.
- D `ok 28`, `Error voto 31 false`, `ok 18`.
- E `ok 28`, `voto 31`, `ok 18`.
- F `ok 28`, poi viene sollevato un `VotoNonValido`.

Risposta: A. Si possono sollevare errori con `throw` e definire nuovi tipi di errore estendendo `Error`. Un errore catturato non interrompe il ciclo. Attenzione: `name` resta `"Error"` finché la sottoclasse non lo imposta (per esempio con `this.name = "VotoNonValido"` nel costruttore), mentre `instanceof` riconosce comunque la classe. *(Dispensa: cap. 6, «Sollevare errori»)*

RACCOLTA

Es. **163.** **(2 punti)** Quali delle seguenti affermazioni sui moduli JavaScript (`<script type="module">`) sono **vere**?

- **A** Ogni modulo ha un proprio scope: le variabili non diventano globali.
- **B** Il codice di un modulo è sempre in modalità strict.
- **C** Gli script di tipo modulo si comportano come `defer` per impostazione predefinita.
- **D** Aprendo la pagina con un doppio clic (URL `file://`) i moduli non vengono caricati.
- E Un modulo può importare solo ciò che è stato esportato con `export default`.
- F Le variabili dichiarate in un modulo sono visibili in tutti gli altri script.
- G Per usare i moduli serve Node.js anche nel browser.
- H Un modulo viene eseguito di nuovo ogni volta che viene importato.
- I L'istruzione `import` può comparire solo dentro una funzione.

Risposte: A, B, C, D. I moduli hanno uno scope proprio e condividono solo ciò che esportano; sono in strict mode e differiti. Il browser li carica come risorse soggette alle regole di sicurezza (CORS), quindi serve un server, anche locale. Un modulo viene valutato una sola volta, anche se importato più volte. *(Dispensa: cap. 6, «Cose da sapere sui moduli»)*

Es. **164.** **(2 punti)** Il modulo `cerchio.js` contiene `export default class Cerchio { ... }` ed `export const PI = 3.14;`. Quale istruzione importa correttamente entrambi?

- **A** `import Cerchio, { PI } from './cerchio.js';`
- B `import { Cerchio, PI } from './cerchio.js';`
- C `import Cerchio, PI from './cerchio.js';`
- D `import default Cerchio, PI from './cerchio.js';`
- E `import * from './cerchio.js';`
- F `require('./cerchio.js').Cerchio, PI;`

Risposta: A. L'export predefinito (al più uno per modulo) si importa senza graffe e con un nome a scelta; gli export con nome si importano fra graffe con il loro nome (rinominabile con `as`). *(Dispensa: cap. 6, «Altre forme di export e import»)*

Es. **165.** **(2 punti)** Che cosa succede quando si accede a una proprietà che un oggetto non possiede?

- **A** JavaScript la cerca nel prototipo, poi nel prototipo del prototipo, e così via.
- B Viene sollevato un `ReferenceError`.
- C Viene creata automaticamente con valore `undefined`.
- D JavaScript la cerca fra le variabili globali.
- E Viene sollevato un `TypeError`, perché l'oggetto non è estensibile.
- F JavaScript la cerca negli oggetti creati con lo stesso costruttore.

Risposta: A. Ogni oggetto ha un prototipo (accessibile con `Object.getPrototypeOf` o `__proto__`). La ricerca risale la catena dei prototipi fino a `Object.prototype`; se la proprietà non si trova si ottiene `undefined`. *(Dispensa: cap. 6, «Prototipi ed ereditarietà»)*

Es. **166.** **(5 punti)** Si spieghi il meccanismo dei **prototipi** in JavaScript e il suo rapporto con le funzioni costruttrici e con le classi.

Traccia della risposta.

- Ogni oggetto ha un prototipo; le proprietà mancanti si cercano lungo la catena dei prototipi (in lettura; le scritture avvengono sull'oggetto stesso).
- Un oggetto creato con `new F()` ha come prototipo `F.prototype`: i metodi definiti lì sono condivisi da tutte le istanze.
- Le classi sono costruite sullo stesso meccanismo: i metodi stanno nel prototipo della classe, `extends` collega i prototipi.
- Le classi aggiungono vincoli: non si possono chiamare senza `new`, sono in strict mode, non vengono inizializzate prima della dichiarazione.

(Dispensa: cap. 6, «Prototipi ed ereditarietà»)

RACCOLTA

Es. **167.** **(5 punti)** Si spieghino le differenze fra un oggetto usato come dizionario e una `Map`, e si descriva a che cosa serve un `Set`, con un esempio.

Traccia della risposta.

- In un oggetto le chiavi sono stringhe (o simboli); in una `Map` qualunque valore, anche oggetti, e `1` e `"1"` restano distinte.
- La `Map` ha `size`, metodi dedicati (`set`, `get`, `has`, `delete`), conserva l'ordine di inserimento ed è iterabile con `for...of` (coppie chiave-valore).
- Un oggetto eredita proprietà dal prototipo, che possono confondersi con le chiavi.
- Un `Set` è una collezione di valori senza duplicati: es. `new Set(lista)` per eliminare le ripetizioni, `has` per l'appartenenza.

(Dispensa: cap. 6, «Mappe» e «Insiemi»)

Es. **168.** **(5 punti)** Si descriva il meccanismo di gestione degli errori in JavaScript (`try`, `catch`, `finally`, `throw`), indicando come si possono definire e riconoscere errori specifici.

Traccia della risposta.

- `try` racchiude il codice che può fallire; se viene sollevata un'eccezione l'esecuzione passa al `catch`, che riceve l'oggetto errore.
- `finally` viene eseguito sempre, con o senza errore (anche dopo un `return`).
- `throw` solleva un errore: di solito un oggetto `Error` o di una sottoclasse (`TypeError`, `ReferenceError`, o classi proprie con `extends Error`).
- Nel `catch` si distingue il tipo con `instanceof` e si rilancia ciò che non si sa gestire; un errore non gestito interrompe lo script.

(Dispensa: cap. 6, «Gestione degli errori»)

Es. **169.** **(5 punti)** Perché sono stati introdotti i **moduli** in JavaScript? Si descrivano le forme di `export` e `import` e il comportamento degli script di tipo modulo nel browser.

Traccia della risposta.

- Senza moduli tutti gli script condividono lo scope globale: conflitti di nomi e dipendenze implicite fra file.
- Un modulo ha un proprio scope ed esporta esplicitamente ciò che vuole condividere: export con nome (`export const x`) ed export predefinito (`export default`).
- `import { x, y as z } from './m.js'` per gli export con nome, `import M from './m.js'` per quello predefinito.
- Nel browser: `<script type="module">`, strict mode, esecuzione differita, una sola valutazione per modulo; servono un server (no `file://`).

(Dispensa: cap. 6, «Moduli»)

Capitolo 7 — JavaScript nel browser

RAC

Es. **170.** **(2 punti)** Qual è l'output in console dopo il caricamento completo del documento HTML riportato di seguito?

```
<!DOCTYPE html>
<html>
<head><title>DOM</title></head>
<body>
  <ul><li>Uno</li></ul>
  <nav>Menu</nav>
  <p>Testo</p>
  <script>
    console.log(document.body.lastElementChild.tagName);
    console.log(document.body.children.length);
    console.log(document.body.firstChild.tagName);
  </script>
</body>
</html>
```

- **A** SCRIPT, 4, undefined.
- B script, 4, undefined.
- C P, 3, undefined.
- D SCRIPT, 4, UL.
- E P, 4, undefined.
- F script, 4, UL.

Risposta: A. L'a-capo e l'indentazione fra un tag e l'altro restano nel DOM come **nodi di testo** (`#text`): ogni figlio del `<body>` è preceduto da uno di essi. Mentre lo script viene eseguito il parser è fermo al tag `<script>`: lo script è già nel DOM (è l'ultimo figlio del `<body>`), ciò che lo segue non ancora. `nodeName` vale `#text` o `#comment` per testo e commenti e il nome del tag in maiuscolo per gli elementi; `tagName` esiste solo per gli elementi (su un nodo di testo dà `undefined`). `children` e le proprietà `...Element...` considerano soltanto gli elementi. *(Dispensa: cap. 7, «Spazi e a-capo: i nodi di testo che non si vedono»)*

Es. **171.** **(2 punti)** Qual è l'output in console dopo il caricamento completo del documento HTML riportato di seguito?

```
<ul>
  <li>A</li>
  <li>B</li>
  <li>C</li>
</ul>
<script>
  function nuovo(testo) {
    const li = document.createElement("li");
    li.textContent = testo;
    return li;
  }
  const ul = document.querySelector("ul");
  const viva = ul.getElementsByTagName("li");
  const statica = ul.querySelectorAll("li");
  ul.prepend(nuovo("N"));
  console.log(viva.length, statica.length);
  console.log(viva[0].textContent, statica[0].textContent);
</script>
```

- **A** 4 3, N A.
- B 4 4, N N.
- C 3 3, A A.
- D 3 4, A N.
- E 4 3, A A.
- F 3 3, N A.

Risposta: A. `getElementsByTagName` restituisce una `HTMLCollection` **viva**, che riflette in ogni momento lo stato del documento; `querySelectorAll` restituisce una `NodeList` **statica**, una fotografia scattata al momento della ricerca, che continua a contenere gli elementi trovati allora (anche quelli rimossi in seguito). `append` inserisce in fondo, `prepend` all'inizio. *(Dispensa: cap. 7, «HTMLCollection e NodeList»)*

RACCOLTA

Es. **172.** **(2 punti)** Qual è l'output in console dopo il caricamento completo del documento HTML riportato di seguito?

```
<ul><li>B</li></ul>
<script>
  function li(testo) {
    const e = document.createElement("li");
    e.textContent = testo;
    return e;
  }
  const ul = document.querySelector("ul");
  ul.after("Y");
  ul.append("C");
  ul.prepend("Z");
  console.log(ul.children.length, ul.childNodes.length);
  console.log(ul.textContent);
</script>
```

- **A** 1 3, ZBC.
- B 3 3, ZBC.
- C 1 3, CBZ.
- D 1 4, ZBYC.
- E 1 3, ZBYC.
- F 1 3, B.

Risposta: A. `prepend` e `append` inseriscono **dentro** l'elemento, all'inizio e in fondo; `before` e `after` **fuori**, subito prima e subito dopo di esso. Le stringhe diventano nodi di testo: contano in `childNodes` ma non in `children`, che contiene solo elementi. `textContent` concatena il testo di tutti i discendenti, nell'ordine. *(Dispensa: cap. 7, «Dove: i quattro punti di inserimento»)*

Es. **173.** **(2 punti)** Si consideri il seguente frammento di documento HTML. Cosa viene stampato in console in seguito ad un click sul `<button>`?

```
<button>Clic</button>
<script>
  const b = document.querySelector("button");
  b.onclick = () => console.log("A");
  b.addEventListener("click", () => console.log("B"));
</script>
```

- **A** A, B.
- B A.
- C B.
- D B, A.

Risposta: A. La proprietà `onclick`, come l'attributo HTML omonimo, contiene **un solo** gestore: ogni assegnamento sostituisce il precedente, compreso il codice dell'attributo. `addEventListener` invece **aggiunge** un gestore senza toccare gli altri. Al clic i gestori vengono eseguiti nell'ordine in cui sono stati registrati: qui per primo quello di `onclick`, impostato prima dei listener. *(Dispensa: cap. 7, «Gestori di eventi»)*

Es. **174.** **(2 punti)** Si consideri il seguente frammento di documento HTML. Cosa viene stampato in console in seguito ad un click sul `<button>`?

```
<div id="d">
  <p id="p">
    <button id="b">Clic</button>
  </p>
</div>
<script>
  document.getElementById("d").addEventListener("click", () => console.log("div"));
  document.addEventListener("click", () => console.log("document"));
  document.getElementById("p").addEventListener("click", () => console.log("p"));
</script>
```

- **A** p, div, document.
- B p.
- C document, div, p.
- D Non viene stampato nulla.
- E div, document, p.
- F p, div.

Risposta: A. Per il *bubbling*, un clic sul pulsante esegue prima i gestori del pulsante, poi quelli del suo genitore e così via, risalendo fino a `document`, qualunque sia l'ordine in cui i gestori sono stati registrati. `stopPropagation()` non interrompe il gestore in cui viene chiamato, ma impedisce all'evento di risalire oltre. *(Dispensa: cap. 7, «Bubbling» e «Fermare il bubbling»)*

RACCOLTA

Es. **175.** **(2 punti)** Si consideri il seguente frammento di documento HTML. Cosa viene stampato in console in seguito ad un click sulla voce «Contatti»?

```
<ul id="menu">
  <li>Home</li>
  <li>News</li>
  <li>Corsi</li>
  <li>Contatti</li>
</ul>
<script>
  document.getElementById("menu").addEventListener("click", function (e) {
    console.log(this.children.length);
    console.log(e.target === this);
    console.log(this.tagName);
  });
</script>
```

- **A** 4, `false`, UL.
- B 0, `true`, LI.
- C 1, `false`, UL.
- D 4, `true`, UL.
- E 9, `false`, UL.
- F 1, `true`, UL.

Risposta: A. Il gestore è registrato sulla `<ul>`, ma il clic avviene su un `<li>`: l'evento risale per *bubbling* fino alla lista. Nel gestore `this` è l'elemento a cui il gestore è stato assegnato (la `<ul>`), mentre `event.target` è l'elemento su cui l'evento è stato effettivamente generato (il `<li>` cliccato): è ciò che rende possibile la delegazione. *(Dispensa: cap. 7, «Delegazione degli eventi»)*

Es. **176.** **(2 punti)** Si considerino lo script `app.js` (a sinistra) e la pagina che lo include (a destra). Qual è l'output in console dopo il caricamento completo della pagina?

```
const h1 = document.body.querySelector("h1");
console.log(h1.innerText);
```

```
<!DOCTYPE html>
<html>
<head>
  <title>Script</title>
  <script defer src="app.js"></script>
</head>
<body>
  <h1>JavaScript</h1>
</body>
</html>
```

- **A** `JavaScript`.
- B Viene sollevato un `TypeError`.
- C `null`.
- D `undefined`.
- E Viene sollevato un `ReferenceError`.
- F Non viene stampato nulla.

Risposta: A. Uno script senza attributi viene scaricato ed eseguito appena il parser lo incontra: nell'`<head>` il `<body>` non esiste ancora, `document.body` vale `null` e chiamarvi un metodo solleva un `TypeError` (con `?.` si ottiene invece `undefined`). Con `defer` lo script esterno viene eseguito dopo l'analisi completa della pagina; lo stesso risultato si ottiene mettendo lo script in fondo al `<body>` o attendendo l'evento `DOMContentLoaded`. *(Dispensa: cap. 7, «Ordine di esecuzione degli script»)*

Es. **177.** **(2 punti)** Quali delle seguenti alternative utilizzano **solo** API comunemente considerate parte del BOM (*Browser Object Model*)?

- **A** `history.back(); let x = location.href;`
- B `window.document.title = "foo"`
- C Nessuna delle altre
- D `location.href = document.body.innerHTML;`
- E `document.querySelector("a.foo")`
- F `document.querySelector("h1").textContent`

Risposta: A. `history` e `location` fanno parte del BOM, che serve a lavorare con il browser stesso (cronologia, indirizzo, schermo, informazioni sul browser). `document`, e tutto ciò che si raggiunge da esso, è invece il DOM, anche quando vi si arriva passando da `window`: le altre alternative usano almeno un'API del DOM. *(Dispensa: cap. 7, «Il DOM e il BOM») — Fonte: facsimile mostrato in aula.*

RACCOLTA

Es. **178.** **(2 punti)** Qual è l'output in console dopo il caricamento completo del documento HTML riportato di seguito?

```
<!DOCTYPE html>
<html>
  <head><title>DOM</title></head>
  <body>
    <script> console.log(document.body.querySelector("h1")?.textContent) </script>
    <h1>Hello</h1>
  </body>
</html>
```

- **A** `undefined`.
- B `<h1>Hello</h1>`.
- C `Hello`.
- D Viene sollevato un `TypeError`.

Risposta: A. Lo script viene eseguito appena il parser lo incontra, quando l'`<h1>` che lo segue non è ancora nel DOM: `querySelector` restituisce `null`. L'operatore `?.` evita l'errore e dà `undefined` (senza `?.` si avrebbe un `TypeError`). *(Dispensa: cap. 7, «Ordine di esecuzione degli script») — Fonte: facsimile mostrato in aula.*

Es. **179.** **(2 punti)** Si consideri il seguente frammento di documento HTML. Cosa viene stampato in console in seguito ad un click sul `<button>`?

```
<div onclick="console.log('div');">
  <p onclick="console.log('p')">
    <button onclick="console.log('b')">click</button>
  </p>
</div>
```

- **A** b, p, div.
- B `div`.
- C b.
- D `div`, p, b.

Risposta: A. Per il *bubbling* vengono eseguiti prima i gestori dell'elemento su cui è avvenuto il clic, poi quelli del genitore e via via degli altri antenati: dal più interno al più esterno. *(Dispensa: cap. 7, «Bubbling») — Fonte: facsimile mostrato in aula.*

Es. **180.** **(2 punti)** Qual è l'output in console dopo il caricamento completo del documento HTML riportato di seguito?

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>Web</title>
    <meta charset="UTF-8"/></head>
<body>
 <script>
   let h = document.head?.children;
   console.log(h[1]?.outerHTML);
 </script></body></html>
```

- **A** `<meta charset="UTF-8">`.
- B `undefined`.
- C Viene sollevato un `TypeError`.
- D `<title>Web</title>`.
- E `null`.
- F Una stringa contenente solo spazi bianchi.

Risposta: A. `document.head.children` contiene solo gli **elementi** figli di `<head>`, senza i nodi di testo: `[0]` è `<title>`, `[1]` è `<meta>`. `outerHTML` è il codice dell'elemento ricostruito a partire dal DOM: la barra di `/>`, che per il parser non conta, non compare. *(Dispensa: cap. 7, «Navigare fra soli elementi») — Fonte: prima prova intercorso del 23 aprile 2024.*

RACCOLTA

Es. **181.** **(2 punti)** Quali delle seguenti istruzioni usano **solo** oggetti del BOM (*Browser Object Model*)?

- **A** `console.log(navigator.userAgent);`
- **B** `history.back();`
- **C** `console.log(screen.availWidth);`
- **D** `location.reload();`
- **E** `let quante = history.length;`
- F `document.title = location.href;`
- G `document.body.style.background = "pink";`
- H `window.document.querySelector("p").remove();`
- I `let h = document.head.children;`
- J `location.href = document.links[0].href;`

Risposte: A, B, C, D, E. Il BOM comprende gli oggetti con cui si lavora con il browser: `location` (l'URL corrente), `history` (la cronologia), `navigator` (browser e piattaforma), `screen` (lo schermo). Tutto ciò che passa da `document` appartiene al DOM, anche se vi si arriva da `window`. *(Dispensa: cap. 7, «Il DOM e il BOM»)*

Es. **182.** **(2 punti)** Qual è l'output in console dopo il caricamento completo del documento HTML riportato di seguito?

```
<script>
  var corso = "TW";
  let anno = 2026;
  function saluta() { return "ciao"; }
  console.log(window.corso, window.anno, typeof window.saluta);
</script>
```

- **A** `TW undefined function.`
- B `TW 2026 function.`
- C `undefined undefined function.`
- D `undefined undefined undefined.`
- E `TW undefined undefined.`
- F Viene sollevato un `ReferenceError`.

Risposta: A. `window` è l'oggetto globale: le funzioni dichiarate a livello globale e le variabili `var` ne diventano proprietà. Le variabili `let` e `const` globali vivono invece in uno scope separato e **non** diventano proprietà di `window`. *(Dispensa: cap. 7, «L'oggetto window»)*

Es. **183.** **(2 punti)** Qual è l'output in console dopo il caricamento completo del documento HTML riportato di seguito?

```
<p class="a">Uno</p>
<script>
  console.log(document.querySelector(".b"));
  console.log(document.querySelectorAll(".b").length);
  console.log(document.getElementById("b"));
</script>
```

- **A** `null`, `0`, `null`.
- B `undefined`, `0`, `undefined`.
- C `null`, `null`, `null`.
- D `undefined`, `undefined`, `undefined`.
- E `null`, poi viene sollevato un `TypeError`.
- F `null`, `undefined`, `null`.

Risposta: A. Quando non trovano nulla, `querySelector` e `getElementById` restituiscono `null`. `querySelectorAll` restituisce invece una `NodeList` **vuota** (con `length` uguale a 0), come le collezioni di `getElementsByTagName` e `getElementsByClassName`: si possono scorrere senza controlli. *(Dispensa: cap. 7, «Trovare elementi»)*

RACCOLTA

Es. **184.** **(2 punti)** Quali dei seguenti metodi e proprietà restituiscono una collezione **viva**, che si aggiorna da sola quando il documento cambia?

- **A** `document.getElementsByTagName("li")`
- **B** `document.getElementsByClassName("x")`
- **C** `elem.children`
- **D** `elem.childNodes`
- E `document.querySelectorAll("li")`
- F `Array.from(elem.children)`
- G `[...document.getElementsByTagName("li")]`
- H `document.querySelector("li")`

Risposte: A, B, C, D. Le `HTMLCollection` (`getElementsByTagName`, `getElementsByClassName`, `children`) sono sempre vive, e lo è anche la `NodeList` di `childNodes`. La `NodeList` di `querySelectorAll` è statica, e così ogni array ottenuto copiando una collezione con `Array.from` o con lo spread. `querySelector` restituisce un solo elemento, non una collezione. *(Dispensa: cap. 7, «HTMLCollection e NodeList»)*

Es. **185.** **(2 punti)** Sia `input` l'oggetto DOM corrispondente a un elemento `<input type="text">` della pagina. Quali delle seguenti espressioni valgono `true`?

- **A** `input instanceof HTMLElement`
- **B** `input instanceof Node`
- **C** `input instanceof EventTarget`
- **D** `input instanceof Element`
- E `input instanceof Text`
- F `input instanceof Document`
- G `input instanceof HTMLAnchorElement`
- H `input instanceof CharacterData`

Risposte: A, B, C, D. Le interfacce del DOM formano una gerarchia: un `<input>` è un `HTMLInputElement`, quindi anche un `HTMLElement`, un `Element`, un `Node` e un `EventTarget`. Non è un nodo di testo (`Text`, che discende da `CharacterData`), né il documento, né un link. *(Dispensa: cap. 7, «La gerarchia delle interfacce»)*

Es. **186.** **(2 punti)** Qual è l'output in console dopo il caricamento completo del documento HTML riportato di seguito?

```
1 <div id="d">Ciao <b>mondo</b></div>
2 <script>
3   const d = document.getElementById("d");
4   console.log(d.innerHTML);
5   console.log(d.textContent);
6   console.log(d.outerHTML);
7 </script>
```

- **A** `Ciao <b>mondo</b>`, `Ciao mondo`, `<div id="d">Ciao <b>mondo</b></div>`.
- B `Ciao mondo`, `Ciao <b>mondo</b>`, `<div id="d">Ciao <b>mondo</b></div>`.
- C `<div id="d">Ciao <b>mondo</b></div>`, `Ciao mondo`, `Ciao <b>mondo</b>`.
- D `Ciao <b>mondo</b>`, `Ciao <b>mondo</b>`, `<div id="d">Ciao <b>mondo</b></div>`.
- E `Ciao <b>mondo</b>`, `Ciao mondo`, `undefined`.
- F `Ciao <b>mondo</b>`, `Ciao`, `<div id="d">Ciao <b>mondo</b></div>`.

Risposta: A. `innerHTML` è il codice HTML contenuto nell'elemento, `outerHTML` quello dell'elemento intero, tag di apertura e chiusura compresi. `textContent` è soltanto il testo di tutti i discendenti, senza markup. *(Dispensa: cap. 7, «innerHTML e outerHTML»)*

RACCOLTA

Es. **187.** **(2 punti)** Qual è l'output in console dopo il caricamento completo del documento HTML riportato di seguito?

```
<input id="x" class="inp ok" custom="y" data-user-id="42">
<script>
  const i = document.querySelector("input");
  i.custom = "z";
  console.log(i.id, i.className, i.custom, i.getAttribute("custom"));
  console.log(i.dataset.userId, i.classList.length);
</script>
```

- **A** `x inp ok z y, 42 2.`
- B `x inp ok z z, 42 2.`
- C `x undefined z y, 42 2.`
- D `x inp ok y y, 42 2.`
- E `x inp ok z y, undefined 2.`
- F `x inp ok z y, 42 1.`

Risposta: A. Per gli attributi **standard** (`id`, `class`...) il browser crea proprietà corrispondenti (per `class` si chiamano `className` e `classList`, perché `class` è una parola riservata). Per un attributo non standard no: `i.custom = "z"` crea solo una proprietà JavaScript, e `getAttribute` continua a leggere il valore scritto nell'HTML. Gli attributi `data-*` si leggono da `dataset`, con i nomi in *camelCase*. *(Dispensa: cap. 7, «Attributi»)*

Es. **188.** **(2 punti)** Qual è l'output in console dopo il caricamento completo del documento HTML riportato di seguito?

```
<ul><li>A</li><li>B</li></ul>
<ol></ol>
<script>
  const h = document.createElement("h1");
  h.textContent = "Titolo";
  console.log(document.querySelectorAll("h1").length);
  const ul = document.querySelector("ul");
  document.querySelector("ol").append(ul.firstElementChild);
  console.log(ul.children.length, document.querySelectorAll("li").length);
</script>
```

- **A** `0, 1 2.`
- B `1, 1 2.`
- C `0, 2 3.`
- D `1, 2 3.`
- E `0, 1 1.`
- F `0, 2 2.`

Risposta: A. `createElement` crea l'elemento solo in memoria: finché non viene inserito con `append`, `prepend`, `before` o `after` non fa parte del documento. `append` di un nodo già presente non lo copia, lo **sposta**: un nodo può stare in un solo punto dell'albero. *(Dispensa: cap. 7, «Come: creare i nodi»)*

Es. **189.** **(2 punti)** Uno script esegue `let n = prompt("Inserisci un numero:");` e l'utente scrive 3 e preme OK. Che cosa contiene `n`?

- **A** La stringa `"3"`.
- B Il numero `3`.
- C Il valore `true`.
- D Il valore `undefined`.
- E Il valore `null`.
- F Un oggetto evento che contiene il valore digitato.

Risposta: A. `prompt` restituisce sempre il testo digitato **come stringa** (per usarlo come numero serve una conversione, per esempio con `parseInt`), oppure `null` se l'utente preme Annulla. `confirm` restituisce un booleano, `alert` nulla. *(Dispensa: cap. 7, «Interagire con l'utente»)*

RACCOLTA

Es. **190.** **(2 punti)** Quali delle seguenti affermazioni su `alert`, `confirm` e `prompt` sono **vere**?

- [x] **A** Sospendono l'esecuzione dello script finché l'utente non risponde.
- [x] **B** `confirm` restituisce `true` se l'utente preme OK e `false` se preme Annulla.
- [x] **C** `prompt` restituisce `null` se l'utente preme Annulla.
- [ ] D `alert` restituisce `true` quando l'utente preme OK.
- [ ] E Lo script prosegue mentre la finestra è aperta, e la risposta arriva in una callback.
- [ ] F `prompt` converte automaticamente in numero un valore numerico digitato.
- [ ] G Sono funzioni del linguaggio JavaScript, disponibili anche in Node.js.
- [ ] H Il loro aspetto si personalizza con i CSS della pagina.

Risposte: A, B, C. Sono finestre di dialogo **bloccanti** fornite dal browser: lo script si ferma finché l'utente non risponde. `alert` non restituisce nulla, `confirm` un booleano, `prompt` il testo digitato (sempre una stringa) o `null`. Non si possono impaginare: nelle applicazioni reali si preferiscono finestre costruite nel DOM. *(Dispensa: cap. 7, «Interagire con l'utente»)*

Es. **191.** **(2 punti)** Qual è l'output complessivo in console dopo il caricamento del documento HTML riportato di seguito e un click sul pulsante?

```
1  <button>Salutami</button>
2  <script>
3    function saluta() { console.log("Ciao!"); }
4    const btn = document.querySelector("button");
5    btn.addEventListener("click", saluta());
6    console.log("pronto");
7  </script>
```

- [x] **A** `Ciao!`, `pronto`.
- [ ] B `pronto`, `Ciao!`.
- [ ] C `Ciao!`, `pronto`, `Ciao!`.
- [ ] D `pronto`.
- [ ] E `Ciao!`, poi viene sollevato un `TypeError`.
- [ ] F `Ciao!`, `Ciao!`, `pronto`.

Risposta: A. Con le parentesi `saluta()` viene **chiamata subito**, durante il caricamento, e ad `addEventListener` arriva il suo valore di ritorno, `undefined`: nessun gestore viene registrato e il clic non stampa nulla. Per registrare un gestore si passa la funzione senza parentesi: `addEventListener("click", saluta)`. *(Dispensa: cap. 7, «Evento e funzione gestore: chi chiama chi»)*

Es. **192.** **(2 punti)** Si consideri il seguente frammento di documento HTML. Cosa viene stampato in console in seguito ad un click sul pulsante «Uno» e poi su «Due»?

```
1  <button id="b1">Uno</button>
2  <button id="b2">Due</button>
3  <script>
4    document.getElementById("b1").addEventListener("click", function () {
5      console.log(this.id);
6    });
7    document.getElementById("b2").addEventListener("click", () => {
8      console.log(this === window);
9    });
10 </script>
```

- [x] **A** `b1`, `true`.
- [ ] B `b1`, `false`.
- [ ] C `undefined`, `true`.
- [ ] D `undefined`, `false`.
- [ ] E `window`, `true`.
- [ ] F `b1`, `undefined`.

Risposta: A. In un gestore definito con `function`, `this` è l'elemento a cui il gestore è stato assegnato. Le *arrow function* non hanno un `this` proprio e usano quello del codice in cui sono scritte: qui il livello globale dello script, dove `this` è `window`. *(Dispensa: cap. 7, «Il valore di this in un gestore»)*

RACCOLTA

Es. **193.** **(2 punti)** Qual è l'output in console dopo il caricamento completo del documento HTML riportato di seguito?

```
<button>Clic</button>
<script>
  const b = document.querySelector("button");
  b.addEventListener("click", (e) => console.log(e.type, e.isTrusted));
  b.dispatchEvent(new Event("click"));
</script>
```

- **A** `click false`.
- B `click true`.
- C Non viene stampato nulla.
- D `click undefined`.
- E Viene sollevato un `TypeError`.
- F `undefined false`.

Risposta: A. Con `dispatchEvent` un evento si può generare da JavaScript: i gestori registrati per quel tipo di evento vengono eseguiti come per un evento vero. La proprietà `isTrusted` permette di distinguerli: vale `false` per gli eventi generati da uno script, `true` per quelli prodotti dall'utente. *(Dispensa: cap. 7, «Generare eventi da JavaScript»)*

Es. **194.** **(2 punti)** Quale evento scatta **appena** l'HTML della pagina è stato analizzato per intero e il DOM è completo, senza attendere che la pagina sia stata caricata e disegnata?

- **A** `DOMContentLoaded`
- B `load`
- C `ready`
- D `submit`
- E `focus`
- F `parsed`

Risposta: A. `DOMContentLoaded` scatta quando il DOM è completo; `load` più tardi, quando la pagina è stata caricata e disegnata per intero. Registrare il codice come gestore di `DOMContentLoaded` permette a uno script di trovare gli elementi della pagina ovunque sia incluso. *(Dispensa: cap. 7, «Eventi»)*

Es. **195.** **(2 punti)** Quale dei seguenti eventi **non** fa *bubbling*?

- **A** `focus`
- B `click`
- C `keydown`
- D `mouseover`
- E `keyup`
- F `mousedown`

Risposta: A. Il *bubbling* vale per la maggior parte degli eventi, ma non per tutti: gli eventi di `focus` non risalgono verso gli antenati. *(Dispensa: cap. 7, «Bubbling»)*

Es. **196.** **(2 punti)** Una tabella ha centinaia di celle `<td>`, a cui si aggiungono righe nuove durante l'uso della pagina. Perché conviene un solo gestore sulla `<table>` invece di uno per cella?

- **A** Il gestore riceve i clic di tutte le celle per *bubbling*, anche di quelle aggiunte dopo, e `event.target` dice quale cella è stata cliccata.
- B Perché i gestori registrati sulle `<td>` vengono ignorati dal browser.
- C Perché un gestore sulla `<table>` viene eseguito prima di quelli delle celle.
- D Perché `this` nel gestore della tabella è la cella cliccata.
- E Perché le celle aggiunte dopo il caricamento non generano eventi.
- F Perché con un solo gestore l'evento non fa *bubbling*.

Risposta: A. È la **delegazione degli eventi**: grazie al *bubbling* basta un gestore sull'antenato comune, e `event.target` indica l'elemento su cui l'evento è stato generato (`this` resta la tabella). Le righe aggiunte dinamicamente funzionano subito, senza registrare nuovi gestori. *(Dispensa: cap. 7, «Delegazione degli eventi»)*

RACCOLTA

Es. **197.** **(2 punti)** Si vuole mostrare in un `<div>` il nome scritto dall'utente in un campo di testo. Quale proprietà conviene assegnare, per evitare che eventuale markup nel nome venga interpretato?

- **A** `textContent`
- B `innerHTML`
- C `outerHTML`
- D `tagName`
- E `nodeName`
- F `classNames`

Risposta: A. `innerHTML` e `outerHTML` passano la stringa al parser HTML: un nome che contiene markup (per esempio un `<img onerror=...>`) verrebbe interpretato, ed è la porta per un attacco di *Cross-Site Scripting* (XSS). `textContent` (o `innerText`) inserisce soltanto testo. *(Dispensa: cap. 7, «innerHTML e outerHTML»)*

Es. **198.** **(2 punti)** Quali delle seguenti affermazioni sull'esecuzione degli script in una pagina sono **vere**?

- **A** Senza attributi, l'analisi della pagina si sospende mentre lo script viene scaricato ed eseguito.
- **B** Uno script esterno con `defer` viene scaricato in parallelo ed eseguito dopo l'analisi completa della pagina.
- **C** Uno script esterno con `async` viene eseguito appena è stato scaricato.
- **D** Gli script di tipo modulo si comportano come `defer` per impostazione predefinita.
- E Uno script con `defer` viene eseguito appena è stato scaricato, interrompendo l'analisi della pagina.
- F Uno script con `async` attende sempre l'evento `DOMContentLoaded`.
- G Uno script senza attributi nell'`<head>` può già accedere agli elementi del `<body>`.
- H Gli script `async` vengono sempre eseguiti nell'ordine in cui compaiono.
- I Senza attributi, gli script vengono eseguiti solo dopo l'evento `load`.

Risposte: A, B, C, D. Uno script normale blocca l'analisi finché non è stato scaricato ed eseguito, e vede solo la parte di pagina che lo precede. `defer` e `async` fanno scaricare lo script in parallelo: con `defer` viene eseguito a pagina analizzata, con `async` appena pronto (quindi in un momento, e in un ordine, non prevedibili). I moduli sono differiti per impostazione predefinita. *(Dispensa: cap. 7, «Ordine di esecuzione degli script»)*

Es. **199.** **(5 punti)** Si spieghino brevemente analogie e differenze tra gli attributi `async` e `defer` in un elemento `<script>`.

Traccia della risposta.

- Analogie: valgono per gli script esterni (con `src`); con entrambi lo script viene scaricato in parallelo all'analisi della pagina, senza sospenderla durante il download.
- `defer`: lo script viene eseguito dopo l'analisi completa della pagina (prima di `DOMContentLoaded`): trova il DOM completo, e più script `defer` rispettano l'ordine in cui compaiono.
- `async`: lo script viene eseguito appena è stato scaricato, anche a pagina non ancora analizzata: non si sa se il DOM sia completo, né in che ordine vengano eseguiti più script `async`.
- `defer` per gli script che usano il DOM o dipendono l'uno dall'altro; `async` per quelli indipendenti. I moduli sono differiti per impostazione predefinita.

(Dispensa: cap. 7, «Ordine di esecuzione degli script») — Fonte: prima prova intercorso del 23 aprile 2024.

Es. **200.** **(5 punti)** Che cosa sono il DOM e il BOM? Si descriva il ruolo dell'oggetto `window` e si facciano esempi di oggetti e operazioni di ciascuno.

Traccia della risposta.

- `window` è l'oggetto globale dell'ambiente del browser (funzioni e variabili `var` globali ne diventano proprietà) e rappresenta la finestra del browser.
- Il DOM rappresenta il contenuto del documento come un albero di nodi (elementi, testo, commenti) con radice `document`: serve a leggere e modificare la pagina (`querySelector`, `textContent`, `append`...).
- Il BOM comprende gli oggetti per lavorare con il browser stesso: `location` (URL), `history` (cronologia), `navigator` (browser e piattaforma), `screen` (schermo).
- Entrambi sono forniti dall'ambiente ospite, non dal linguaggio: in Node.js non esistono.

(Dispensa: cap. 7, «L'ambiente del browser»)

Es. **201.** **(5 punti)** Si spieghino le differenze fra `HTMLCollection` e `NodeList`, indicando quali metodi e proprietà le restituiscono e che cosa significa che una collezione è *viva*.

Traccia della risposta.

- `HTMLCollection` contiene solo elementi; `NodeList` può contenere nodi di qualunque tipo (anche testo e commenti).
- `getElementsByTagName`, `getElementsByClassName` e `children` restituiscono `HTMLCollection`; `querySelectorAll`, `getElementsByName` e `childNodes` restituiscono `NodeList`.
- Viva: riflette in ogni momento lo stato del documento (le `HTMLCollection`, `childNodes`); statica: una fotografia al momento della ricerca (`querySelectorAll`).
- Entrambe hanno `length` e indici e si scorrono con `for...of`, ma non sono array: per `map` o `filter` si converte con `Array.from` o con lo spread.

(Dispensa: cap. 7, «HTMLCollection e NodeList»)

Es. **202.** **(5 punti)** Si descrivano i modi per associare un gestore a un evento in una pagina web, il meccanismo del *bubbling* e il pattern della delegazione degli eventi.

Traccia della risposta.

- Attributo HTML `onclick="..."` (codice come stringa), proprietà `elem.onclick = f` (un solo gestore: un nuovo assegnamento lo sostituisce), `addEventListener("click", f)` (più gestori, rimovibili con `removeEventListener`).
- Il gestore è una callback: si passa la funzione senza chiamarla; il browser la invoca a ogni evento passandole l'oggetto evento, e `this` è l'elemento del gestore.
- *Bubbling*: dopo i gestori dell'elemento su cui è avvenuto l'evento vengono eseguiti quelli dei suoi antenati, fino a `document`; `stopPropagation()` ferma la risalita.
- Delegazione: un solo gestore sull'antenato comune, che usa `event.target` per sapere dove è avvenuto l'evento; funziona anche per gli elementi aggiunti in seguito.

(Dispensa: cap. 7, «Eventi» e «Propagazione degli eventi»)

Es. **203.** **(5 punti)** Si confrontino `innerHTML`, `outerHTML` e `textContent`, e si spieghi perché assegnare a `innerHTML` dati forniti dall'utente è pericoloso.

Traccia della risposta.

- `innerHTML` è il codice HTML contenuto nell'elemento, `outerHTML` quello dell'intero elemento (tag compresi), `textContent` solo il testo dei discendenti.
- Sono scrivibili: assegnare `innerHTML` sostituisce il contenuto, assegnare `outerHTML` l'elemento intero; la stringa passa dal parser HTML, che la interpreta (e la corregge) come farebbe con un file.
- Se la stringa contiene dati dell'utente, l'eventuale markup viene interpretato (`<script>`, attributi come `onerror`): è la via per il *Cross-Site Scripting* (XSS).
- Per inserire testo si usa `textContent` (o `innerText`), oppure si creano i nodi con `createElement` e `createTextNode`.

(Dispensa: cap. 7, «innerHTML e outerHTML»)

Capitolo 8 — Asincronismo e richieste di rete

Es. **204.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
setTimeout(() => console.log("A"), 300);
setTimeout(() => console.log("B"), 300);
console.log("C");
console.log("D");
setTimeout(() => {
  console.log("E");
  setTimeout(() => console.log("F"), 100);
}, 100);
```

- **A** C, D, E, F, A, B.
- B C, D, A, B, E, F.
- C C, D, E, A, B, F.
- D C, D, E, F, B, A.
- E C, D, F, E, A, B.
- F C, D, E, A, B.

Risposta: A. Il codice sincrono viene eseguito tutto per primo, nell'ordine. `setTimeout` ritorna subito: allo scadere del timer il browser mette la callback nella coda, e l'event loop la esegue quando lo stack è vuoto, anche con un ritardo di 0 ms. Le callback partono quindi in ordine di scadenza (a parità di ritardo, nell'ordine in cui i timer sono stati avviati); un timer avviato dentro una callback comincia a contare solo quando quella callback viene eseguita. *(Dispensa: cap. 8, «Sincronismo e asincronismo»)*

Es. **205.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
const p = new Promise((resolve, reject) => {
  setTimeout(() => resolve(5), 100);
});
p.then((v) => { console.log(v); throw new Error("ko"); })
  .finally(() => console.log("finally"))
  .catch((e) => { console.log(e.message); return 100; })
  .then((v) => { console.log(v); throw new Error("ko"); })
  .catch((e) => { console.log(e.message); return 100; })
  .then((v) => console.log(v));
console.log("sync");
```

- **A** `sync, 5, finally, ko, 100, ko, 100.`
- B `sync, 5, finally, ko.`
- C `5, finally, ko, 100, ko, 100, sync.`
- D `sync, 5, finally, undefined, 100, ko, 100.`
- E `sync, 5, finally, Error: ko, 100, ko, 100.`
- F `sync, 5, finally, ko, 100, undefined, 100.`

Risposta: A. I gestori registrati con `then`, `catch` e `finally` vengono eseguiti solo dopo il codice sincrono, anche se la Promise è già risolta. Ogni anello restituisce una nuova Promise: un `then` riceve il valore restituito dall'anello precedente; un errore sollevato (o un rifiuto) salta i `then` fino al primo `catch`, che con il suo valore di ritorno fa ripartire la catena. `finally` esegue la sua callback in ogni caso e lascia passare invariato il valore (o l'errore). *(Dispensa: cap. 8, «then, catch e finally» e «Concatenare le Promise»)*

RACCOLTA

Es. **206.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
const attesa = (ms, valore, ok) => new Promise((resolve, reject) => {
  setTimeout(() => ok ? resolve(valore) : reject(new Error(valore)), ms);
});
Promise.allSettled([
  attesa(100, "a", true),
  attesa(400, "b", true),
  attesa(200, "c", true),
])
  .then((r) => console.log("ok", r.map((x) => x.status).join(",")))
  .catch((e) => console.log("ko", e.message));
```

- **A** `ok fulfilled,fulfilled,fulfilled.`
- B `ok fulfilled,fulfilled,rejected.`
- C `ok resolved,resolved,resolved.`
- D `ok rejected,fulfilled,fulfilled.`
- E `ok fulfilled,rejected,fulfilled.`

Risposta: A. `Promise.all` si risolve con i risultati di tutte, nell'ordine dell'array (non in quello di completamento), e rifiuta appena una rifiuta. `Promise.allSettled` attende che tutte si sistemino e descrive l'esito di ciascuna (`fulfilled` o `rejected`). `Promise.race` assume l'esito della prima che si sistema, in un senso o nell'altro; `Promise.any` il valore della prima che si risolve, ignorando i rifiuti. *(Dispensa: cap. 8, «I metodi statici della classe Promise»)*

Es. **207.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
let prom = new Promise(function(resolve, reject) { setTimeout(() => { resolve("A"); }, 1000); });
prom.finally(value => console.log("B")).then(val => console.log(val))
     .catch(value => console.log("X"));
console.log("Done");
```

- **A** Done, B, A.
- B A, B, Done.
- C Done, A, B.
- D X, B, Done.
- E Done, X.
- F B, A, Done.

Risposta: A. Il codice sincrono termina per primo: `Done`. Dopo un secondo la Promise si risolve con `"A"`: `finally` esegue la sua callback (che non riceve argomenti: `value` è `undefined`) e lascia passare il valore, che arriva al `then` successivo. Il `catch` non viene eseguito, perché nessun passo fallisce. *(Dispensa: cap. 8, «then, catch e finally») — Fonte: prima prova intercorso del 23 aprile 2024.*

Es. **208.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
console.log("1");
const p = new Promise((resolve) => {
  console.log("2");
  resolve("3");
  console.log("4");
});
p.then((v) => console.log(v));
console.log("5");
```

- **A** 1, 2, 4, 5, 3.
- B 1, 2, 3, 4, 5.
- C 1, 5, 2, 4, 3.
- D 1, 2, 5, 3.
- E 1, 2, 4, 3, 5.
- F 1, 5, 2, 3, 4.

Risposta: A. L'executor viene invocato **subito**, dal costruttore: `2` e `4` sono stampati durante la creazione della Promise. `resolve` sistema la Promise ma non interrompe l'executor. Il gestore registrato con `then` viene eseguito solo dopo la fine del codice sincrono, anche se la Promise è già risolta. *(Dispensa: cap. 8, «Creare una Promise»)*

RACCOLTA

Es. **209.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
const p = new Promise((resolve, reject) => {
  resolve("primo");
  reject(new Error("errore"));
  resolve("secondo");
});
p.then((v) => console.log("ok", v))
  .catch((e) => console.log("ko", e.message));
```

- **A** `ok primo`.
- B `ok secondo`.
- C `ko errore`.
- D `ok primo`, `ko errore`.
- E Viene sollevato un `Error`.
- F `ok primo`, `ok secondo`.

Risposta: A. Una Promise che ha lasciato lo stato `pending` è *settled*, e da quel momento il suo stato non cambia più: le chiamate successive a `reject` e `resolve` vengono ignorate. *(Dispensa: cap. 8, «Creare una Promise»)*

Es. **210.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
const p = new Promise((resolve) => resolve(1));
p.resolve(42);
p.then((v) => console.log("valore", v));
console.log("fine");
```

- **A** Viene sollevato un `TypeError`.
- B `valore 42`, `fine`.
- C `fine`, `valore 42`.
- D `fine`, `valore 1`.
- E `fine`, poi viene sollevato un `TypeError`.
- F `valore 1`, `fine`.

Risposta: A. Un oggetto Promise **non ha** un metodo `resolve`: `p.resolve(42)` solleva un `TypeError` (*p.resolve is not a function*) e lo script si ferma lì. Il `resolve` che riceve l'executor è una funzione legata a quella Promise; `Promise.resolve(valore)` è un metodo statico che crea una nuova Promise già risolta. *(Dispensa: cap. 8, «Anatomia di una Promise: chi fa cosa»)*

Es. **211.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
const p = Promise.resolve(1);
p.then((r) => { console.log(r); return r * 2; });
p.then((r) => { console.log(r); return r * 2; });
p.then((r) => { console.log(r); return r * 2; });
```

- **A** 1, 1, 1.
- B 1, 2, 4.
- C 1.
- D 2, 2, 2.
- E 1, 2, 3.
- F 2, 4, 8.

Risposta: A. I tre gestori sono registrati sulla **stessa** Promise: sono indipendenti e ricevono tutti il suo valore, 1. Il valore restituito da un gestore alimenta solo la Promise restituita da quel `then`, che qui nessuno usa. Per passare il risultato da un passo al successivo bisogna concatenare: `p.then(...).then(...)`. *(Dispensa: cap. 8, «Concatenare le Promise»)*

RACCOLTA

Es. **212.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
Promise.any([
  Promise.reject(new Error("a")),
  new Promise((res, rej) => setTimeout(() => rej(new Error("b")), 100)),
]).then((v) => console.log("ok", v))
  .catch((e) => console.log(e.name, e.errors.length));
```

- [A] `AggregateError 2.`
- [B] `Error 1.`
- [C] `Error 2.`
- [D] `ok undefined.`
- [E] `AggregateError 1.`
- [F] `ok a.`

Risposta: A. `Promise.any` si risolve con la prima Promise **risolta**, ignorando i rifiuti. Se tutte rifiutano, rifiuta con un `AggregateError`, che raccoglie nella proprietà `errors` tutti gli errori (qui due). *(Dispensa: cap. 8, «I metodi statici della classe Promise»)*

Es. **213.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
function primo(v, cb) { setTimeout(() => cb(v + 1), 100); }
function secondo(v, cb) { cb(v * 10); }
primo(1, (r1) => {
  secondo(r1, (r2) => console.log("risultato", r2));
});
secondo(5, (r) => console.log("subito", r));
console.log("fine");
```

- [A] `subito 50, fine, risultato 20.`
- [B] `risultato 20, subito 50, fine.`
- [C] `fine, subito 50, risultato 20.`
- [D] `subito 50, risultato 20, fine.`
- [E] `subito 50, fine, risultato 11.`
- [F] `fine, risultato 20, subito 50.`

Risposta: A. Passare una callback non rende asincrona un'operazione: `secondo` la chiama subito, quindi `subito 50` viene stampato durante l'esecuzione del codice sincrono. `primo` invece la chiama dentro un `setTimeout`: il risultato (`(1 + 1) * 10`) arriva dopo, quando il codice sincrono è terminato. *(Dispensa: cap. 8, «Callback»)*

Es. **214.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
async function f() { return 1; }
const r = f();
console.log(typeof r, r instanceof Promise);
r.then((v) => console.log(v));
console.log("fine");
```

- [A] `object true, fine, 1.`
- [B] `number false, fine, 1.`
- [C] `object true, 1, fine.`
- [D] `function false, fine, 1.`
- [E] `number false, 1, fine.`
- [F] `number false`, poi viene sollevato un `TypeError`.

Risposta: A. Una funzione `async` restituisce **sempre** una Promise: il valore restituito (qui 1) viene avvolto in una Promise risolta. Il gestore registrato con `then` viene eseguito dopo la fine del codice sincrono. *(Dispensa: cap. 8, «async»)*

RACCOLTA

Es. **215.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
async function f() {
  console.log("B");
  const v = await new Promise((res) => setTimeout(() => res("D"), 100));
  console.log(v);
  return "E";
}
console.log("A");
f().then((x) => console.log(x));
console.log("C");
```

- [A] A, B, C, D, E.
- [B] A, C, B, D, E.
- [C] A, B, D, E, C.
- [D] A, C, D, E, B.
- [E] A, B, C, E, D.
- [F] A, B, D, C, E.

Risposta: A. La chiamata `f()` esegue subito il corpo della funzione fino al primo `await` (quindi `B` viene stampato prima di `C`). `await` sospende **solo** `f`: il resto del programma prosegue e stampa `C`. Quando la Promise si risolve, `f` riprende (`D`) e il suo valore di ritorno risolve la Promise restituita da `f()` (`E`). *(Dispensa: cap. 8, «await»)*

Es. **216.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
function carica() {
  const dati = await Promise.resolve(42);
  return dati;
}
console.log("inizio");
```

- [A] Viene sollevato un `SyntaxError`.
- [B] `inizio`.
- [C] Non viene stampato nulla.
- [D] Viene sollevato un `ReferenceError`.
- [E] `inizio`, poi viene sollevato un `TypeError`.
- [F] `inizio`, poi viene sollevato un `SyntaxError`.

Risposta: A. `await` si può usare solo dentro le funzioni `async`: qui `carica` è una funzione normale, e lo script non supera nemmeno l'analisi sintattica. Un `SyntaxError` impedisce l'esecuzione dell'intero script, anche delle righe che lo precedono o che non lo usano. *(Dispensa: cap. 8, «await»)*

Es. **217.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
async function leggi(ok) {
  if (!ok) throw new Error("rifiutata");
  return "dati";
}
async function main() {
  try {
    console.log(await leggi(true));
    console.log(await leggi(false));
    console.log("dopo");
  } catch (e) {
    console.log("errore:", e.message);
  }
}
main();
```

- [A] `dati`, `errore: rifiutata`.
- [B] `dati`, `errore: rifiutata`, `dopo`.
- [C] `dati`, `undefined`, `dopo`.
- [D] `dati`, poi viene sollevato un `Error`.
- [E] `errore: rifiutata`.
- [F] `dati`, `rifiutata`, `dopo`.

Risposta: A. Un errore sollevato in una funzione `async` diventa una Promise rifiutata; `await` su una Promise rifiutata **solleva** l'errore, che si cattura con un normale `try/catch`. L'istruzione dopo l'`await` fallito non viene eseguita. *(Dispensa: cap. 8, «async e await»)*

RACCOLTA

Es. **218.** **(2 punti)** Qual è l'output in console dopo il caricamento completo del documento HTML riportato di seguito?

```
<script>
  localStorage.setItem("visite", 1);
  localStorage.visite = Number(localStorage.getItem("visite")) + 1;
  const v = localStorage.getItem("visite");
  console.log(typeof v, v + 1, localStorage.getItem("utente"));
</script>
```

- **A** `string 21 null.`
- B `number 3 null.`
- C `string 21 undefined.`
- D `number 3 undefined.`
- E `string 11 null.`
- F `number 2 null.`

Risposta: A. `localStorage` memorizza **solo stringhe**: il numero viene convertito in stringa al salvataggio, e `getItem` restituisce `"2"`, per cui `v + 1` concatena. Le proprietà dell'oggetto sono un'alternativa a `getItem` e `setItem`. Per una chiave assente `getItem` restituisce `null`. *(Dispensa: cap. 8, «localStorage»)*

Es. **219.** **(2 punti)** Quali delle seguenti affermazioni sui meccanismi di memorizzazione nel browser sono **vere**?

- **A** I cookie vengono inviati al server in tutte le richieste verso lo stesso dominio, tramite l'header `Cookie`.
- **B** `localStorage` memorizza solo valori stringa.
- **C** I dati in `sessionStorage` sopravvivono a un aggiornamento della pagina, ma non alla chiusura della scheda.
- **D** Esiste un `localStorage` distinto per ogni combinazione di dominio, protocollo e porta.
- E I dati in `localStorage` vengono inviati al server a ogni richiesta.
- F I dati in `localStorage` vengono cancellati alla chiusura del browser.
- G I dati memorizzati nel browser non sono modificabili dall'utente, quindi il server può fidarsene.
- H `sessionStorage` è condiviso fra tutte le schede aperte sullo stesso sito.
- I IndexedDB può memorizzare soltanto coppie di stringhe.

Risposte: A, B, C, D. I cookie fanno parte di HTTP e viaggiano con le richieste; `localStorage` e `sessionStorage` no, e possono quindi contenere molti più dati (solo stringhe). `localStorage` sopravvive anche al riavvio del browser, `sessionStorage` è legato alla singola scheda. IndexedDB è un vero database. Tutti questi dati sono sul computer dell'utente, che può leggerli e modificarli: il server non deve fidarsene. *(Dispensa: cap. 8, «Memorizzare dati nel browser»)*

Es. **220.** **(2 punti)** Come vengono tipicamente impostati i cookie, e come tornano al server?

- **A** Il server li imposta con l'header `Set-Cookie` di una risposta; il browser li rimanda con l'header `Cookie` nelle richieste successive allo stesso dominio.
- B Il server li imposta con l'header `Cookie` di una risposta; il browser li rimanda con l'header `Set-Cookie`.
- C Il browser li invia al server solo quando uno script legge `document.cookie`.
- D Il server li legge direttamente dal disco dell'utente quando gli servono.
- E Il browser li invia solo nella prima richiesta di ogni sessione di navigazione.
- F Il browser li aggiunge come parametri alla query string di ogni URL.

Risposta: A. I cookie servono a superare l'assenza di stato di HTTP: il server li imposta con `Set-Cookie`, il browser li memorizza e li allega, con l'header `Cookie`, a tutte le richieste successive verso lo stesso dominio. Da JavaScript si leggono e scrivono con `document.cookie`. *(Dispensa: cap. 8, «Cookie»)*

RACCOLTA

Es. **221.** **(2 punti)** Quali delle seguenti affermazioni sull'esecuzione asincrona del codice JavaScript di una pagina web sono **vere**?

- **A** Il codice JavaScript della pagina viene eseguito su un solo thread.
- **B** Una callback di `setTimeout` viene eseguita solo quando lo stack delle chiamate è vuoto.
- **C** Un calcolo sincrono molto lungo blocca la pagina anche se si trova dentro una callback.
- **D** Mentre il timer di un `setTimeout` scorre, il thread di JavaScript può eseguire altro codice.
- E Allo scadere del timer, la callback interrompe il codice in esecuzione.
- F `setTimeout(f, 0)` esegue `f` immediatamente, prima dell'istruzione successiva.
- G Le callback asincrone vengono eseguite in parallelo, su thread diversi.
- H Il timer di `setTimeout` viene contato dal thread di JavaScript, che resta occupato fino alla scadenza.

Risposte: A, B, C, D. C'è un solo thread: le attese (timer, rete) sono gestite dal browser, che allo scadere mette la callback nella coda; l'event loop la esegue quando lo stack è vuoto. Una callback non interrompe mai il codice in esecuzione, e asincrono non vuol dire parallelo: un calcolo lungo blocca comunque tutto. *(Dispensa: cap. 8, «Sincronismo e asincronismo»)*

Es. **222.** **(2 punti)** Uno script esegue `const r = await fetch("/non-esiste");` e il server risponde con stato 404. Che cosa succede?

- **A** La Promise si risolve con un oggetto `Response` in cui `r.ok` vale `false` e `r.status` vale 404.
- B La Promise viene rifiutata, e l'`await` solleva un errore.
- C La Promise si risolve con `null`, perché la risorsa non esiste.
- D La Promise si risolve, e `r.ok` vale `true` perché il server ha risposto.
- E La Promise resta per sempre nello stato `pending`.
- F `fetch` solleva subito un `TypeError`, prima di inviare la richiesta.

Risposta: A. La Promise di `fetch` viene rifiutata solo se la risposta non arriva affatto (rete assente, server irraggiungibile). Un 404 o un 500 sono risposte HTTP a tutti gli effetti: si riconoscono controllando `response.ok` (vero solo per gli stati da 200 a 299) o `response.status`. *(Dispensa: cap. 8, «Lavorare con la risposta»)*

Es. **223.** **(2 punti)** Quali delle seguenti affermazioni sull'oggetto `Response` restituito da `fetch` sono **vere**?

- **A** La Promise di `fetch` si risolve appena arrivano gli header: il corpo può non essere ancora disponibile.
- **B** `response.json()` restituisce una Promise, che si risolve con il corpo interpretato come JSON.
- **C** Il corpo si può leggere una sola volta: dopo `response.text()`, `response.json()` non funziona.
- **D** `response.ok` vale `true` solo per gli stati da 200 a 299.
- E `response.json()` restituisce direttamente l'oggetto, senza Promise.
- F `response.ok` vale `true` per qualunque risposta del server.
- G Per leggere gli header bisogna prima aver letto il corpo.
- H `fetch` restituisce direttamente l'oggetto `Response`, senza Promise.
- I Il corpo si legge come stringa dalla proprietà `response.text`.

Risposte: A, B, C, D. `fetch` restituisce una Promise che si risolve con la `Response` quando arrivano gli header (`status`, `ok`, `headers`). Il corpo si legge dopo, con metodi che restituiscono a loro volta Promise (`text()`, `json()`, `blob()`...), e una volta sola, perché leggerlo lo consuma. *(Dispensa: cap. 8, «Lavorare con la risposta»)*

Es. **224.** **(2 punti)** Quale delle seguenti chiamate invia una richiesta `POST` con un corpo JSON?

- **A** `fetch("/api", { method: "POST", headers: { "Content-Type": "application/json" }, body: JSON.stringify(dati) })`
- B `fetch.post("/api", dati)`
- C `fetch("/api", "POST", JSON.stringify(dati))`
- D `fetch("POST /api", { body: dati })`
- E `fetch("/api", { type: "POST", data: dati })`
- F `fetch("/api").post(JSON.stringify(dati))`

Risposta: A. Il secondo argomento di `fetch` è un oggetto di opzioni: `method` per il metodo HTTP, `headers` per gli header della richiesta e `body` per il corpo, che va serializzato (qui con `JSON.stringify`) e descritto con l'header `Content-Type`. *(Dispensa: cap. 8, «L'API fetch»)*

RACCOLTA

Es. **225.** **(5 punti)** Si descriva il modello di esecuzione di JavaScript nel browser (thread, Web API, coda dei callback, event loop), spiegando con un esempio perché `setTimeout(f, 0)` non esegue `f` immediatamente.

Traccia della risposta.

- Il codice della pagina gira su un solo thread, che esegue script, gestori di eventi e aggiornamento della pagina: un'attesa sincrona blocca tutto.
- Le operazioni lente (timer, richieste di rete, eventi) sono gestite dal browser tramite le Web API: la chiamata ritorna subito e l'attesa procede fuori dal thread.
- Al termine il browser mette la callback nella coda dei callback; l'event loop la esegue quando lo stack delle chiamate è vuoto.
- Es.: `console.log("A"); setTimeout(() => console.log("B"), 0); console.log("C");` stampa A, C, B. Asincrono non vuol dire parallelo.

(Dispensa: cap. 8, «Sincronismo e asincronismo»)

Es. **226.** **(5 punti)** Che cos'è una Promise in JavaScript? Se ne descrivano gli stati, il ruolo dell'executor e delle funzioni `resolve` e `reject`, i metodi `then`, `catch` e `finally`, e il motivo per cui sono preferibili alle callback annidate.

Traccia della risposta.

- Un oggetto che collega un codice produttore (che richiede tempo) a un codice consumatore: rappresenta un risultato disponibile in futuro.
- Stati: `pending`, poi `fulfilled` (con un valore) o `rejected` (con un errore); una volta *settled* non cambia più. Stato e risultato non sono leggibili direttamente.
- L'executor, passato al costruttore, viene eseguito subito e chiama `resolve(valore)` o `reject(errore)`; il consumatore registra i gestori con `then`, `catch` e `finally` (eseguito in ogni caso, senza argomenti).
- Ogni `then` restituisce una nuova Promise (*chaining*): passi in sequenza senza annidamento, e un errore salta i `then` fino al primo `catch`, mentre con le callback annidate si arriva al *Callback Hell*.

(Dispensa: cap. 8, «Promise»)

Es. **227.** **(5 punti)** A che cosa servono le parole chiave `async` e `await`? Si spieghi il loro rapporto con le Promise e come si gestiscono gli errori.

Traccia della risposta.

- `async` davanti a una funzione fa sì che restituisca sempre una Promise: il valore restituito viene avvolto in una Promise risolta, un errore in una rifiutata.
- `await`, solo dentro le funzioni `async`, sospende la funzione finché la Promise non è sistemata, e ne restituisce il valore (o ne solleva l'errore).
- Non blocca il browser: sospende solo la funzione `async`, mentre il resto del programma prosegue.
- Gli errori si gestiscono con i normali `try/catch`; il codice asincrono si legge come codice sincrono, senza catene di `then`: è zucchero sintattico sulle Promise.

(Dispensa: cap. 8, «async e await»)

Es. **228.** **(5 punti)** Si confrontino cookie, `localStorage`, `sessionStorage` e IndexedDB. Quali dati non vi andrebbero mai memorizzati, e perché?

Traccia della risposta.

- Cookie: piccole stringhe impostate di solito dal server con `Set-Cookie` e rimandate dal browser in tutte le richieste verso lo stesso dominio; da JavaScript con `document.cookie`.
- `localStorage`: coppie chiave/valore (solo stringhe) non inviate al server, una per combinazione di dominio, protocollo e porta; sopravvivono al riavvio del browser.
- `sessionStorage`: stessa interfaccia, ma legato alla scheda (sopravvive all'aggiornamento, non alla chiusura). IndexedDB: un database nel browser, per le applicazioni offline.
- Sono sul computer dell'utente, ispezionabili e modificabili: niente segreti, e il server non deve fidarsi di questi dati (es. i prezzi di un carrello).

(Dispensa: cap. 8, «Memorizzare dati nel browser»)

Es. **229.** **(5 punti)** Si descriva l'API `fetch`: che cosa restituisce, quando la sua Promise viene risolta o rifiutata, e come si legge il corpo della risposta. Si scriva una funzione `async` che scarica un documento JSON gestendo gli errori.

Traccia della risposta.

- `fetch(url, opzioni)` avvia una richiesta HTTP (opzioni: metodo, header, corpo) e restituisce una Promise che si risolve in un oggetto `Response`.
- Si risolve appena arrivano gli header (`status`, `ok`, `headers`); viene rifiutata solo se la risposta non arriva: un 404 o un 500 la risolvono, con `ok` falso.
- Il corpo si legge con metodi che restituiscono Promise (`text()`, `json()`, `blob()`...), una volta sola.
- Es.: `const r = await fetch(url); if (!r.ok) throw new Error(r.status); return await r.json();`, dentro un `try/catch` per gli errori di rete.

(Dispensa: cap. 8, «Richieste di rete»)

Capitolo 9 — Il Web sotto il cofano

Es. **230.** **(2 punti)** Quale codice di stato dovrebbe restituire il server nella seguente situazione: la risorsa si trova **temporaneamente** a un altro indirizzo, e si deve continuare a usare quello vecchio?

- **A** `302 Found`
- B `301 Moved Permanently`
- C `304 Not Modified`
- D `404 Not Found`
- E `200 OK`
- F `503 Service Unavailable`

Risposta: A. `401` vuol dire «non so chi sei» (manca l'autenticazione), `403` «so chi sei, ma non puoi». `301` e `302` sono spostamenti definitivo e temporaneo; `304` dice che la copia in cache è ancora buona. `502` segnala una risposta non valida ricevuta da un server intermedio, `503` un server momentaneamente non disponibile. `201` accompagna la creazione di una risorsa, `204` un successo senza corpo, `405` un metodo non ammesso su un percorso esistente. *(Dispensa: cap. 9, «La risposta»)*

Es. **231.** **(2 punti)** A quale servizio è assegnata, per convenzione, la porta 53?

- **A** DNS
- B HTTP
- C HTTPS
- D SSH
- E SMTP (invio di posta)
- F FTP

Risposta: A. Alle applicazioni più diffuse sono assegnate porte note: 80 per HTTP, 443 per HTTPS, 22 per SSH, 25 per SMTP, 53 per il DNS, 21 per FTP. Per questo nelle URL `http://` e `https://` la porta si può omettere. *(Dispensa: cap. 9, «Indirizzi IP e porte»)*

Es. **232.** **(2 punti)** Quale versione di HTTP invia più richieste in parallelo come **flussi multiplexati** su un'unica connessione TCP, con messaggi in formato binario?

- **A** HTTP/2
- B HTTP/1.0
- C HTTP/1.1
- D HTTP/3
- E HTTPS

Risposta: A. HTTP/1.0 (1996) usava spesso connessioni non persistenti; HTTP/1.1 (1997) le rende persistenti per impostazione predefinita; HTTP/2 (2015) multiplexa più flussi su un'unica connessione TCP, con messaggi binari; HTTP/3 (2022) usa QUIC, su UDP. HTTPS non è una versione: è HTTP dentro un canale TLS. *(Dispensa: cap. 9, «HTTP più da vicino»)*

RACCOLTA

Es. **233.** **(2 punti)** Quali delle seguenti affermazioni sulle architetture delle applicazioni di rete sono **vere**?

- **A** Nell'architettura client-server i client non comunicano direttamente fra loro.
- **B** Nell'architettura client-server il server ha un indirizzo fisso e noto, ed è sempre acceso.
- **C** In un'applicazione peer-to-peer lo stesso processo può fare da client in uno scambio e da server in un altro.
- **D** Un servizio client-server può essere realizzato da molti server, che il client percepisce come uno solo.
- E Nell'architettura peer-to-peer c'è un host sempre acceso che coordina tutti gli scambi.
- F Nell'architettura client-server ogni client inoltra le richieste degli altri client.
- G Un'applicazione web è il tipico esempio di architettura peer-to-peer.
- H Nell'architettura peer-to-peer i nodi sono eterogenei: uno fa sempre da server.
- I Nell'architettura client-server è il server ad avviare la comunicazione.

Risposte: A, B, C, D. Nel modello client-server i nodi sono eterogenei: un server sempre acceso, con indirizzo noto, serve le richieste dei client, che non parlano fra loro (anche se dietro ci sono molti server o interi data center). Nel peer-to-peer i nodi sono idealmente omogenei e comunicano direttamente; il processo che avvia uno scambio fa da client, e i ruoli cambiano da uno scambio all'altro. *(Dispensa: cap. 9, «Client-server e peer-to-peer»)*

Es. **234.** **(2 punti)** Che cos'è un *socket*?

- **A** L'interfaccia software fra il livello applicativo e il livello di trasporto, attraverso cui un processo invia e riceve messaggi sulla rete.
- B Il cavo che collega un host al primo router della rete.
- C Il protocollo di trasporto usato da HTTP al posto di TCP.
- D Un file di configurazione che associa i nomi degli host agli indirizzi IP.
- E Il processo del server web che resta in ascolto sulla porta 80.
- F Un messaggio di controllo scambiato durante l'*handshake* di TCP.

Risposta: A. Il socket è la «cassetta postale» del processo: l'applicazione vi imbuca i messaggi, e l'infrastruttura di trasporto, controllata dal sistema operativo, li consegna al socket del destinatario. Dal lato del trasporto lo sviluppatore sceglie solo il protocollo (TCP o UDP) e pochi parametri. *(Dispensa: cap. 9, «Il socket»)*

Es. **235.** **(2 punti)** Per inviare un messaggio a un processo in esecuzione su un altro host servono un indirizzo IP e un numero di porta. A che cosa servono, rispettivamente?

- **A** L'indirizzo IP identifica l'host (livello di rete), la porta il processo su quell'host (livello di trasporto).
- B L'indirizzo IP identifica il processo, la porta l'host.
- C L'indirizzo IP identifica l'utente, la porta il browser che usa.
- D Entrambi identificano l'host: la porta serve solo quando l'indirizzo IP è privato.
- E L'indirizzo IP identifica il sito, la porta la pagina richiesta.
- F La porta identifica la scheda di rete dell'host, l'indirizzo IP il router.

Risposta: A. Su un host girano molte applicazioni di rete: l'indirizzo IP (32 bit in IPv4, 128 in IPv6) trova la macchina, il numero di porta (da 0 a 65535) trova il processo. Anche il browser usa una porta, assegnata al momento dal sistema operativo, per ricevere la risposta. *(Dispensa: cap. 9, «Indirizzi IP e porte»)*

RACCOLTA

Es. **236.** **(2 punti)** Che cosa stabilisce un **protocollo di livello applicativo**?

- **A** I tipi di messaggio scambiati, per esempio richieste e risposte.
- **B** La sintassi di ciascun tipo di messaggio: i campi e come sono delimitati.
- **C** Il significato delle informazioni contenute nei campi.
- **D** Le regole su quando e come un processo invia messaggi e risponde.
- E Il percorso dei pacchetti attraverso i router della rete.
- F Il linguaggio di programmazione con cui scrivere client e server.
- G La codifica elettrica dei bit sul cavo.
- H L'indirizzo IP da assegnare a ciascun host.

Risposte: A, B, C, D. Un protocollo applicativo (HTTP, DNS, SMTP...) definisce tipi, sintassi e semantica dei messaggi e le regole dello scambio. L'instradamento, la trasmissione fisica e l'assegnazione degli indirizzi sono compito di altri livelli e di altri protocolli (per esempio DHCP per gli indirizzi). *(Dispensa: cap. 9, «I protocolli applicativi»)*

Es. **237.** **(2 punti)** Quali delle seguenti affermazioni sul DNS sono **vere**?

- **A** È un database distribuito su migliaia di server, non un unico server centrale.
- **B** I nomi sono organizzati in un albero: dalla radice ai TLD come `.it`, poi ai domini di secondo livello come `unina.it`.
- **C** Resolver e dispositivi tengono in cache le risposte per un tempo stabilito, il TTL.
- **D** Il browser di norma interroga un *resolver*, che scende nell'albero per suo conto.
- E Tutte le richieste di risoluzione arrivano a un unico server centrale.
- F Il server *root* conosce gli indirizzi IP di tutti gli host di Internet.
- G Il DNS assegna gli indirizzi IP agli host quando si collegano alla rete.
- H Un nome si legge da sinistra a destra, dalla radice verso l'host.
- I Le risposte del DNS non vengono mai memorizzate, per essere sempre aggiornate.

Risposte: A, B, C, D. Il DNS è una rubrica distribuita e gerarchica: la radice sa chi gestisce ciascun TLD, il server del TLD sa chi gestisce ciascun dominio, il server autoritativo conosce gli indirizzi. Le risposte restano in cache per la durata del TTL. Gli indirizzi agli host li assegna DHCP. Un nome si legge da destra a sinistra risalendo verso la radice. *(Dispensa: cap. 9, «Dalla URL all'indirizzo: il DNS»)*

Es. **238.** **(2 punti)** Il resolver deve trovare l'indirizzo di `www.unina.it` e non ha nulla in cache. A quali server chiede, nell'ordine?

- **A** Al server *root*, al server del TLD `.it`, al server autoritativo di `unina.it`.
- B Al server autoritativo di `unina.it`, al server del TLD `.it`, al server *root*.
- C Al server del TLD `.it`, al server *root*, al server autoritativo di `unina.it`.
- D Soltanto al server *root*, che conosce tutti gli indirizzi.
- E Al server autoritativo di `www.unina.it`, poi al server *root* per conferma.
- F A un server DHCP, che gli assegna l'indirizzo di `www.unina.it`.

Risposta: A. Il resolver scende nell'albero un livello alla volta: il server root non conosce l'indirizzo, ma sa chi gestisce `.it`; il server del TLD sa chi gestisce `unina.it`; il server autoritativo di `unina.it` conosce l'indirizzo. La risposta resta poi in cache per la durata del TTL. *(Dispensa: cap. 9, «Dalla URL all'indirizzo: il DNS»)*

RACCOLTA

Es. **239.** **(2 punti)** Quali delle seguenti affermazioni su TCP e UDP sono **vere**?

- **A** TCP è orientato alla connessione: prima di scambiare dati i processi fanno un *handshake*.
- **B** TCP offre un trasferimento affidabile: i dati arrivano integri e in ordine.
- **C** UDP non garantisce che i messaggi arrivino, né in che ordine.
- **D** HTTP/3 usa UDP, insieme al protocollo QUIC che aggiunge l'affidabilità.
- E UDP è più lento di TCP perché ritrasmette i pacchetti persi.
- F TCP non richiede alcun *handshake* iniziale.
- G HTTP ha sempre usato soltanto UDP.
- H UDP garantisce la consegna in ordine, ma non l'integrità dei dati.
- I Con TCP lo sviluppatore deve gestire a mano ritrasmissioni e perdite.

Risposte: A, B, C, D. TCP è orientato alla connessione e affidabile, al prezzo di handshake, conferme e ritrasmissioni; UDP è leggero e senza garanzie. HTTP ha usato storicamente TCP per l'affidabilità; HTTP/3 usa QUIC, che implementa l'affidabilità sopra UDP. In entrambi i casi l'applicazione delega questi problemi ai livelli inferiori. *(Dispensa: cap. 9, «TCP o UDP»)*

Es. **240.** **(2 punti)** Quali dei seguenti sono vantaggi di QUIC (e quindi di HTTP/3) rispetto a TCP?

- **A** Apertura della connessione e negoziazione crittografica avvengono insieme, di solito in un solo *round trip*.
- **B** La perdita di un pacchetto blocca solo il flusso a cui appartiene, non le altre richieste.
- **C** È implementato di solito nel software applicativo, quindi si aggiorna più facilmente.
- D Non usa conferme né ritrasmissioni, quindi non perde mai tempo.
- E Non cifra i dati, e per questo è più veloce.
- F Usa una connessione TCP separata per ogni richiesta.
- G Elimina la necessità del DNS.
- H Trasmette i messaggi HTTP come semplice testo ASCII.

Risposte: A, B, C. QUIC combina handshake di trasporto e TLS (un round trip, zero riprendendo una connessione precedente), usa flussi separati evitando il *head-of-line blocking* di TCP, e vive nel software applicativo invece che nel kernel. Come TCP usa conferme e timer per ritrasmettere i dati persi, e integra TLS. *(Dispensa: cap. 9, «QUIC»)*

Es. **241.** **(2 punti)** Con HTTP/2 tre richieste viaggiano in parallelo sulla stessa connessione TCP, e si perde un pacchetto della prima. Che cosa succede?

- **A** Anche i dati delle altre due richieste che arrivano dopo restano fermi finché il pacchetto non viene ritrasmesso.
- B Si ferma solo la prima richiesta; le altre due vengono consegnate subito.
- C La connessione viene chiusa e le tre richieste ripartono da capo.
- D Il pacchetto perso viene ignorato e la prima risposta arriva incompleta.
- E Il browser apre tre nuove connessioni, una per richiesta.
- F Il server rimanda tutte e tre le risposte su UDP.

Risposta: A. TCP vede un unico flusso di byte da consegnare in ordine: un pacchetto perso blocca anche i dati che seguono, di qualunque richiesta siano. È il *head-of-line blocking*. HTTP/3 lo evita con flussi QUIC separati: si blocca solo il flusso a cui appartiene il pacchetto perso. *(Dispensa: cap. 9, «QUIC»)*

RACCOLTA

Es. **242.** **(2 punti)** Una pagina richiede tre oggetti allo stesso server. Che cosa si risparmia usando una connessione **persistente** invece di connessioni non persistenti?

- **A** L'apertura di due connessioni: tutte le richieste e risposte viaggiano sulla stessa.
- B Due delle tre richieste: il server invia tutti gli oggetti con una sola risposta.
- C La risoluzione DNS, che con le connessioni persistenti non serve.
- D La cifratura TLS, che si applica solo alle connessioni non persistenti.
- E Nulla: le connessioni persistenti servono solo a mantenere il login.
- F Gli header delle risposte, che vengono inviati una volta sola.

Risposta: A. Con connessioni non persistenti (HTTP/1.0) ogni coppia richiesta–risposta apre una connessione nuova; con quelle persistenti, predefinite da HTTP/1.1 (`Connection: keep-alive`), le richieste successive riusano la stessa, risparmiando le aperture (e le negoziazioni TLS). *(Dispensa: cap. 9, «Connessioni persistenti e non persistenti»)*

Es. **243.** **(2 punti)** Quali dei seguenti metodi HTTP sono **sicuri** (*safe*)?

- **A** `GET`
- **B** `HEAD`
- **C** `OPTIONS`
- **D** `QUERY`
- E `POST`
- F `PUT`
- G `DELETE`
- H `PATCH`

Risposte: A, B, C, D. Un metodo è sicuro se è di sola lettura: il client non chiede alcuna modifica sul server. Lo sono `GET`, `HEAD`, `OPTIONS` e `QUERY`; `POST`, `PUT`, `PATCH` e `DELETE` chiedono invece di modificare lo stato. *(Dispensa: cap. 9, «Metodi sicuri e idempotenti»)*

Es. **244.** **(2 punti)** Quali dei seguenti metodi HTTP sono **idempotenti ma non sicuri**?

- **A** `PUT`
- **B** `DELETE`
- C `POST`
- D `GET`
- E `HEAD`
- F `OPTIONS`
- G `QUERY`

Risposte: A, B. Un metodo è idempotente se l'effetto di più richieste identiche è lo stesso di una sola. I metodi sicuri lo sono tutti; fra quelli che modificano lo stato lo sono `PUT` (il nuovo stato completo) e `DELETE` (la risorsa resta cancellata). `POST` non è né sicuro né idempotente. *(Dispensa: cap. 9, «Metodi sicuri e idempotenti»)*

Es. **245.** **(2 punti)** Per un doppio clic, la stessa richiesta arriva due volte al server. In quali dei seguenti casi lo stato del server, dopo la seconda, è lo stesso che dopo la prima?

- **A** `PUT /api/compiti/42` con lo stato completo del compito.
- **B** `DELETE /api/compiti/42`.
- **C** `PATCH /api/compiti/42` con `{"scadenza": "2026-11-12"}`, che imposta un valore.
- **D** `GET /api/compiti/42`.
- E `POST /api/compiti` con i dati di un nuovo compito.
- F `PATCH /api/compiti/42` che aggiunge un elemento alla lista degli allegati.
- G `POST /api/ordini` con il contenuto del carrello.
- H `PATCH /api/contatori/7` che incrementa un contatore.

Risposte: A, B, C, D. `PUT` e `DELETE` sono idempotenti (la seconda `DELETE` riceverà probabilmente `404`, ma la risorsa resta cancellata), e `GET` non modifica nulla. Una `POST` ripetuta crea un duplicato. Per `PATCH` dipende dal documento di patch: impostare un valore è idempotente, aggiungere o incrementare no. *(Dispensa: cap. 9, «I metodi, uno per uno»)*

BOZZA

Es. **246.** **(2 punti)** Qual è la differenza fra `PUT` e `PATCH`?

- **A** `PUT` invia lo stato completo che la risorsa deve assumere; `PATCH` applica una modifica parziale allo stato attuale.
- B `PUT` crea sempre una nuova risorsa; `PATCH` la cancella.
- C `PUT` è sicuro; `PATCH` no.
- D `PATCH` invia lo stato completo; `PUT` solo i campi da modificare.
- E `PUT` si usa solo per i file; `PATCH` solo per i documenti JSON.
- F Nessuna: sono due nomi dello stesso metodo.

Risposta: A. Il corpo di una `PUT` contiene il nuovo stato completo della risorsa all'URI indicato (che viene creata o sostituita): ripeterla non cambia nulla, quindi è idempotente. `PATCH` descrive solo una modifica parziale, e la sua idempotenza dipende dal tipo di modifica. *(Dispensa: cap. 9, «I metodi, uno per uno»)*

Es. **247.** **(2 punti)** Un browser ha in cache una risorsa ricevuta con l'header `ETag: "abc123"`. Come può chiedere al server se è ancora valida, senza riscaricarla se non è cambiata?

- **A** Inviando la richiesta con `If-None-Match: "abc123"`: se la risorsa non è cambiata il server risponde `304 Not Modified`, senza corpo.
- B Inviando `If-None-Match: "abc123"`: se la risorsa non è cambiata il server risponde `404 Not Found`.
- C Inviando una `DELETE` della copia vecchia e poi una nuova `GET`.
- D Inviando l'header `ETag: "abc123"`: il server risponde sempre `200 OK` con il corpo.
- E Inviando `Cache-Control: no-store`: il server risponde `204 No Content`.
- F Non può: deve sempre riscaricare la risorsa.

Risposta: A. Il server associa a ogni versione di una risorsa un'etichetta, l'`ETag`. Il client la rimanda con `If-None-Match`: se la risorsa non è cambiata, il server risponde `304 Not Modified` senza ritrasmetterla (non è un errore: «la tua copia è ancora buona»). *(Dispensa: cap. 9, «I metodi, uno per uno»)*

Es. **248.** **(2 punti)** Perché è stato introdotto il metodo HTTP `QUERY`?

- **A** Per inviare un'interrogazione complessa nel corpo della richiesta mantenendo una semantica di sola lettura: sicura, idempotente e memorizzabile in cache.
- B Per sostituire `GET`, che è stato deprecato.
- C Per modificare parzialmente una risorsa, come `PATCH` ma più veloce.
- D Per interrogare il DNS direttamente da HTTP.
- E Per permettere a `POST` di essere messa in cache.
- F Per inviare query SQL direttamente al database del server.

Risposta: A. Con `GET` l'interrogazione deve stare tutta nella query string, che ha limiti di lunghezza e finisce nei log; con una `POST` si può usare il corpo, ma la richiesta non è più né sicura né idempotente. `QUERY` unisce le due cose: un corpo come la `POST`, la semantica di sola lettura della `GET`. *(Dispensa: cap. 9, «I metodi, uno per uno»)*

Es. **249.** **(2 punti)** Prima di inviare una `POST` con corpo JSON a un altro dominio, il browser invia automaticamente una richiesta `OPTIONS`. A che cosa serve?

- **A** È il *preflight* CORS: il browser chiede al server se quella richiesta è permessa dall'origine della pagina.
- B A scaricare l'elenco delle opzioni del form da inviare.
- C A cifrare il corpo della richiesta prima dell'invio.
- D A chiedere al server di creare la risorsa in anticipo.
- E A misurare il tempo di risposta del server.
- F A cancellare i cookie del dominio di destinazione.

Risposta: A. `OPTIONS` chiede quali opzioni di comunicazione sono disponibili per una risorsa; è sicuro e idempotente. I browser lo usano per il *preflight* CORS: prima di una richiesta da un'altra origine chiedono al server se metodo e header sono permessi. *(Dispensa: cap. 9, «I metodi, uno per uno»)*

RACCOLTA

Es. **250.** **(2 punti)** Quali garanzie offre TLS a una comunicazione HTTPS?

- [**A**] Riservatezza: chi intercetta i dati lungo il percorso vede solo byte cifrati.
- [**B**] Autenticazione del server, tramite un certificato firmato da un'autorità di cui il browser si fida.
- [**C**] Integrità: il destinatario si accorge se i dati sono stati modificati in transito.
- [D] Che il gestore del sito sia onesto e affidabile.
- [E] L'anonimato del client nei confronti del server.
- [F] Che il server non subisca mai attacchi.
- [G] Che la comunicazione sia più veloce di HTTP.
- [H] Che i dati siano compressi.

Risposte: A, B, C. HTTPS è lo stesso HTTP dentro un canale TLS, che garantisce riservatezza, autenticazione del server e integrità. Non dice nulla sull'onestà di chi gestisce il sito: anche i siti di *phishing* hanno certificati validi e mostrano il lucchetto. *(Dispensa: cap. 9, «HTTPS: HTTP dentro un canale TLS»)*

Es. **251.** **(2 punti)** Nella sezione *Tempistiche* della scheda *Rete* dei DevTools, una richiesta ha una fase *In attesa* molto lunga. Che cosa indica?

- [**A**] Che il server ha impiegato molto tempo a elaborare la richiesta prima di inviare il primo byte della risposta.
- [B] Che la risoluzione DNS è stata lenta.
- [C] Che la negoziazione TLS è fallita.
- [D] Che la risposta era molto grande da scaricare.
- [E] Che la richiesta è rimasta in coda nel browser perché non c'era connessione.
- [F] Che il browser ha impiegato molto tempo a disegnare la pagina.

Risposta: A. *In attesa* è il tempo fra l'invio della richiesta e l'arrivo del primo byte della risposta (*Time To First Byte*): il tempo di elaborazione del server. DNS, connessione e TLS hanno barre proprie, e il tempo per scaricare il corpo è la *Ricezione*. *(Dispensa: cap. 9, «Dal clic alla pagina: il viaggio completo»)*

Es. **252.** **(2 punti)** Quando si fa clic su un link `https://` verso un sito mai visitato, in che ordine avvengono i passi seguenti?

- [**A**] Analisi della URL, DNS, connessione (TCP o QUIC), TLS, richiesta, risposta.
- [B] DNS, analisi della URL, TLS, connessione, richiesta, risposta.
- [C] Analisi della URL, connessione, DNS, richiesta, TLS, risposta.
- [D] Analisi della URL, TLS, DNS, connessione, richiesta, risposta.
- [E] Connessione, DNS, analisi della URL, richiesta, TLS, risposta.
- [F] Analisi della URL, DNS, richiesta, connessione, TLS, risposta.

Risposta: A. Dalla URL il browser ricava schema, host, porta e percorso; il DNS traduce il nome in un indirizzo IP; solo allora può aprire la connessione verso IP e porta, negoziare TLS (certificato e chiavi di sessione) e infine inviare la richiesta e ricevere la risposta. *(Dispensa: cap. 9, «Dal clic alla pagina: il viaggio completo»)*

Es. **253.** **(5 punti)** Si confrontino TCP e UDP e si spieghi perché le versioni recenti di HTTP usano QUIC. Quali problemi di TCP risolve?

Traccia della risposta.

- TCP: orientato alla connessione (handshake), affidabile (dati integri e in ordine), più lento per conferme e ritrasmissioni. UDP: senza connessione e senza garanzie, leggero.
- HTTP ha usato TCP per l'affidabilità; HTTP/3 usa QUIC, che implementa l'affidabilità sopra UDP.
- QUIC combina apertura della connessione e TLS (un round trip, zero riprendendo una connessione) ed evita l'*head-of-line blocking* con flussi separati.
- È implementato nel software applicativo, non nel kernel: si aggiorna più facilmente. Il modo di scrivere le richieste HTTP non cambia.

(Dispensa: cap. 9, «HTTP più da vicino»)

RACCOLTA

Es. **254.** **(5 punti)** Che cosa vuol dire che un metodo HTTP è *sicuro* e che è *idempotente*? Si classifichino i metodi principali e si spieghi perché queste proprietà sono utili.

Traccia della risposta.

- Sicuro: di sola lettura, non chiede modifiche (`GET`, `HEAD`, `OPTIONS`, `QUERY`).
- Idempotente: più richieste identiche hanno lo stesso effetto di una sola (tutti i sicuri, più `PUT` e `DELETE`); conta lo stato del server, non la risposta.
- `POST` non è né sicuro né idempotente; per `PATCH` dipende dal tipo di modifica.
- Browser, proxy e librerie possono ripetere da soli le richieste idempotenti e mettere in cache le risposte alle richieste sicure; una `POST` ripetuta può creare un duplicato (e una `GET` non deve mai modificare lo stato).

(Dispensa: cap. 9, «Metodi sicuri e idempotenti»)

Es. **255.** **(5 punti)** Si descriva il funzionamento del DNS: com'è organizzato lo spazio dei nomi, chi risponde a una richiesta di risoluzione e come si evita di sovraccaricare la rete.

Traccia della risposta.

- Traduce i nomi degli host in indirizzi IP: le persone ricordano nomi, la rete instrada verso indirizzi.
- Database distribuito e gerarchico: radice, TLD (`.it`, `.com`), domini di secondo livello e sottodomini; nessun server centrale.
- Il browser chiede a un resolver, che interroga il server root, poi quello del TLD, poi l'autoritativo del dominio, che conosce l'indirizzo.
- I risultati restano in cache (resolver e dispositivi) per la durata del TTL.

(Dispensa: cap. 9, «Dalla URL all'indirizzo: il DNS»)

Es. **256.** **(5 punti)** Si descrivano i passi che il browser compie dal clic su un link `https://` alla visualizzazione della risposta, indicando quali si possono evitare nelle visite successive.

Traccia della risposta.

- Analisi della URL (schema, host, porta, percorso); risoluzione DNS del nome in indirizzo IP.
- Apertura della connessione (TCP o QUIC) verso IP e porta; negoziazione TLS (certificato, chiavi di sessione).
- Invio della richiesta (metodo, destinazione, header, corpo); attesa ed elaborazione del server; ricezione della risposta (stato, header, corpo).
- Visite successive: DNS in cache, connessione persistente già aperta (niente connessione né TLS), risorse in cache verificate con `ETag` e `304`.

(Dispensa: cap. 9, «Dal clic alla pagina: il viaggio completo»)

Capitolo 10 — Il lato server del Web

Es. **257.** **(2 punti)** Secondo la specifica CGI, come arriva al programma esterno la *query string* della richiesta?

- [A] Come **variabili d'ambiente**.
- [B] Sullo **standard input** del programma.
- [C] È ciò che il programma scrive sullo **standard output**.
- [D] Come argomenti sulla riga di comando del programma.
- [E] In un file temporaneo il cui nome è passato al programma.
- [F] Tramite un cookie impostato dal server.

Risposta: A. La specifica CGI stabilisce come server e programma comunicano: header e query string arrivano come variabili d'ambiente, il corpo della richiesta sullo standard input; ciò che il programma scrive sullo standard output (header, riga vuota, corpo) diventa la risposta. *(Dispensa: cap. 10, «CGI: Common Gateway Interface»)*

Es. **258.** **(2 punti)** Un server serve soltanto i file della sua *document root*. Quali delle seguenti esigenze **non** si possono soddisfare in questo modo?

- [A] Personalizzare una pagina per l'utente che la richiede.
- [B] Registrare un ordine in seguito all'invio di un form.
- [C] Gestire milioni di pagine con la stessa struttura senza duplicarla in ogni file.
- [D] Servire un foglio di stile CSS condiviso da tutte le pagine.
- [E] Servire immagini e script JavaScript.
- [F] Aggiornare il contenuto della pagina nel browser con JavaScript.
- [G] Servire la stessa pagina a tutti gli utenti, in HTTPS.

Risposte: A, B, C. Un server statico restituisce sempre lo stesso documento: niente personalizzazione, nessuna azione in seguito a una richiesta, e una struttura comune da ripetere in ogni file (per Wikipedia, terabyte di dati duplicati e milioni di file da rigenerare per cambiare la barra di navigazione). Il dinamismo di JavaScript avviene dentro il browser. *(Dispensa: cap. 10, «I limiti delle web app statiche»)*

Es. **259.** **(2 punti)** Che cos'è CGI (*Common Gateway Interface*)?

- [A] Una specifica che permette a un server web di eseguire programmi esterni per elaborare le richieste HTTP.
- [B] Un linguaggio di programmazione per scrivere pagine dinamiche.
- [C] Un motore di template per generare HTML.
- [D] Il protocollo con cui il browser invia i dati dei form.
- [E] Un'estensione di JavaScript eseguita sul server.
- [F] Un formato binario per le risposte HTTP.

Risposta: A. CGI (primi anni Novanta) è una specifica di interfaccia: definisce come il server passa gli input a un programma esterno, scritto in qualunque linguaggio (anche Bash), e come ne raccoglie l'output per costruire la risposta. *(Dispensa: cap. 10, «CGI: Common Gateway Interface»)*

Es. **260.** **(2 punti)** Lo script CGI seguente stampa una riga vuota dopo l'header `Content-type`. Perché?

```
1 #!/bin/bash
2 echo "Content-type: text/html; charset=utf-8"
3 echo ""
4 echo "<!DOCTYPE html>"
5 echo "<html><head><title>Hello CGI!</title></head><body>"
6 echo "<h1>Hello CGI!</h1>"
7 date +"<p>%d/%m/%Y alle %H:%M:%S</p></body></html>"
```

- [A] Perché la riga vuota separa gli header dal corpo, come in ogni risposta HTTP.
- [B] Per far andare a capo il testo nel browser.
- [C] Perché la specifica CGI richiede una riga vuota alla fine dell'output.
- [D] Per segnalare al server che lo script è terminato.
- [E] Per svuotare il buffer dello standard output.
- [F] Non serve: è solo una scelta di stile.

Risposta: A. Il programma CGI scrive a mano la risposta: prima gli header, poi una riga vuota, poi il corpo. È esattamente la struttura di una risposta HTTP. *(Dispensa: cap. 10, «CGI: Common Gateway Interface»)*

RACCOLTA

Es. **261.** **(2 punti)** Qual è il motivo principale per cui CGI è inefficiente su un sito con molto traffico?

- **A** Per ogni richiesta viene creato un nuovo processo, e il costo di avvio supera quello dell'elaborazione.
- B Perché i programmi CGI possono essere scritti solo in Bash.
- C Perché CGI non può leggere il corpo delle richieste `POST`.
- D Perché ogni programma CGI deve aprire una nuova connessione TCP con il browser.
- E Perché CGI trasmette le risposte in formato binario.
- F Perché le pagine generate con CGI non possono essere messe in cache.

Risposta: A. Oltre a essere scomodo (HTML prodotto con delle `echo`, query string e corpo da analizzare a mano), CGI crea e distrugge un processo per ogni richiesta. Le architetture successive tengono invece un pool di processi già pronti. *(Dispensa: cap. 10, «CGI: Common Gateway Interface»)*

Es. **262.** **(2 punti)** Si consideri la seguente pagina PHP. Che cosa riceve il browser che la richiede?

```
1 <ul>
2   <?php $array = ["Hello", "PHP"] ?>
3   <?php foreach($array as $item): ?>
4     <li><?= $item ?></li>
5   <?php endforeach; ?>
6 </ul>
```

- **A** Solo l'HTML generato: una lista con due `<li>`, senza alcuna traccia del codice PHP.
- B Il file così com'è: il codice PHP viene eseguito dal browser.
- C L'HTML della pagina, con il ciclo `foreach` da eseguire con JavaScript.
- D Una lista vuota: il ciclo viene eseguito solo dopo il caricamento.
- E Solo il codice fra `<?php` e `?>`, senza l'HTML.
- F Un errore, perché il browser non conosce PHP.

Risposta: A. Il codice fra i delimitatori `<?php ... ?>` viene eseguito **sul server**, e il suo output inserito nell'HTML; il testo fuori dai delimitatori resta invariato (`<?=` abbrevia `<?php echo`). Al browser arriva HTML puro. *(Dispensa: cap. 10, «PHP»)*

Es. **263.** **(2 punti)** In PHP, come si legge il valore del campo `todo` di un form inviato con il metodo `POST`?

- **A** `$_POST['todo']`
- B `$_GET['todo']`
- C `$_COOKIE['todo']`
- D `$request.body.todo`
- E `getenv('todo')`
- F `$_POST.todo()`

Risposta: A. Uno dei vantaggi dello scripting lato server è la pre-elaborazione della richiesta: in PHP i parametri sono già negli array associativi `$_GET` e `$_POST`, e i cookie in `$_COOKIE`. *(Dispensa: cap. 10, «PHP»)*

Es. **264.** **(2 punti)** In un'architettura con NGINX e PHP-FPM, quali delle seguenti affermazioni sono **vere**?

- **A** NGINX inoltra a PHP-FPM, tramite una connessione TCP, le richieste per i file `.php`.
- **B** NGINX serve da sé i file statici (immagini, CSS, JavaScript).
- **C** I processi di PHP-FPM esistono già e restano vivi fra una richiesta e l'altra.
- **D** Se tutti i worker di PHP-FPM sono occupati, le richieste attendono in coda.
- E Per ogni richiesta PHP-FPM crea un nuovo processo, come CGI.
- F PHP-FPM serve anche i file statici, che NGINX non sa gestire.
- G NGINX esegue direttamente il codice PHP.
- H Il browser invia le richieste per i file `.php` direttamente a PHP-FPM, sulla porta 9000.

Risposte: A, B, C, D. Il salto rispetto a CGI è il **pool** di processi già pronti, riusati fra una richiesta e l'altra. NGINX riceve tutte le richieste, serve i file statici e inoltra a PHP-FPM (sulla porta TCP 9000) solo quelle per i file `.php`; le richieste in eccesso attendono in coda. *(Dispensa: cap. 10, «Un'architettura moderna: NGINX e PHP-FPM»)*

RACCOLTA

Es. **265.** **(5 punti)** Che cos'è CGI? Si descriva come un server web e un programma CGI si scambiano i dati di una richiesta e della risposta, e quali sono i limiti di questo approccio.

Traccia della risposta.

- Specifica che permette al server web di eseguire programmi esterni per elaborare le richieste e generare pagine dinamiche.
- Header e query string arrivano al programma come variabili d'ambiente, il corpo sullo standard input; l'output su standard output (header, riga vuota, corpo) diventa la risposta.
- Limiti: HTML costruito a mano, richiesta da analizzare a mano.
- Soprattutto: un nuovo processo per ogni richiesta, il cui costo di avvio domina su un sito con molto traffico.

(Dispensa: cap. 10, «CGI: Common Gateway Interface»)

Es. **266.** **(5 punti)** Si descriva l'architettura basata su NGINX e PHP-FPM, spiegando in che cosa migliora l'approccio CGI.

Traccia della risposta.

- NGINX riceve tutte le richieste HTTP e serve da sé i file statici.
- Le richieste per i file `.php` vengono inoltrate, via TCP (porta 9000), a PHP-FPM.
- PHP-FPM mantiene un pool di processi worker già avviati, riusati fra le richieste; se sono tutti occupati le richieste attendono in coda.
- Rispetto a CGI non si crea un processo per richiesta; lo scripting lato server pre-elabora la richiesta (`$_GET`, `$_POST`, `$_COOKIE`) e mescola codice e HTML.

(Dispensa: cap. 10, «Un'architettura moderna: NGINX e PHP-FPM»)

Es. **267.** **(5 punti)** Quali limiti ha un sito composto da sole pagine statiche? Si spieghi, anche con un esempio, che cosa cambia quando le pagine vengono generate lato server.

Traccia della risposta.

- Una pagina statica è un file della *document root*: a ogni richiesta il server invia sempre lo stesso documento (JavaScript aggiunge dinamismo solo nel browser).
- Non si può personalizzare la pagina per l'utente, i contenuti che cambiano spesso vanno riscritti a mano e non si può *agire* in seguito a una richiesta (per esempio registrare un ordine).
- Con milioni di pagine la struttura comune si ripete in ogni file: per Wikipedia (circa 60 milioni di pagine) sono circa 9 TB di dati duplicati, e cambiare la barra di navigazione vorrebbe dire rigenerare tutti i file.
- Con la programmazione lato server il server genera la pagina al volo, in base ai parametri della richiesta e ai dati: la stessa URL può produrre risposte diverse (CGI, PHP, Node.js).

(Dispensa: cap. 10, «I limiti delle web app statiche» e «La programmazione lato server»)

Capitolo 11 — Node.js

Es. **268.** **(2 punti)** Qual è l'output dell'esecuzione del seguente modulo con Node.js? Il file `dati.txt` esiste e contiene la stringa `ciao`.

```
1 import fs from "fs";
2
3 console.log("inizio");
4 fs.readFile("dati.txt", "utf8", (err, testo) => console.log("asincrono:", testo));
5 const t = fs.readFileSync("dati.txt", "utf8");
6 console.log("sincrono:", t);
7 console.log("fine");
```

- **A** `inizio`, `sincrono: ciao`, `fine`, `asincrono: ciao`.
- B `inizio`, `asincrono: ciao`, `sincrono: ciao`, `fine`.
- C `inizio`, `sincrono: ciao`, `asincrono: ciao`, `fine`.
- D `inizio`, `fine`, `asincrono: ciao`, `sincrono: ciao`.
- E `inizio`, `fine`, `sincrono: ciao`, `asincrono: ciao`.
- F `inizio`, `sincrono: undefined`, `fine`, `asincrono: ciao`.

Risposta: A. `fs.readFile` è asincrona: avvia la lettura, che procede fuori dal thread di JavaScript, e ritorna subito; la callback viene eseguita dall'event loop dopo la fine del codice sincrono. `fs.readFileSync` invece blocca l'esecuzione finché il file non è stato letto, e restituisce il contenuto. In Node.js le operazioni sincrone sono l'eccezione. *(Dispensa: cap. 11, «Il modello di Node.js»)*

Es. **269.** **(2 punti)** Qual è l'output dell'esecuzione del seguente frammento con Node.js?

```
1 setTimeout(() => console.log("timer"), 0);
2 const fine = Date.now() + 300;
3 while (Date.now() < fine) { }      // un calcolo lungo e sincrono
4 console.log("calcolo finito");
```

- **A** `calcolo finito`, `timer`.
- B `timer`, `calcolo finito`.
- C `calcolo finito`.
- D `timer`.
- E Non viene stampato nulla.

Risposta: A. Il codice JavaScript gira in un solo thread: un calcolo lungo e sincrono occupa l'event loop, e la callback del timer, pur scaduta, può essere eseguita solo quando lo stack si svuota. Su un server, lo stesso calcolo bloccherebbe **tutte** le richieste, non solo la propria. *(Dispensa: cap. 11, «Il modello di Node.js»)*

Es. **270.** **(2 punti)** Quali delle seguenti affermazioni su Node.js sono **vere**?

- **A** È un ambiente di esecuzione JavaScript basato sul motore V8 di Google.
- **B** Adotta un modello di I/O basato sugli eventi e non bloccante.
- **C** Il codice JavaScript di un'applicazione viene eseguito in un solo thread.
- **D** Le operazioni di I/O vere e proprie sono affidate a worker interni al runtime.
- E Crea un nuovo thread per ogni richiesta HTTP che riceve.
- F È un framework per scrivere applicazioni web, come Express.
- G È un linguaggio di programmazione derivato da Java.
- H Rende parallelo il codice JavaScript sui core della CPU.
- I Mette a disposizione gli oggetti `window` e `document`.

Risposte: A, B, C, D. Node.js è un *runtime* JavaScript (motore V8) con I/O asincrono e non bloccante. Il nostro codice gira in un unico thread, l'event loop; l'I/O è delegato ai worker del runtime, e la sua callback torna all'event loop. Express è un framework costruito su Node.js; DOM e BOM non esistono fuori dal browser. *(Dispensa: cap. 11, «Che cos'è Node.js»)*

Es. **271.** **(2 punti)** In un application server che dedica un thread del pool a ogni richiesta, perché il modello è inefficiente quando le richieste fanno molto I/O (database, file, API esterne)?

- **A** Perché i thread passano gran parte del tempo bloccati ad aspettare la fine delle operazioni di I/O, e il pool limita il numero di richieste servibili insieme.
- B Perché i thread non possono accedere al database contemporaneamente.
- C Perché ogni thread deve compilare di nuovo il codice dell'applicazione.
- D Perché i thread condividono un unico socket e devono fare a turno.
- E Perché il sistema operativo esegue un solo thread per volta su ogni processore.
- F Perché ogni thread apre una nuova connessione TCP verso il client.

Risposta: A. Nel modello multi-thread un thread resta legato alla sua richiesta anche mentre aspetta l'I/O (il cameriere fermo al tavolo). Il pool ha dimensione fissa: le richieste concorrenti non possono superarla. Node.js usa invece un solo thread che non aspetta mai: l'I/O procede altrove e la callback arriva come evento. *(Dispensa: cap. 11, «Il modello multi-thread»)*

Es. **272.** **(2 punti)** Quali dei seguenti oggetti e moduli sono disponibili in un programma Node.js?

- **A** Il modulo `fs`, per leggere e scrivere file.
- **B** Il modulo `http`, per creare un server.
- **C** La funzione `setTimeout`.
- **D** L'oggetto `console`.
- E L'oggetto `document`.
- F L'oggetto `window`.
- G L'oggetto `localStorage`.
- H La funzione `alert`.

Risposte: A, B, C, D. Node.js e il browser condividono il linguaggio, non l'ambiente: in Node mancano il DOM, il BOM e le API della piattaforma web (`document`, `window`, `localStorage`, `alert`), ma c'è una libreria standard con moduli per il filesystem e la rete. Timer e console esistono in entrambi. *(Dispensa: cap. 11, «Differenze rispetto all'ambiente browser»)*

RACCOLTA

Es. **273.** **(2 punti)** Nel `package.json` di un progetto la proprietà `scripts` contiene l'attività `"build": "node build.js"`. Con quale comando la si esegue?

- **A** `npm run build`
- B `npm build`
- C `node build`
- D `npm install build`
- E `npm start build`
- F `node package.json build`

Risposta: A. Le attività definite in `scripts` si eseguono con `npm run <nome>`; solo per quelle standard, come `start` e `test`, basta `npm <nome>`. *(Dispensa: cap. 11, «Creare un pacchetto»)*

Es. **274.** **(2 punti)** Quali delle seguenti affermazioni su npm e sul file `package.json` sono **vere**?

- **A** `npm install <pacchetto>` aggiunge il pacchetto alla sezione `dependencies`.
- **B** Le dipendenze vengono scaricate nella cartella `node_modules`, insieme alle loro dipendenze.
- **C** `"type": "module"` abilita la sintassi `import`/`export` dei moduli ES.
- **D** La cartella `node_modules` non va messa sotto controllo di versione: si ricostruisce con `npm install`.
- E `node_modules` va versionata, perché senza non si può ricostruire il progetto.
- F `npm install` scarica solo il pacchetto richiesto, non le sue dipendenze.
- G `package.json` contiene il codice dell'applicazione.
- H `"main"` indica il comando da eseguire con `npm start`.
- I Le dipendenze vengono installate nella cartella del sistema, comune a tutti i progetti.

Risposte: A, B, C, D. `package.json` descrive il pacchetto (come il `pom.xml` di Maven): nome, versione, file principale (`main`), tipo di moduli, attività (`scripts`) e dipendenze. npm scarica le dipendenze, ricorsivamente, in `node_modules`, che si ricostruisce da `package.json` e per questo resta fuori dal controllo di versione. *(Dispensa: cap. 11, «npm: il gestore di pacchetti»)*

Es. **275.** **(2 punti)** Si consideri il seguente server. Quando viene eseguita la funzione passata a `createServer`?

```
1 import http from 'http';
2
3 let server = http.createServer(function (request, response) {
4   response.writeHead(200, { "Content-Type": "text/html" });
5   response.end(`<p>Sono le ${new Date().toTimeString()}</p>`);
6 });
7 server.listen(3000);
```

- **A** Ogni volta che arriva una richiesta: è una callback, invocata dall'event loop.
- B Una sola volta, all'avvio del server.
- C Una volta al secondo, finché il server resta acceso.
- D Solo per la prima richiesta: le successive ricevono la stessa risposta.
- E Quando viene chiamata `listen`, per ciascuna porta.
- F Solo per le richieste `POST`.

Risposta: A. `createServer` registra la funzione come gestore delle richieste: l'event loop la invoca a ogni richiesta, e la pagina viene costruita ogni volta (qui con l'ora corrente). Non c'è alcun file HTML sul disco. *(Dispensa: cap. 11, «Una prima applicazione web con Node.js»)*

Es. **276.** **(2 punti)** A che cosa serve `nodemon`?

- **A** A riavviare automaticamente il server quando cambia il codice sorgente.
- B A installare e cambiare le versioni di Node.js.
- C A monitorare il traffico di rete del server.
- D A eseguire il server in più processi paralleli.
- E A scaricare le dipendenze mancanti all'avvio.
- F A mostrare le variabili in scope durante il debug.

Risposta: A. Durante lo sviluppo, `nodemon app.js` al posto di `node app.js` sorveglia i file (`js`, `mjs`, `cjs`, `json`) e riavvia il processo a ogni modifica (*live reloading*). Per le versioni di Node si usano gestori come `nvm`. *(Dispensa: cap. 11, «Ricaricamento automatico»)*

RACCOLTA

Es. **277.** **(5 punti)** Quali sono gli svantaggi (se ve ne sono) dell'utilizzo di un approccio *multi-threaded request handling* per un'applicazione web che effettua frequenti accessi in lettura/scrittura a un database? Motivare la risposta.

Traccia della risposta.

- Ogni richiesta occupa un thread (o processo) del pool per tutta la sua durata, anche mentre attende le operazioni di I/O sul database.
- Con accessi frequenti i thread passano la maggior parte del tempo bloccati: inefficienza (memoria e risorse spese per aspettare).
- Scalabilità limitata: il pool ha dimensione fissa, e non si servono più richieste concorrenti di quanti sono i thread (il ristorante con un cameriere per tavolo).
- Alternativa: modello a eventi con I/O non bloccante (Node.js): un thread non aspetta mai, l'I/O procede altrove e la sua callback torna all'event loop.

(Dispensa: cap. 11, «Il modello multi-thread») — Fonte: prima prova intercorso del 23 aprile 2024.

Es. **278.** **(5 punti)** Si descriva il modello di esecuzione di Node.js, spiegando per quali applicazioni è adatto e per quali no.

Traccia della risposta.

- Un unico processo: il codice JavaScript gira in un solo thread, l'event loop, che preleva un evento alla volta dalla coda e ne esegue il gestore.
- Le operazioni di I/O sono asincrone: vengono affidate ai worker del runtime, e l'event loop passa subito all'evento successivo; a operazione conclusa la callback torna in coda.
- Adatto ai carichi dominati dall'I/O (molte richieste che aspettano database, file, rete): un solo thread ne serve moltissime.
- Poco adatto ai carichi dominati dalla CPU: un calcolo lungo e sincrono blocca l'event loop e quindi tutte le richieste.

(Dispensa: cap. 11, «Il modello di Node.js»)

Es. **279.** **(5 punti)** Che cos'è npm? Si descrivano il ruolo del file `package.json` e della cartella `node_modules`, e si spieghi perché `node_modules` non va aggiunta al controllo di versione.

Traccia della risposta.

- npm è il gestore di pacchetti di Node.js, installato insieme al runtime, con il più grande repository di pacchetti per un singolo linguaggio.
- `package.json` (creato da `npm init`) descrive il pacchetto: nome, versione, file principale (`main`), `"type": "module"` per usare `import`/`export`, gli `scripts` eseguibili con `npm run <nome>` (`npm start`, `npm test`) e le dipendenze.
- `npm install <pacchetto>` aggiunge la dipendenza a `dependencies` e la scarica in `node_modules`, insieme alle sue dipendenze (ricorsivamente).
- `node_modules` si ricostruisce con `npm install` a partire da `package.json`: basta versionare il proprio codice e il descrittore.

(Dispensa: cap. 11, «npm: il gestore di pacchetti»)

Capitolo 12 — Applicazioni web con Node.js

RACCOLTA

Es. **280.** **(2 punti)** Si consideri la seguente applicazione Node.js, in ascolto sulla porta 3000. Quali risposte (codice di stato e corpo) ottengono, nell'ordine, le richieste `DELETE /`, `GET /Todo` e `POST /todo`?

```
import http from "http";

http.createServer((request, response) => {
  switch (request.url) {
    case "/":
      response.writeHead(200);
      response.end("Home");
      break;
    case "/todo":
      if (request.method === "GET") {
        response.writeHead(200);
        response.end("La lista");
      } else {
        response.writeHead(405);
        response.end("Metodo non supportato");
      }
      break;
    default:
      response.writeHead(404);
      response.end("Pagina non trovata");
  }
}).listen(3000);
```

- **A** `200 Home, 404 Pagina non trovata, 405 Metodo non supportato.`
- B `405 Metodo non supportato, 404 Pagina non trovata, 405 Metodo non supportato.`
- C `200 Home, 200 La lista, 405 Metodo non supportato.`
- D `200 Home, 404 Pagina non trovata, 404 Pagina non trovata.`
- E `200 La lista, 404 Pagina non trovata, 405 Metodo non supportato.`
- F `200 Home, 404 Pagina non trovata, 200 La lista.`

Risposta: A. Lo `switch` confronta `request.url` con stringhe **esatte**, e `request.url` contiene anche la query string: `/todo?ordina=si`, `/todo/` e `/Todo` non sono `/todo` e finiscono nel ramo 404. Il metodo viene controllato solo per `/todo`: lì ogni metodo diverso da `GET` riceve 405 (il percorso esiste, il metodo no); `/` risponde a qualunque metodo. *(Dispensa: cap. 12, «Instradare in base al percorso» e «Instradare anche in base al metodo»)*

Es. **281.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
function decodifica(corpo) {
  let dati = {};
  for (let pezzo of corpo.split("&")) {
    let [nome, valore] = pezzo.split("=");
    nome = decodeURIComponent(nome.replace(/\+/g, " "));
    valore = decodeURIComponent(valore.replace(/\+/g, " "));
    dati[nome] = valore;
  }
  return dati;
}
const dati = decodifica("todo=a%2Bb%3Dc&quando=fra+due+giorni");
console.log(dati.todo);
console.log(dati.quando);
```

- **A** `a+b=c, fra due giorni.`
- B `a b=c, fra due giorni.`
- C `a%2Bb%3Dc, fra due giorni.`
- D `a%2Bb%3Dc, fra+due+giorni.`
- E `a+b=c, fra+due+giorni.`

Risposta: A. Il corpo di un form è codificato come URL: gli spazi diventano `+` e i caratteri speciali sequenze `%XX` (`%2B` è il `+`, `%3D` l'`=`, `%26` la `&`, `%25` il `%`). Per decodificarlo bisogna sostituire i `+` con spazi **e poi** applicare `decodeURIComponent`: saltare un passaggio, o farli nell'ordine sbagliato, produce errori che compaiono solo con certi input (un `+` scritto dall'utente diventerebbe uno spazio). *(Dispensa: cap. 12, «Interpretare la stringa»)*

RACCOLTA

Es. **282.** **(2 punti)** Si consideri il template Pug `pagina.pug` riportato di seguito, trasformato in HTML con la chiamata a destra. Quale codice HTML viene generato?

```
ul
  each item in items
    li #{item.nome} (#{item.prezzo})
  else
    li Nessun prodotto
```

```
pug.renderFile("pagina.pug", {
  items: [
    { nome: "Mele", prezzo: 4 },
    { nome: "Pane", prezzo: 2 },
  ]
});
```

A `<ul><li>Mele (4)</li><li>Pane (2)</li></ul>`

B `<ul><li>#{item.nome} (#{item.prezzo})</li></ul>`

C `<ul><li>Mele (4)</li><li>Pane (2)</li><li>Nessun prodotto</li></ul>`

D `<ul><li>Pane (2)</li><li>Mele (4)</li></ul>`

E `<ul></ul>`

F `<li>Mele (4)</li><li>Pane (2)</li>`

Risposta: A. `each` ripete il blocco annidato per ogni elemento della collezione, nell'ordine; il ramo `else` viene usato **solo** se la collezione è vuota. `#{...}` valuta l'espressione e ne inserisce il valore. Pug produce HTML compatto, senza spazi fra i tag. *(Dispensa: cap. 12, «Iterazioni»)*

Es. **283.** **(2 punti)** Si consideri il template Pug `pagina.pug` riportato di seguito, trasformato in HTML con la chiamata a destra. Quale codice HTML viene generato?

```
if !user
  a(href="/login") Accedi
else if user.isAdmin
  p Ai tuoi ordini, #{user.name}
else
  p Bentornato, #{user.name}
```

```
pug.renderFile("pagina.pug", {
  user: { name: "Ada", isAdmin: true }
});
```

A `<p>Ai tuoi ordini, Ada</p>`

B `<p>Bentornato, Ada</p>`

C `<p>Bentornato, </p>`

D `<p>Ai tuoi ordini, Ada</p><p>Bentornato, Ada</p>`

E `<a href="/login">Accedi</a>`

F `<a href="/login">Accedi</a><p>Ai tuoi ordini, Ada</p><p>Bentornato, Ada</p>`

Risposta: A. In un condizionale Pug finisce nell'HTML **solo** il ramo scelto. Se `user` manca, `!user` è vero e si mostra il link di accesso (le espressioni degli altri rami non vengono nemmeno valutate); altrimenti si sceglie in base a `user.isAdmin`. *(Dispensa: cap. 12, «Condizionali»)*

RACCOLTA

Es. **284.** **(2 punti)** Il template `pagina.pug` (a destra) estende `layout.pug` (a sinistra). Quale codice HTML genera `pagina.pug`?

```
1 html
2   body
3     block intestazione
4       h1 Il mio sito
5     block contenuto
6     block piede
7       footer Predefinito
```

```
1 extends layout.pug
2
3 block piede
4   footer Contatti
5 block contenuto
6   p Benvenuto!
7 block intestazione
8   h1 Offerte
```

A `<html><body><h1>Offerte</h1><p>Benvenuto!</p><footer>Contatti</footer></body></html>`

B `<html><body><footer>Contatti</footer><p>Benvenuto!</p><h1>Offerte</h1></body></html>`

C `<html><body><h1>Il mio sito</h1><footer>Predefinito</footer></body></html>`

D `<html><body><h1>Il mio sito</h1><h1>Offerte</h1><p>Benvenuto!</p><footer>Predefinito</footer><footer>Contatti</footer></body></html>`

E `<html><body><h1>Il mio sito</h1><footer>Predefinito</footer><footer>Contatti</footer><p>Benvenuto!</p><h1>Offerte</h1></body></html>`

Risposta: A. Con `extends` la pagina figlia riprende il layout e **sostituisce** i blocchi che ridefinisce, nella posizione che hanno nel layout; i blocchi che non ridefinisce mantengono il contenuto predefinito (un blocco vuoto nel layout, se non ridefinito, non produce nulla). *(Dispensa: cap. 12, «Ereditarietà fra template»)*

Es. **285.** **(2 punti)** Si consideri il template Pug `pagina.pug` riportato di seguito, trasformato in HTML con la chiamata a destra. Quale codice HTML viene generato?

```
1 - let titolo = "Prodotti"
2 h1 #{titolo.toUpperCase()}
3 p Offerta: #{descrizione}
```

```
1 pug.renderFile("pagina.pug", {
2   descrizione: "<b>sconto</b>"
3 });
```

A `<h1>PRODOTTI</h1><p>Offerta: <b>sconto</b></p>`

B `<h1>PRODOTTI</h1><p>Offerta: <b>sconto</b></p>`

C `- let titolo = "Prodotti"<h1>PRODOTTI</h1><p>Offerta: <b>sconto</b></p>`

D `<h1>#{titolo.toUpperCase()}</h1><p>Offerta: #{descrizione}</p>`

E `<h1>Prodotti</h1><p>Offerta: <b>sconto</b></p>`

F `<h1>PRODOTTI</h1><p>Offerta: sconto</p>`

Risposta: A. Una riga che inizia con `-` contiene codice JavaScript eseguito ma non stampato. Il codice fra `#{` e `}` viene valutato e il risultato **sottoposto a escaping**: i caratteri `<` e `>` diventano `<` e `>`, e il markup contenuto nei dati arriva al browser come testo, non come HTML (è ciò che impedisce l'iniezione di script). *(Dispensa: cap. 12, «Locals: passare i dati al template»)*

Es. **286.** **(2 punti)** Quale codice HTML genera il seguente template Pug?

```
1 nav#menu
2   a.attivo(href="/") Home
3   a(href="/todo") La lista
```

A `<nav id="menu"><a class="attivo" href="/">Home</a><a href="/todo">La lista</a></nav>`

B `<nav class="menu"><a id="attivo" href="/">Home</a><a href="/todo">La lista</a></nav>`

C `<nav id="menu"><a class="attivo" href="/">Home<a href="/todo">La lista</a></a></nav>`

D `<nav#menu><a.attivo href="/">Home</a><a href="/todo">La lista</a></nav#menu>`

E `<nav id="menu"></nav><a class="attivo" href="/">Home</a><a href="/todo">La lista</a>`

F `<nav id="menu"><a class="attivo" href="/" Home></a><a href="/todo" La lista></a></nav>`

Risposta: A. Come nei selettori CSS, `#` abbrevia l'attributo `id` e `.` l'attributo `class`; gli altri attributi si scrivono fra parentesi tonde, e il testo dopo il tag ne è il contenuto. L'indentazione stabilisce l'annidamento: i due link, allo stesso livello, sono fratelli dentro il `<nav>`. *(Dispensa: cap. 12, «Pug»)*

RACCOLTA

Es. **287.** **(2 punti)** Si consideri la seguente applicazione Node.js, in ascolto sulla porta 3000. Quali risposte (codice di stato e corpo) ottengono, nell'ordine, le richieste `GET /todo`, `POST /todo` (con corpo `todo=Pane`), `POST /todo` (con corpo `todo=Latte`) e `GET /todo`?

```
import http from "http";

let todos = [];
http.createServer((request, response) => {
  if (request.url === "/todo" && request.method === "POST") {
    let corpo = "";
    request.on("data", (pezzo) => { corpo += pezzo; });
    request.on("end", () => {
      todos.push(new URLSearchParams(corpo).get("todo"));
      response.writeHead(303, { "Location": "/todo" });
      response.end();
    });
  } else if (request.url === "/todo") {
    response.writeHead(200);
    response.end(todos.join(", ") || "Lista vuota");
  } else {
    response.writeHead(404);
    response.end("Pagina non trovata");
  }
}).listen(3000);
```

- **A** `200 Lista vuota`, `303 -> /todo`, `303 -> /todo`, `200 Pane, Latte`.
- B `200 Lista vuota`, `200 Pane`, `200 Pane, Latte`, `200 Pane, Latte`.
- C `200 Lista vuota`, `303 -> /todo`, `303 -> /todo`, `200 Latte`.
- D `200 ,303 -> /todo`, `303 -> /todo`, `200 Pane, Latte`.
- E `200 Lista vuota`, `201 Pane`, `201 Latte, 200 Pane, Latte`.
- F `200 Lista vuota`, `404 Pagina non trovata`, `404 Pagina non trovata`, `200 Lista vuota`.

Risposta: A. Il gestore della `POST` legge il corpo (che arriva a pezzi, con gli eventi `data` ed `end`), salva l'elemento e risponde con un reindirizzamento `303` verso `/todo`: è il pattern *Post/Redirect/Get*, grazie al quale ricaricare la pagina non reinvia il form. L'array `todos` resta in memoria fra una richiesta e l'altra, finché il server non viene riavviato. *(Dispensa: cap. 12, «Mettere insieme i pezzi»)*

Es. **288.** **(2 punti)** Un'applicazione gestisce `GET` e `POST` sul percorso `/todo`. Quale risposta è più corretta per una richiesta `DELETE /todo`?

- **A** `405 Method Not Allowed`
- B `404 Not Found`
- C `400 Bad Request`
- D `200 OK` con la lista
- E `204 No Content`
- F `500 Internal Server Error`

Risposta: A. Quando il percorso esiste ma il metodo non è previsto, la risposta corretta è `405`, non `404`: client e *proxy* sanno distinguere i due casi, e la distinzione torna utile soprattutto nelle API REST. *(Dispensa: cap. 12, «Instradare anche in base al metodo»)*

Es. **289.** **(2 punti)** Un router confronta `request.url` con la stringa `"/delete"`. Perché una richiesta `GET /delete?id=3` non viene riconosciuta, e come si rimedia?

- **A** `request.url` contiene anche la query string; si separano percorso e parametri con `new URL(request.url, ...)`, usando `pathname` e `searchParams`.
- B `request.url` contiene anche il nome dell'host; si rimedia con `request.url.toLowerCase()`.
- C Le richieste con parametri arrivano sempre con il metodo `POST`.
- D Node.js scarta le richieste con la query string; bisogna usare un form.
- E `request.url` vale `undefined` quando c'è una query string.
- F Il router dovrebbe confrontare `request.method` invece di `request.url`.

Risposta: A. Per `/delete?id=3`, `request.url` vale proprio `"/delete?id=3"`, che non è uguale a `"/delete"`. La classe `URL` separa il percorso (`url.pathname`) dai parametri (`url.searchParams.get("id")`). *(Dispensa: cap. 12, «Estendere l'applicazione: cancellare un elemento»)*

RACCOLTA

Es. **290.** **(2 punti)** Per cancellare un elemento, un'applicazione mostra accanto a ogni voce un link `<a href="/delete?id=3">`. Perché è una cattiva idea?

- **A** Perché una `GET` dovrebbe essere sicura: browser che precaricano i link, crawler o estensioni potrebbero cancellare elementi senza che nessuno l'abbia chiesto.
- B Perché i link non possono contenere parametri nella query string.
- C Perché il server non può leggere i parametri di una `GET`.
- D Perché una `GET` non può ricevere come risposta una pagina HTML.
- E Perché l'identificativo dovrebbe essere la posizione dell'elemento nell'array.
- F Non è una cattiva idea: le `GET` sono il metodo giusto per cancellare.

Risposta: A. Le operazioni che modificano lo stato del server vanno affidate a metodi non sicuri: qui un form con metodo `POST` e l'identificativo in un campo nascosto (o, in un'API, una `DELETE`). *(Dispensa: cap. 12, «Estendere l'applicazione: cancellare un elemento»)*

Es. **291.** **(2 punti)** Che problema risolve il pattern *Post/Redirect/Get*?

- **A** Ricaricando la pagina mostrata dopo l'invio di un form, il browser non reinvia la `POST` (e non salva di nuovo i dati), perché la pagina è il risultato di una `GET`.
- B Rende più veloce l'invio dei form, perché il corpo viaggia nell'URL.
- C Impedisce che i dati del form vengano intercettati in rete.
- D Permette di inviare un form senza ricaricare la pagina.
- E Trasforma le `GET` in `POST` per renderle sicure.
- F Evita che il server debba leggere il corpo della richiesta.

Risposta: A. Dopo una `POST` il server risponde con un reindirizzamento (`303 See Other`, o `302`) verso la pagina da mostrare; il browser la chiede con una `GET`, che si può ricaricare quante volte si vuole. *(Dispensa: cap. 12, «Mettere insieme i pezzi»)*

Es. **292.** **(2 punti)** In un server scritto con il modulo `http` di Node.js, come si legge il corpo di una richiesta `POST`?

- **A** Raccogliendo i pezzi con l'evento `data` della richiesta e ricomponendoli quando arriva l'evento `end`.
- B Leggendo la proprietà `request.body`, già pronta quando parte il gestore.
- C Chiamando `request.read()`, che attende in modo sincrono tutto il corpo.
- D Dalla query string di `request.url`.
- E Dall'header `Content-Length`, che contiene i dati.
- F Con `fs.readFile`, perché il corpo viene salvato su un file temporaneo.

Risposta: A. Il corpo potrebbe non essere ancora arrivato quando il gestore parte. `request` è uno stream leggibile: emette `data` per ogni pezzo e `end` alla fine. Per esporre il risultato si può avvolgere il tutto in una Promise. *(Dispensa: cap. 12, «Il corpo arriva a pezzi»)*

Es. **293.** **(2 punti)** Nei link per cancellare gli elementi di una lista si usa un identificativo (`/delete?id=7`) invece della posizione nell'array. Perché?

- **A** Perché dopo una cancellazione le posizioni degli elementi successivi cambiano, e un link generato prima punterebbe all'elemento sbagliato.
- B Perché la posizione nell'array non si può scrivere in un URL.
- C Perché gli array di JavaScript non hanno indici numerici.
- D Perché l'identificativo è più corto della posizione.
- E Perché la posizione è un dato sensibile da non mostrare.
- F Non c'è differenza: è solo una convenzione.

Risposta: A. Un identificativo resta legato all'elemento; la posizione no. Un identificativo che non esiste più produce invece, giustamente, un `404`. *(Dispensa: cap. 12, «Estendere l'applicazione: cancellare un elemento»)*

RACCOLTA

Es. **294.** **(2 punti)** Quali delle seguenti affermazioni sui motori di template (come Pug) sono **vere**?

- **A** Permettono di descrivere la struttura della pagina in un file separato, con dei segnaposto riempiti con i dati a tempo di esecuzione.
- **B** Con `include` e con l'ereditarietà (`extends` e `block`) le parti comuni si scrivono una volta sola.
- **C** In Pug l'indentazione sostituisce i tag di chiusura.
- **D** I dati interpolati con `#{...}` vengono sottoposti a escaping.
- E Il template viene inviato al browser, che lo trasforma in HTML.
- F Pug è l'unico motore di template disponibile per Node.js.
- G In Pug gli attributi si scrivono fra parentesi quadre.
- H Un template non può contenere condizionali né cicli.
- I Il ramo `else` di un `each` viene eseguito dopo l'ultimo elemento.

Risposte: A, B, C, D. Il template engine genera l'HTML sul server, combinando il modello con i *locals*. Pug ha una sintassi sensibile agli spazi (attributi fra parentesi tonde), supporta inclusioni, ereditarietà, condizionali e cicli (con `else` per le collezioni vuote). Altri motori: EJS, Nunjucks, LiquidJS... *(Dispensa: cap. 12, «Motori di template»)*

Es. **295.** **(5 punti)** Realizzando un'applicazione web con il solo modulo `http` di Node.js bisogna affrontare tre problemi: routing, templating e analisi del corpo delle richieste. Si descriva ciascuno e come lo si può risolvere.

Traccia della risposta.

- Routing: scegliere il codice che serve la richiesta in base a percorso (`pathname`, senza la query) e metodo; `404` se il percorso non esiste, `405` se il metodo non è previsto. Con uno `switch` diventa difficile con parametri nel percorso e controlli di accesso.
- Templating: generare l'HTML da un modello con segnaposto e dati (Pug: inclusioni, ereditarietà, locals con escaping, condizionali, cicli), invece di concatenare stringhe.
- Corpo: arriva a pezzi (eventi `data` ed `end`), va ricomposto e decodificato (`&`, `=`, `+` come spazio, `decodeURIComponent`).
- Dopo una `POST` conviene rispondere con un reindirizzamento (Post/Redirect/Get); le `GET` non devono modificare lo stato.

(Dispensa: cap. 12, «Routing», «Motori di template» e «Analizzare il corpo delle richieste»)

Es. **296.** **(5 punti)** Che cos'è un motore di template? Si descrivano, con piccoli esempi in Pug, l'interpolazione dei dati, i condizionali, le iterazioni e l'ereditarietà fra template.

Traccia della risposta.

- Genera l'HTML combinando un modello con segnaposto e i dati (*locals*) passati a `render`/`renderFile`; evita di concatenare stringhe e favorisce il riuso.
- Interpolazione: `h1 #{product.name}` valuta l'espressione e ne fa l'escaping; le righe che iniziano con `-` sono codice eseguito ma non stampato.
- Condizionali (`if`, `else if`, `else`: solo il ramo scelto finisce nell'HTML) e iterazioni (`each item in items`, con `else` per la collezione vuota).
- Ereditarietà: il layout definisce i `block`, la pagina con `extends` ridefinisce quelli che le servono; gli altri mantengono il contenuto predefinito. Con `include` si inseriscono frammenti comuni.

(Dispensa: cap. 12, «Motori di template»)

Es. **297.** **(5 punti)** Nella lista di cose da fare realizzata con Node.js, che cosa succede se, dopo aver aggiunto un elemento con il form, l'utente ricarica la pagina? Si descriva il pattern *Post/Redirect/Get* e si spieghi perché cancellare un elemento con un semplice collegamento (`/delete?id=3`) non è una buona idea.

Traccia della risposta.

- La pagina mostrata è la risposta alla `POST`: ricaricandola, il browser chiede se reinviare il form, e l'elemento verrebbe salvato due volte.
- Post/Redirect/Get: il gestore della `POST` salva e risponde con un reindirizzamento (`303` e `Location: /todo`); il browser chiede da solo la lista con una `GET`, che si può ricaricare senza effetti.
- Una `GET` è un metodo sicuro: non deve modificare lo stato del server. Crawler, precaricamento dei link o estensioni potrebbero «visitare» `/delete?id=3` e cancellare l'elemento.
- La versione corretta usa un form con `method="POST"` e l'identificativo in un campo nascosto; il router deve guardare il `pathname` (`new URL(...)`), perché `request.url` contiene anche la query.

(Dispensa: cap. 12, «Mettere insieme i pezzi» ed «Estendere l'applicazione: cancellare un elemento»)

Capitolo 13 — Gestione delle sessioni

Es. **298.** **(2 punti)** Si consideri la seguente applicazione Node.js, in ascolto sulla porta 3000, che memorizza lo stato di autenticazione nei cookie. Nessuno ha mai effettuato il login. Quali risposte (codice di stato e corpo) ottengono, nell'ordine, le richieste `GET /todo` (senza cookie), `GET /todo` con l'header `Cookie: auth=true; username=bob` e `GET /todo` con l'header `Cookie: username=luigi`?

```
1  import http from "http";
2
3  function cookie(request) {
4    const lista = {};
5    for (const c of (request.headers.cookie || "").split(";")) {
6      const [nome, valore] = c.trim().split("=");
7      if (nome) lista[nome] = valore;
8    }
9    return lista;
10 }
11
12 http.createServer((request, response) => {
13   const c = cookie(request);
14   if (c.auth === "true") {
15     response.writeHead(200);
16     response.end("Lista di " + c.username);
17   } else {
18     response.writeHead(401);
19     response.end("Non autorizzato");
20   }
21 }).listen(3000);
```

- **A** `401 Non autorizzato`, `200 Lista di bob`, `401 Non autorizzato`.
- B `401 Non autorizzato`, `401 Non autorizzato`, `401 Non autorizzato`.
- C `401 Non autorizzato`, `401 Non autorizzato`, `200 Lista di luigi`.
- D `200 Lista di undefined`, `200 Lista di bob`, `200 Lista di luigi`.
- E `401 Non autorizzato`, `200 Lista di bob`, `200 Lista di luigi`.
- F `401 Non autorizzato`, `403 Forbidden`, `401 Non autorizzato`.

Risposta: A. Il server si fida di ciò che il browser gli rimanda: chiunque può impostarsi da solo i cookie `auth=true` e `username=bob` (bastano i DevTools) e ottenere la lista di bob **senza mai passare dal login**. È il difetto critico dell'approccio a cookie: i dati sono interamente controllati dal client. *(Dispensa: cap. 13, «I problemi dell'approccio a cookie»)*

RACCOLTA

Es. **299.** **(2 punti)** Si consideri la seguente applicazione Node.js, in ascolto sulla porta 3000, che conserva le sessioni sul server. Quali risposte (codice di stato e corpo) ottengono, nell'ordine, tre richieste `GET /todo` con gli header `Cookie: sessionId=9b1d6bcd-bbfd`, `Cookie: auth=true; username=bob` e `Cookie: sessionId=42`?

```
import http from "http";

const sessioni = new Map();        // la sessione creata da un login di luigi
sessioni.set("9b1d6bcd-bbfd", new Map([["username", "luigi"]]));

function sessionId(request) {
  const m = (request.headers.cookie || "").match(/sessionId=([^;]+)/);
  return m ? m[1] : undefined;
}

http.createServer((request, response) => {
  const sessione = sessioni.get(sessionId(request));
  if (sessione) {
    response.writeHead(200);
    response.end("Lista di " + sessione.get("username"));
  } else {
    response.writeHead(401);
    response.end("Non autorizzato");
  }
}).listen(3000);
```

- **A** `200 Lista di luigi`, `401 Non autorizzato`, `401 Non autorizzato`.
- B `200 Lista di luigi`, `200 Lista di bob`, `401 Non autorizzato`.
- C `200 Lista di luigi`, `200 Lista di bob`, `200 Lista di luigi`.
- D `200 Lista di luigi`, `401 Non autorizzato`, `200 Lista di undefined`.
- E `401 Non autorizzato`, `401 Non autorizzato`, `401 Non autorizzato`.
- F `200 Lista di 9b1d6bcd-bbfd`, `401 Non autorizzato`, `401 Non autorizzato`.

Risposta: A. Con le sessioni i dati restano sul server: il client possiede solo un identificatore opaco. Un cookie `username` inventato non conta nulla, e un identificatore che non corrisponde a una sessione esistente non trova alcun dato (`get` restituisce `undefined`). *(Dispensa: cap. 13, «Implementare le sessioni da zero»)*

Es. **300.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
function parseCookies(header) {
  const list = {};
  header.split(";").forEach((cookie) => {
    let [name, ...rest] = cookie.split("=");
    name = name.trim();
    const value = rest.join("=").trim();
    if (!name || !value) return;
    list[name] = decodeURIComponent(value);
  });
  return list;
}
const c = parseCookies("auth=true; username=luigi; tema=a=b; vuoto=");
console.log(Object.keys(c).length, c.username, c.tema, typeof c.auth);
```

- **A** `3 luigi a=b string`.
- B `4 luigi a=b string`.
- C `3 luigi a string`.
- D `3 luigi a=b boolean`.
- E `4 luigi a boolean`.
- F `3 luigi b string`.

Risposta: A. L'header `Cookie` contiene coppie `nome=valore` separate da `;`. Il cookie `vuoto` non ha valore e viene scartato. Ricomponendo con `join("=")` tutto ciò che segue il primo `=`, il valore di `tema` resta `a=b`. I valori dei cookie sono stringhe: `auth` vale `"true"`, non il booleano `true`. *(Dispensa: cap. 13, «Recuperare le informazioni di sessione»)*

RACCOLTA

Es. **301.** **(2 punti)** Perché un'applicazione web ha bisogno di un meccanismo di *tracciamento della sessione*?

- **A** Perché HTTP è *stateless*: una richiesta non contiene informazioni sulle precedenti, e il server non saprebbe, per esempio, che l'utente ha già effettuato il login.
- B Perché HTTP chiude la connessione TCP dopo ogni risposta.
- C Perché i browser non possono inviare due richieste allo stesso server.
- D Perché senza sessione le risposte non possono essere cifrate.
- E Perché il DNS deve sapere quale utente sta usando il sito.
- F Perché HTTP/1.1 non permette di inviare form con il metodo `POST`.

Risposta: A. Il tracciamento della sessione mantiene informazioni con stato sulle interazioni di un utente attraverso più richieste HTTP, così il server può riconoscerlo e adattare le risposte. *(Dispensa: cap. 13, «Il problema»)*

Es. **302.** **(2 punti)** Dopo il login, il server risponde con la risposta seguente. Che cosa fa il browser?

```
1 HTTP/1.1 302 Found
2 Location: /
3 Set-Cookie: auth=true; max-age=3600
4 Set-Cookie: username=luigi; max-age=3600
```

- **A** Memorizza due cookie distinti, validi per un'ora, poi segue il reindirizzamento con una `GET /` che li include nell'header `Cookie`.
- B Memorizza un solo cookie, perché ogni risposta può impostarne uno.
- C Ignora i cookie, perché una risposta `302` non può impostarne.
- D Memorizza i cookie per 3600 giorni.
- E Rimanda subito i cookie al server con una `POST /`.
- F Mostra all'utente una pagina con i cookie ricevuti.

Risposta: A. Ogni header `Set-Cookie` imposta un cookie (`max-age` è in secondi: 3600 è un'ora). Il browser memorizza i cookie qualunque sia il codice di stato, poi segue il `Location` con una `GET` e li allega, come a ogni richiesta successiva allo stesso sito. Il reindirizzamento dopo la `POST` di login evita di reinviare le credenziali aggiornando la pagina. *(Dispensa: cap. 13, «Gestire il login»)*

Es. **303.** **(2 punti)** Un'applicazione memorizza nei cookie `auth=true` e `username=luigi` dopo il login, e controlla quei cookie a ogni richiesta. Quali delle seguenti affermazioni sono **vere**?

- **A** Un utente può impostarsi da solo `auth=true` senza mai passare dal login.
- **B** Un utente può cambiare `username` e farsi riconoscere come un altro utente.
- **C** Con i cookie firmati dal server la manomissione si riconosce, ma i dati restano sul client.
- **D** I browser limitano la dimensione e il numero dei cookie per dominio.
- E I cookie non sono modificabili, perché li ha impostati il server.
- F Il problema si risolve rendendo i valori più lunghi.
- G I cookie possono contenere una quantità illimitata di dati.
- H Il browser invia i cookie solo alla pagina di login.
- I Il problema riguarda solo i cookie impostati da JavaScript.

Risposte: A, B, C, D. I cookie sono interamente controllati dal client: dai DevTools si leggono e si modificano con un doppio clic. In più sono limitati (di solito 4096 byte ciascuno, da 20 a 50 per dominio). La soluzione sono le sessioni: i dati restano sul server, il cookie porta solo un identificatore. *(Dispensa: cap. 13, «I problemi dell'approccio a cookie»)*

RACCOLTA

Es. **304.** **(2 punti)** Quali delle seguenti affermazioni sulle sessioni web sono **vere**?

- **A** I dati della sessione sono conservati sul server.
- **B** Il client riceve solo un identificatore di sessione, di solito in un cookie.
- **C** Uno stesso utente può avere più sessioni, per esempio da due browser diversi.
- **D** Un identificatore che non corrisponde a una sessione esistente non dà accesso a nessun dato.
- E Il browser conserva in un cookie tutti i dati della sessione, cifrati.
- F L'identificatore di sessione può essere un contatore, perché il server lo controlla.
- G Le sessioni rendono HTTP un protocollo con stato.
- H Per ogni sessione il server apre una connessione TCP dedicata che resta aperta.
- I Ogni utente può avere una sola sessione alla volta.

Risposte: A, B, C, D. Una sessione è una struttura di coppie chiave-valore gestita sul server e identificata da un token univoco e imprevedibile, passato al client come cookie. HTTP resta *stateless*: è l'applicazione a ricollegare le richieste alla sessione. Ogni login crea una sessione nuova. *(Dispensa: cap. 13, «Le sessioni web»)*

Es. **305.** **(2 punti)** Perché l'identificatore di sessione va generato con un generatore crittograficamente sicuro, come `randomUUID` del modulo `crypto`, e non con un contatore o con `Math.random()`?

- **A** Perché chi conosce un identificatore valido è, per il server, il proprietario della sessione: un identificatore prevedibile si indovina, e permette di rubare la sessione di un altro.
- B Perché un contatore non può essere memorizzato in un cookie.
- C Perché `Math.random()` restituisce numeri troppo lunghi per un cookie.
- D Perché `randomUUID` cifra i dati della sessione.
- E Perché un identificatore casuale rende la sessione più veloce da trovare nella `Map`.
- F Perché i browser rifiutano i cookie con valori numerici.

Risposta: A. Con identificatori progressivi basterebbe provare il valore successivo. `Math.random()` non è pensato per la sicurezza; `randomUUID` produce 122 bit casuali, e indovinarne uno valido è praticamente impossibile. *(Dispensa: cap. 13, «Implementare le sessioni da zero»)*

Es. **306.** **(2 punti)** Un'applicazione conserva le sessioni in una `Map` in memoria. Quali problemi può avere in produzione?

- **A** Al riavvio del server tutte le sessioni spariscono, e gli utenti devono rifare il login.
- **B** Con più macchine dietro un bilanciatore di carico, una richiesta servita da una macchina non trova la sessione creata su un'altra.
- C I dati della sessione diventano visibili al client.
- D Una `Map` non può contenere più di 20 sessioni.
- E Le sessioni scadono dopo ogni richiesta.
- F Gli utenti possono modificare i dati della sessione dai DevTools.

Risposte: A, B. La memoria del processo non sopravvive ai riavvii e non è condivisa fra più macchine. In produzione le sessioni si conservano in un archivio condiviso (Redis, un database), oppure si usano token autoconsistenti e firmati. *(Dispensa: cap. 13, «Usare le sessioni»)*

Es. **307.** **(2 punti)** Oltre ai cookie, con quali altri metodi si può trasmettere l'identificatore di sessione?

- **A** Con l'*URL rewriting*: l'identificatore viene aggiunto a ogni URL come parametro.
- **B** Con campi nascosti nei form, inviati insieme agli altri dati.
- C Nell'header `Host` della richiesta.
- D Nel codice di stato della risposta.
- E Nell'indirizzo IP del client.
- F Nel nome del file HTML richiesto.

Risposte: A, B. I cookie sono l'approccio più diffuso. URL rewriting e campi nascosti funzionano anche senza cookie, ma espongono l'identificatore in luoghi (URL, log, link condivisi) dove è facile che venga copiato. *(Dispensa: cap. 13, «Altri metodi di tracciamento»)*

RACCOLTA

Es. **308.** **(5 punti)** Si spieghi il problema del tracciamento della sessione in HTTP e si confrontino l'approccio che memorizza lo stato nei cookie e quello basato sulle sessioni web.

Traccia della risposta.

- HTTP è *stateless*: per riconoscere un utente fra più richieste (es. dopo il login) serve uno stato mantenuto dall'applicazione.
- Stato nei cookie (`Set-Cookie: auth=true`, `username=...`): semplice, ma i cookie sono controllati dal client e si manomettono facilmente; limiti di dimensione e numero. I cookie firmati riconoscono la manomissione ma lasciano i dati al client.
- Sessioni: i dati restano sul server, in una struttura chiave-valore; il client riceve solo un identificatore opaco (di solito un cookie) che il server usa per ritrovarli.
- L'identificatore deve essere imprevedibile (generatore crittograficamente sicuro): chi lo conosce è il proprietario della sessione.

(Dispensa: cap. 13, «Le sessioni web»)

Es. **309.** **(5 punti)** Si descriva come implementare da zero un meccanismo di sessione in Node.js: strutture dati, che cosa avviene al login e a ogni richiesta successiva, e quali accorgimenti servono in produzione.

Traccia della risposta.

- Una `Map` dagli identificatori alle sessioni, ciascuna a sua volta una `Map` di coppie chiave-valore.
- Al login: si crea una sessione con un identificatore casuale (`randomUUID`), vi si salvano i dati (es. `username`) e si invia l'identificatore con `Set-Cookie`, di solito con un reindirizzamento.
- A ogni richiesta: si legge il cookie dall'header `Cookie`, si cerca la sessione; se non esiste l'utente non è autenticato (`401` o reindirizzamento al login).
- In produzione: sessioni in un archivio condiviso (Redis, database), che sopravvive ai riavvii ed è comune a più macchine; scadenza delle sessioni.

(Dispensa: cap. 13, «Implementare le sessioni da zero»)

Es. **310.** **(5 punti)** Un'applicazione memorizza lo stato nei cookie `auth` e `username`. Si descriva quali header si scambiano browser e server al login e nelle richieste successive, e si spieghi perché questo approccio non è sicuro.

Traccia della risposta.

- Il form invia le credenziali con una `POST`; se sono giuste il server risponde con un reindirizzamento (`302 Found`, `Location: /`) e due header `Set-Cookie` (`auth=true; max-age=3600` e `username=luigi; max-age=3600`), altrimenti ripropone il login con `401`.
- Il browser memorizza i cookie e da quel momento li allega a ogni richiesta allo stesso sito in un unico header `Cookie: auth=true; username=luigi`, che il server deve analizzare (coppie separate da `;`, nome e valore da `=`).
- Le pagine protette controllano `auth`: se manca rispondono `401` o reindirizzano al login.
- I cookie sono controllati dal client: con i DevTools si cambia `username` o ci si imposta `auth=true` senza login. I cookie firmati rivelano la manomissione, ma i dati restano sul client: la soluzione sono le sessioni web.

(Dispensa: cap. 13, «Un approccio ingenuo: i cookie» e «I problemi dell'approccio a cookie»)

Capitolo 14 — CSS moderno e adattivo

Es. **311.** **(2 punti)** Qual è la specificità del selettore `ul > li:has(> a.link)`?

- **A** (0, 1, 3)
- B (0, 0, 2)
- C (0, 1, 4)
- D (0, 2, 3)
- E (1, 1, 3)
- F (0, 1, 2)

Risposta: A. Le pseudo-classi funzionali non contano come pseudo-classi: `:where()` vale sempre zero, con tutto il suo contenuto; `:is()`, `:not()` e `:has()` valgono quanto il loro argomento **più specifico** (non la somma degli argomenti), anche se a selezionare l'elemento è un altro argomento. Qui `ul > li` vale (0, 0, 2) e la parte funzionale (0, 1, 1): in tutto (0, 1, 3). *(Dispensa: cap. 14, «La specificità delle pseudo-classi funzionali»)*

Es. **312.** **(2 punti)** Si consideri il seguente insieme di regole CSS incluse in un singolo foglio di stile importato nella pagina web il cui `<body>` è riportato di seguito. Qual è il colore finale dell'elemento `#primo`?

```css
#contenuto :where(li) { color: orange; }
:is(#contenuto, main) .attiva { color: teal; }
li:not(.attiva) { color: purple; }
li:has(+ .voce) { color: red; }
```

```html
<main id="contenuto">
  <ul class="lista">
    <li class="voce attiva" id="primo">Uno</li>
    <li class="voce">Due</li>
  </ul>
</main>
```

- **A** teal
- B red
- C black
- D orange
- E purple

Risposta: A. Specificità delle regole: `#contenuto :where(li)` (1, 0, 0), `:is(#contenuto, main) .attiva` (1, 1, 0), `li:not(.attiva)` (non seleziona l'elemento), `li:has(+ .voce)` (0, 1, 1). Vince la specificità più alta fra le regole che selezionano l'elemento (a parità, l'ultima nel foglio di stile): `:is(#contenuto, main) .attiva`. *(Dispensa: cap. 14, «La specificità delle pseudo-classi funzionali»)*

Es. **313.** **(2 punti)** Si consideri il seguente foglio di stile, applicato alla pagina il cui `<body>` è riportato di seguito. Qual è il colore finale dell'elemento `#dentro`?

```css
.tema { --colore: red; }
.nota { color: var(--accento, green); }
```

```html
<div class="tema">
  <p class="nota" id="dentro">Avviso</p>
</div>
<p class="nota" id="fuori">Altro avviso</p>
```

- **A** green
- B black
- C teal
- D red
- E blue

Risposta: A. `--accento` non è definita da nessuna parte: `var()` usa il valore di riserva, `green`. *(Dispensa: cap. 14, «Custom properties»)*

Es. **314.** **(2 punti)** Si considerino le regole seguenti, applicate a un `div` di classe `elemento` contenuto in un `div` di classe `contenitore`. Quanto è largo l'elemento (con `1rem` pari a 16px)?

```css
.contenitore {
  width: 400px;
}
.elemento {
  width: calc(100%-20px);
}
```

- **A** 400px
- B 380px
- C 200px
- D 150px
- E 20px
- F 420px

Risposta: A. Nella proprietà `width` le percentuali si riferiscono alla larghezza del contenitore (400px). `min()` e `max()` scelgono il più piccolo e il più grande degli argomenti, `clamp(minimo, preferito, massimo)` vincola il valore preferito fra i due estremi. Dentro `calc()` gli operatori `+` e `-` vanno circondati da spazi: senza, la dichiarazione non è valida e viene scartata, e il `div` resta largo quanto il contenitore. *(Dispensa: cap. 14, «Valori calcolati»)*

RACCOLTA

Es. **315.** **(2 punti)** Un contenitore di classe `contenitore` ha tre figli vuoti, nell'ordine di classe `a`, `b` e `c`, a cui si applicano le regole seguenti. Quanto sono larghi, rispettivamente, i tre figli?

```
.contenitore {
  display: flex;
  width: 300px;
}
.a { flex: 1 2 200px; }
.b { flex: 1 1 200px; }
.c { flex: 1 1 200px; }
```

- **A** 50px, 125px e 125px
- B 100px, 100px e 100px
- C 200px, 200px e 200px
- D 125px, 125px e 50px
- E 70px, 115px e 115px

Risposta: A. `flex: grow shrink basis`: si parte dalle basi (qui 200px, 200px, 200px, in tutto 600px) e le si confronta con il contenitore (300px). Lo spazio che avanza si distribuisce in proporzione a `flex-grow`; quello che manca si toglie in proporzione a `flex-shrink` **moltiplicato per la base**. *(Dispensa: cap. 14, «La scorciatoia flex»)*

Es. **316.** **(2 punti)** Si consideri il seguente foglio di stile, applicato alla pagina riportata di seguito e visualizzata in una finestra larga 1000px. Di che colore è lo sfondo delle due card?

```
.card { background-color: lightblue; }
main { container-type: inline-size; }
aside { container-type: inline-size; }
@container (max-width: 300px) {
  .card { background-color: lightgreen; }
}
```

```
<main style="width: 250px">
  <div class="card">Main</div>
</main>
<aside style="width: 220px">
  <div class="card">Aside</div>
</aside>
```

- **A** Nel `<main>` verde (`lightgreen`), nell'`<aside>` verde (`lightgreen`).
- B Nel `<main>` azzurro (`lightblue`), nell'`<aside>` azzurro (`lightblue`).
- C Nel `<main>` azzurro (`lightblue`), nell'`<aside>` verde (`lightgreen`).
- D Nel `<main>` verde (`lightgreen`), nell'`<aside>` azzurro (`lightblue`).

Risposta: A. Una media query osserva il **viewport**, che qui è largo 1000px. Una container query osserva il contenitore più vicino, cioè l'antenato con `container-type`, e vale per i suoi **discendenti**: una card diventa verde solo se sta in un contenitore largo al più 300px. *(Dispensa: cap. 14, «Container query»)*

Es. **317.** **(2 punti)** Si consideri il seguente foglio di stile, che usa l'annidamento nativo, applicato alla pagina il cui `<body>` è riportato di seguito. Qual è il colore finale dell'elemento `h2`?

```
.card {
  color: black;

  p {
    color: green;
  }

  & > h2 {
    color: red;
  }

  .scuro & {
    color: white;
  }
}
```

```
<section class="scuro">
  <div class="card">
    <h2>Titolo</h2>
    <p>Testo <span>bis</span></p>
    <small>Nota</small>
  </div>
</section>
```

- **A** red
- B black
- C green
- D white

Risposta: A. Una regola annidata senza `&` si combina con quella esterna come **discendente**; `&` si riferisce esplicitamente al selettore della regola genitore, e può stare anche in fondo (`.scuro &`). `& > h2` equivale a `.card > h2`: il titolo è rosso. *(Dispensa: cap. 14, «Annidamento nativo»)*

RACCOLTA

Es. **318.** **(2 punti)** Si consideri il seguente insieme di regole CSS incluse in un singolo foglio di stile importato nella pagina web il cui `<body>` è riportato di seguito. Qual è il colore finale dell'elemento `#a`?

```css
li { color: black; }
li:has(+ .scelta) { color: red; }
ol:has(.attiva) li { color: blue; }
```

```html
<ol>
  <li id="a">A</li>
  <li class="scelta">B</li>
  <li class="attiva">C</li>
</ol>
```

- **A** blue
- B red
- C black
- D green

Risposta: A. Entrambe le regole colorate selezionano `#a`: `li:has(+ .scelta)` perché il suo fratello immediatamente successivo ha la classe `scelta` (specificità (0, 1, 1)), `ol:has(.attiva) li` perché l'elenco contiene un elemento `.attiva` (specificità (0, 1, 2)). Vince la seconda. *(Dispensa: cap. 14, «:has()»)*

Es. **319.** **(2 punti)** Si consideri il seguente foglio di stile, applicato alla pagina il cui `<body>` è riportato di seguito. Qual è il colore di sfondo finale dell'elemento `ol`?

```css
ol:has(.attiva) {
  // gli elenchi con un elemento attivo
  background-color: lightgray;
  color: navy;
}
```

```html
<ol>
  <li>A</li>
  <li class="attiva">B</li>
</ol>
```

- **A** transparent
- B lightgray
- C navy
- D white

Risposta: A. In CSS i commenti si scrivono solo con `/* */`. `//` viene letto come l'inizio di una dichiarazione non valida, che arriva fino al primo punto e virgola: viene scartata insieme a `background-color`. Lo sfondo resta quello predefinito, trasparente (`color: navy` invece si applica). *(Dispensa: cap. 14, «:has()»)*

Es. **320.** **(2 punti)** Su uno smartphone la barra degli indirizzi compare e scompare durante lo scorrimento. Quale unità corrisponde all'altezza del viewport **corrente**, che cambia di conseguenza?

- **A** `dvh`
- B `svh`
- C `lvh`
- D `vw`
- E `vmin`
- F `rem`

Risposta: A. `svh` è l'altezza con i controlli del browser visibili (la più piccola), `lvh` quella con i controlli nascosti (la più grande, come `vh`), `dvh` l'altezza corrente, che passa dall'una all'altra mentre si scorre. *(Dispensa: cap. 14, «L'altezza del viewport sui dispositivi mobili»)*

Es. **321.** **(2 punti)** A quale terna di `flex-grow`, `flex-shrink` e `flex-basis` equivale `flex: 1`?

- **A** `1 1 0%`
- B `1 1 auto`
- C `1 0 auto`
- D `0 1 auto`
- E `1 0 0%`
- F `0 0 auto`

Risposta: A. `flex: 1` equivale a `1 1 0%`: base nulla, quindi tutto lo spazio viene diviso in parti uguali, indipendentemente dal contenuto. `flex: auto` è `1 1 auto`, `flex: none` è `0 0 auto`, e il valore iniziale è `0 1 auto`. *(Dispensa: cap. 14, «La scorciatoia flex»)*

RACCOLTA

Es. **322.** **(2 punti)** Un contenitore flex ha `gap: 30px` e `padding: 10px`. Dove si trova lo spazio di 30px?

- **A** Solo fra un figlio e l'altro (anche fra le righe, se vanno a capo), mai fra i figli e il bordo del contenitore.
- B Attorno a ogni figlio, anche verso il bordo del contenitore.
- C Solo fra il primo figlio e il bordo sinistro del contenitore.
- D Fra il bordo e il contenuto del contenitore, come il padding.
- E Solo fra le righe, quando i figli vanno a capo.
- F `gap` funziona solo con Grid: nel contenitore flex non ha effetto.

Risposta: A. `gap` si usa sia con Grid sia con Flexbox e aggiunge spazio solo fra gli elementi; verso il bordo provvede il `padding` del contenitore. *(Dispensa: cap. 14, «gap con Grid e Flexbox»)*

Es. **323.** **(2 punti)** Un'immagine larga quanto il viewport ha `srcset="foto-400.jpg 400w, foto-800.jpg 800w, foto-1200.jpg 1200w"`. Quale file scaricherà presumibilmente uno smartphone con un viewport largo 375px e densità di pixel 2?

- **A** `foto-800.jpg`
- B `foto-400.jpg`
- C `foto-1200.jpg`
- D Tutti e tre i file, poi sceglie.
- E Il file indicato in `src`, perché `srcset` vale solo sui desktop.

Risposta: A. Senza `sizes` l'immagine si considera larga `100vw`: servono $375 \cdot 2 = 750$ pixel, e il candidato più adatto è quello da 800. Il browser scarica solo il file scelto; `src` resta il valore di riserva per chi non supporta `srcset`. *(Dispensa: cap. 14, «srcset»)*

Es. **324.** **(2 punti)** Quali delle seguenti affermazioni sull'elemento `<picture>` sono **vere**?

- **A** Il browser usa il primo `<source>` la cui media query è vera.
- **B** Se nessuna media query è vera, viene usato il file dell'`<img>`.
- **C** L'`<img>` è obbligatoria: è l'elemento che viene davvero mostrato.
- **D** Permette l'*art direction*: versioni diverse dell'immagine per condizioni diverse.
- E Il browser scarica tutti i file indicati e poi sceglie il migliore.
- F Il browser usa l'ultimo `<source>` la cui media query è vera.
- G Se c'è almeno un `<source>`, l'`<img>` viene ignorata.
- H Funziona solo con immagini dello stesso formato e dello stesso ritaglio.

Risposte: A, B, C, D. I `<source>` cambiano solo il file dell'`<img>`, che resta l'elemento mostrato; si usa il primo `<source>` adatto, altrimenti il file dell'`<img>`, e viene scaricato solo il file scelto. L'*art direction* sceglie il contenuto più adatto (per esempio un ritaglio diverso), non solo la risoluzione. *(Dispensa: cap. 14, «picture e art direction»)*

Es. **325.** **(2 punti)** Quale media query applica degli stili solo agli utenti che hanno chiesto, nelle impostazioni, di ridurre le animazioni?

- **A** `@media (prefers-reduced-motion: reduce)`
- B `@media (prefers-reduced-motion: reduced)`
- C `@media (prefers-color-scheme: dark)`
- D `@media (reduce-motion: true)`
- E `@media (prefers-contrast: more)`
- F `@supports (animation: none)`

Risposta: A. Le media query sulle preferenze dell'utente riflettono le impostazioni del browser o del sistema: `prefers-reduced-motion`, `prefers-color-scheme`, `prefers-contrast`, `prefers-reduced-transparency`. Il valore è `reduce`: con `reduced` la media query non è valida e viene ignorata. *(Dispensa: cap. 14, «Media query sulle preferenze dell'utente»)*

RACCOLTA

Es. **326.** **(2 punti)** A che cosa serve la regola `@supports (display: grid) { ... }`?

- **A** Ad applicare gli stili che contiene solo se il browser supporta `display: grid`, lasciando agli altri le regole di riserva.
- B A caricare una libreria che aggiunge il supporto per Grid ai browser vecchi.
- C Ad applicare gli stili solo se l'elemento ha già `display: grid`.
- D A segnalare un errore nella console se il browser non supporta Grid.
- E A trasformare automaticamente i layout Flexbox in Grid.
- F Ad applicare gli stili solo sugli schermi larghi.

Risposta: A. `@supports` verifica il supporto di una proprietà con un certo valore. Prima si scrive una regola che funziona ovunque, poi la si arricchisce dove possibile: è il *progressive enhancement. (Dispensa: cap. 14, «@supports»)*

Es. **327.** **(2 punti)** Dentro la regola `.card { ... }`, che differenza c'è fra la regola annidata `&:hover { ... }` e la regola annidata `:hover { ... }`?

- **A** La prima equivale a `.card:hover` (la card sotto il puntatore), la seconda a `.card :hover` (un qualunque discendente della card sotto il puntatore).
- B Nessuna: in entrambi i casi il selettore è `.card:hover`.
- C La prima equivale a `.card :hover`, la seconda a `.card:hover`.
- D La seconda non è valida: le pseudo-classi non si possono annidare.
- E La prima vale solo per il primo figlio della card.
- F La seconda vale per qualunque elemento della pagina sotto il puntatore.

Risposta: A. Senza `&` un selettore annidato si combina con quello esterno come **discendente**. `&` rappresenta il selettore della regola genitore e serve quando la regola riguarda l'elemento stesso o un combinatore diverso dallo spazio: tutta la differenza sta in uno spazio. *(Dispensa: cap. 14, «Il selettore &»)*

Es. **328.** **(5 punti)** Si descrivano le pseudo-classi funzionali `:is()`, `:where()`, `:not()` e `:has()`, con un esempio d'uso ciascuna, e le regole con cui contribuiscono alla specificità.

Traccia della risposta.

- `:is()` raggruppa selettori alternativi (es. `:is(header, main, footer) a em`); `:where()` fa lo stesso ma con specificità zero, utile per stili predefiniti facili da sovrascrivere.
- `:not()` esclude gli elementi che corrispondono ad almeno un argomento (es. `li:not(.selected, .active)`).
- `:has()` seleziona in base a discendenti o elementi collegati, con selettori relativi (es. `ol:has(.active)`, `li:has(+ .selected)`).
- Specificità: non contano come pseudo-classi; `:where()` vale zero; le altre valgono quanto il loro argomento più specifico, qualunque sia quello che seleziona l'elemento.

(Dispensa: cap. 14, «Selettori moderni»)

Es. **329.** **(5 punti)** Che cosa sono le *custom property* del CSS? Se ne descrivano dichiarazione e uso, il rapporto con cascata ed ereditarietà, i valori di riserva e la regola `@property`.

Traccia della risposta.

- Valori definiti da chi scrive il foglio di stile e riutilizzabili: nome con due trattini (`--primary-color: #663399`, maiuscole significative), letti con `var()`.
- Partecipano alla cascata, sono ereditate e si possono sovrascrivere: definite in `:root` valgono per tutta la pagina, ridefinite in un elemento valgono per i suoi discendenti (temi, modalità scura con una media query).
- `var(--nome, riserva)`: il secondo argomento si usa se la proprietà non è definita.
- `@property` registra la proprietà con tipo (`syntax`), ereditarietà (`inherits`) e valore iniziale (`initial-value`).

(Dispensa: cap. 14, «Custom properties»)

Es. **330.** **(5 punti)** Che limite delle media query superano le *container query*? Si mostri con un esempio come si dichiara un contenitore e come si scrive una query.

Traccia della risposta.

- Le media query osservano il viewport: un componente (es. una card) non può adattarsi allo spazio del contenitore in cui si trova, grande o piccolo che sia.
- Le container query permettono di dare stile a un componente in base alle dimensioni di un suo antenato, il contenitore.
- Il contenitore si dichiara con `container-type: inline-size` (si interroga sulla larghezza).
- Le query si scrivono con `@container (max-width: 300px) { ... }` e valgono per i discendenti del contenitore più vicino, non per il contenitore stesso.

(Dispensa: cap. 14, «Container query»)

Es. **331.** **(5 punti)** Perché servono le immagini responsive? Si descrivano l'attributo `srcset` (con i due tipi di descrittori e l'attributo `sizes`) e l'elemento `<picture>`.

Traccia della risposta.

- Scaricare un'immagine ad alta risoluzione per rimpicciolirla su uno schermo piccolo spreca byte: si forniscono più file (candidati) e il browser scarica solo quello adatto.
- `srcset` con descrittori di larghezza (`800w`, la larghezza del file) o di densità (`2x`); `src` resta il valore di riserva.
- `sizes` dice quanto sarà larga l'immagine (predefinito `100vw`): larghezza per densità dà i pixel necessari, e il browser sceglie il candidato più adatto.
- `<picture>` con più `<source media=... srcset=...>` e un'`<img>` obbligatoria: si usa il primo `<source>` adatto; serve per l'*art direction* (contenuti diversi, non solo risoluzioni).

(Dispensa: cap. 14, «Immagini responsive»)

Capitolo 15 — Express

Es. **332.** **(2 punti)** Si consideri la seguente applicazione Express, in ascolto sulla porta 3000. Quali risposte (codice di stato e corpo) ottengono, nell'ordine, le richieste `GET /webtech`, `GET /corsi/42` e `GET /corsi/42/esami`?

```
1  import express from "express";
2
3  const app = express();
4
5  app.get(/^\/w.*tech$/, (req, res) => res.send("B"));
6  app.get("/webtech", (req, res) => res.send("A"));
7  app.get("/corsi/:id", (req, res) => res.send("C " + req.params.id));
8  app.post("/corsi/:id", (req, res) => res.send("D " + req.params.id));
9
10 app.listen(3000);
```

- **A** `200 B, 200 C 42, 404 Cannot GET /corsi/42/esami.`
- B `200 A, 200 C 42, 404 Cannot GET /corsi/42/esami.`
- C `200 B, 200 C 42, 200 C 42/esami.`
- D `200 B, 404 Cannot GET /corsi/42, 404 Cannot GET /corsi/42/esami.`
- E `200 B, 404 Cannot GET /corsi/42, 200 C 42/esami.`
- F `404 Cannot GET /webtech, 200 C 42, 200 C 42/esami.`

Risposta: A. Quando più rotte corrispondono vince **la prima** dichiarata. L'espressione regolare `/^\/w.*tech$/` vuole tutto il percorso, dalla `w` dopo la barra fino a `tech` in fondo. Un parametro (`:id`) cattura esattamente un segmento, e `req.params` contiene stringhe. Per un percorso senza rotta, o per un metodo senza rotta, Express risponde `404` (*Cannot METODO percorso*). *(Dispensa: cap. 15, «Percorsi delle rotte» e «Parametri di rotta»)*

RACCOLTA

Es. **333.** **(2 punti)** Si consideri la seguente applicazione Express, in ascolto sulla porta 3000. Che cosa risponde a una richiesta `GET /x`?

```
1  import express from "express";
2
3  const app = express();
4
5  app.use((req, res, next) => { req.passi = ["A"]; next(); });
6  app.get("/x", (req, res, next) => { req.passi.push("B"); next(); });
7  app.get("/x", (req, res) => res.send(req.passi.concat("C").join(",")));
8  app.use((req, res, next) => { req.passi.push("D"); next(); });
9  app.use("/y", (req, res, next) => { req.passi.push("E"); next(); });
10 app.get("/x", (req, res) => res.send(req.passi.join(",")));
11
12 app.listen(3000);
```

- **A** `200 A,B,C.`
- B `200 A,B,C,D.`
- C `200 A,B,C,D,E.`
- D `200 A.`
- E `200 C,B,A.`

Risposta: A. Middleware e gestori vengono eseguiti **nell'ordine in cui sono dichiarati**, ciascuno solo se corrisponde alla richiesta: `app.use` senza percorso vale per tutte, `app.use("/x", ...)` per `/x` e per i percorsi che cominciano con `/x/`, `app.get` solo per le `GET` con quel percorso esatto (`/x/:id` vuole un segmento in più). `next()` passa al successivo; il primo che risponde chiude il ciclo, e chi viene dopo non viene eseguito. *(Dispensa: cap. 15, «I middleware» e «Più gestori per una rotta»)*

Es. **334.** **(2 punti)** Si consideri la seguente applicazione Express, in ascolto sulla porta 3000. Quali risposte (codice di stato e corpo) ottengono, nell'ordine, le richieste `GET /ok` e `GET /errore`?

```
1  import express from "express";
2
3  const app = express();
4
5  app.get("/ok", (req, res) => res.send("ok"));
6  app.get("/errore", (req, res) => {
7    throw new Error("Rotto");
8  });
9  app.use((err, req, res, next) => {
10   res.send("Gestito: " + err.message);
11 });
12
13 app.listen(3000);
```

- **A** `200 ok`, `200 Gestito: Rotto.`
- B `200 ok`, `500 Internal Server Error.`
- C `200 ok`, `500 Gestito: Rotto.`
- D `500 Gestito: Rotto`, `200 Gestito: Rotto.`
- E `200 ok`, `500 Error: Rotto.`

Risposta: A. Il gestore personalizzato riceve l'errore ma non imposta il codice di stato: la risposta parte con il valore predefinito, 200. Un gestore di errori si riconosce dall'**arità**: quattro parametri, `(err, req, res, next)`. *(Dispensa: cap. 15, «Gestione degli errori»)*

RACCOLTA

Es. **335.** **(2 punti)** Si consideri la seguente applicazione Express, in ascolto sulla porta 3000. Che cosa rispondono due richieste `POST /form`, la prima con il corpo `msg=Ciao+mondo&num=42` e l'header `Content-Type: application/x-www-form-urlencoded`, la seconda con il corpo `{"msg": "Ciao", "num": 7}` e l'header `Content-Type: application/json`?

```
import express from "express";

const app = express();

app.post("/form", (req, res) => {
  res.send(`msg=${req.body?.msg}; num=${req.body?.num}`);
});

app.listen(3000);
```

- **A** 200 `msg=undefined; num=undefined`, 200 `msg=undefined; num=undefined`.
- B 200 `msg=undefined; num=undefined`, 200 `msg=Ciao; num=7`.
- C 200 `msg=Ciao+mondo; num=42`, 200 `msg=Ciao; num=7`.
- D 200 `msg=Ciao mondo; num=42`, 200 `msg=undefined; num=undefined`.
- E 200 `msg=Ciao+mondo; num=42`, 200 `msg=undefined; num=undefined`.
- F 200 `msg=Ciao mondo; num=42`, 200 `msg=Ciao; num=7`.

Risposta: A. `express.urlencoded` analizza i corpi codificati come URL (quelli dei form, decodifica compresa), `express.json` quelli in JSON: è il `Content-Type` a decidere, e ciascuno ignora le richieste che non sono nel suo formato. Se nessun middleware analizza il corpo, `req.body` resta indefinito. *(Dispensa: cap. 15, «express.urlencoded» e «express.json»)*

Es. **336.** **(2 punti)** Si consideri la seguente applicazione Express, in ascolto sulla porta 3000. Quali risposte (codice di stato e corpo) ottengono, nell'ordine, le richieste `GET /a`, `GET /b`, `GET /c` e `GET /d`?

```
import express from "express";

const app = express();

app.get("/a", (req, res) => res.send({ ok: true }));
app.get("/b", (req, res) => res.sendStatus(404));
app.get("/c", (req, res) => res.status(201).send("creato"));
app.get("/d", (req, res) => res.redirect("/a"));

app.listen(3000);
```

- **A** 200 `{"ok":true}`, 404 Not Found, 201 creato, 302 -> /a.
- B 200 `[object Object]`, 404 Not Found, 201 creato, 302 -> /a.
- C 200 `{"ok":true}`, 404 , 201 creato, 302 -> /a.
- D 200 `{"ok":true}`, 404 Not Found, 200 creato, 302 -> /a.
- E 200 `{"ok":true}`, 404 Not Found, 201 creato, 200 `{"ok":true}`.
- F 200 `{"ok":true}`, 200 404, 201 creato, 301 -> /a.

Risposta: A. `res.send` con un oggetto lo converte in JSON (e imposta `Content-Type: application/json`); `res.sendStatus` imposta il codice e invia come corpo la sua rappresentazione testuale; `res.status` imposta il codice e restituisce `res`, per concatenare l'invio; `res.redirect` risponde con un 302 e l'header `Location` (il browser seguirebbe da solo il reindirizzamento). *(Dispensa: cap. 15, «Rispondere al client»)*

RACCOLTA

Es. **337.** **(2 punti)** Si consideri la seguente applicazione Express, in ascolto sulla porta 3000. Quali risposte (codice di stato e corpo) ottengono, nell'ordine, le richieste `GET /custom/echo/ciao`, `GET /echo/ciao` e `GET /custom/echo`?

```
1  import express from "express";
2
3  const app = express();
4
5  const router = express.Router();
6  router.get("/echo/:valore", (req, res) => res.send(req.params.valore));
7
8  app.use("/custom", router);
9
10 app.listen(3000);
```

- **A** `200 ciao`, `404 Cannot GET /echo/ciao`, `404 Cannot GET /custom/echo`.
- B `200 ciao`, `200 ciao`, `404 Cannot GET /custom/echo`.
- C `404 Cannot GET /custom/echo/ciao`, `200 ciao`, `404 Cannot GET /custom/echo`.
- D `200 ciao`, `404 Cannot GET /echo/ciao`, `200 .`
- E `200 ciao`, `404 Cannot GET /echo/ciao`, `200 undefined`.
- F `200 /custom/echo/ciao`, `404 Cannot GET /echo/ciao`, `404 Cannot GET /custom/echo`.

Risposta: **A.** Un router modulare montato con `app.use("/custom", router)` gestisce percorsi **relativi** a quel prefisso: `/custom/echo/ciao` arriva alla rotta `/echo/:valore`, mentre `/echo/ciao` da solo non esiste. Il parametro deve catturare un segmento: `/custom/echo` non corrisponde. *(Dispensa: cap. 15, «Rotte concatenabili e router modulari»)*

Es. **338.** **(2 punti)** Si consideri la seguente applicazione Express, in ascolto sulla porta 3000. Quali risposte (codice di stato e corpo) ottengono, nell'ordine, le richieste `GET /book`, `DELETE /book` e `PUT /book`?

```
1  import express from "express";
2
3  const app = express();
4
5  app.route("/book")
6    .get((req, res) => res.send("Get a book"))
7    .post((req, res) => res.send("Add a book"))
8    .delete((req, res) => res.send("Delete a book"));
9
10 app.listen(3000);
```

- **A** `200 Get a book`, `200 Delete a book`, `404 Cannot PUT /book`.
- B `200 Get a book`, `200 Delete a book`, `405 Method Not Allowed`.
- C `200 Get a book`, `200 Delete a book`, `200 Add a book`.
- D `200 Get a book`, `404 Cannot DELETE /book`, `404 Cannot PUT /book`.
- E `200 Get a book`, `200 Delete a book`, `200 Get a book`.
- F `200 Get a book`, `200 Delete a book`, `500 Internal Server Error`.

Risposta: **A.** `app.route()` specifica il percorso una volta sola e concatena i gestori per metodo. Per un metodo che non ha gestore (qui `PUT`) Express risponde 404, come per un percorso inesistente. *(Dispensa: cap. 15, «Rotte concatenabili e router modulari»)*

RACCOLTA

Es. **339.** **(2 punti)** Si consideri la seguente applicazione Express, in ascolto sulla porta 3000. Che cosa rispondono tre richieste `GET /contatore`: la prima senza cookie, la seconda con l'header `Cookie: count=41`, la terza con l'header `Cookie: count=abc`?

```
1  import express from "express";
2  import cookieParser from "cookie-parser";
3
4  const app = express();
5  app.use(cookieParser());
6
7  app.get("/contatore", (req, res) => {
8    const n = req.cookies.count ? parseInt(req.cookies.count) + 1 : 1;
9    res.cookie("count", n, { maxAge: 10 * 60 * 1000 });
10   res.send(`Visita numero ${n}`);
11 });
12
13 app.listen(3000);
```

- **A** `200 Visita numero 1`, `200 Visita numero 42`, `200 Visita numero NaN`.
- B `200 Visita numero 1`, `200 Visita numero 1`, `200 Visita numero 1`.
- C `200 Visita numero 1`, `200 Visita numero 2`, `200 Visita numero 3`.
- D `200 Visita numero 1`, `200 Visita numero 411`, `200 Visita numero abc1`.
- E `200 Visita numero 1`, `200 Visita numero 42`, `200 Visita numero 1`.
- F `200 Visita numero 1`, `200 Visita numero 42`, `400 Bad Request`.

Risposta: A. `cookie-parser` mette i cookie della richiesta in `req.cookies`, come stringhe. Il contatore sta nel client, che può inviare il valore che vuole: il server crede a `41`, e con `abc` `parseInt` dà `NaN`. Con `express-session` il contatore starebbe invece sul server, e il cookie conterrebbe solo l'identificatore firmato della sessione. *(Dispensa: cap. 15, «cookie-parser»)*

Es. **340.** **(2 punti)** Che cosa si intende con *inversione del controllo* (la «legge di Hollywood»), caratteristica distintiva dei framework?

- **A** Che è il framework a controllare il flusso di esecuzione e a invocare il codice scritto dallo sviluppatore quando serve, e non il contrario.
- B Che il codice dello sviluppatore invoca le funzioni del framework, che poi gli restituiscono il controllo.
- C Che il client controlla il server, invece del contrario.
- D Che le richieste HTTP vengono elaborate in ordine inverso.
- E Che il framework impone di scrivere il codice dalla fine all'inizio.
- F Che il database controlla la logica dell'applicazione.

Risposta: A. «Non chiamarci tu, ti chiamiamo noi»: con una libreria è il nostro codice a chiamare le funzioni, che restituiscono il controllo; con un framework il nostro codice viene innestato nel framework, che riceve la richiesta e lo invoca (per esempio i gestori delle rotte). *(Dispensa: cap. 15, «Inversione del controllo»)*

Es. **341.** **(2 punti)** Express si definisce un framework *unopinionated*. Che cosa significa?

- **A** Che non impone un modo specifico di organizzare l'applicazione: le scelte (struttura, librerie, convenzioni) restano allo sviluppatore.
- B Che non ha una documentazione ufficiale.
- C Che non permette di usare motori di template.
- D Che impone un insieme rigido di convenzioni decise dai suoi autori.
- E Che non gestisce il routing.
- F Che funziona solo con le API REST.

Risposta: A. Un framework fortemente *opinionated* arriva con convenzioni rigide: più coerenza e velocità, ma più rigidità e una curva di apprendimento più ripida. Express è minimalista e unopinionated: per questo, per esempio, non esiste un unico modo giusto di organizzarne un'applicazione. *(Dispensa: cap. 15, «Framework opinionated»)*

RACCOLTA

Es. **342.** **(2 punti)** Quali delle seguenti affermazioni sui middleware di Express sono **vere**?

- **A** Ricevono come argomenti `req`, `res` e `next`.
- **B** Vengono eseguiti nell'ordine in cui sono dichiarati.
- **C** Possono aggiungere proprietà all'oggetto richiesta, che i middleware successivi trovano già pronte.
- **D** Se non chiamano `next()` e non rispondono, la richiesta resta appesa.
- E Vengono eseguiti dopo che la risposta è stata inviata.
- F Montati con `app.use` senza percorso, valgono solo per la rotta `/`.
- G Express li ordina automaticamente per percorso.
- H Devono sempre inviare una risposta.
- I Possono essere usati solo per l'autenticazione.

Risposte: A, B, C, D. I middleware stanno nel mezzo del ciclo richiesta-risposta: elaborano la richiesta (log, corpo, cookie, sessione, autenticazione...) e passano al successivo con `next()`, oppure rispondono e chiudono il ciclo. Dimenticare `next()` senza rispondere lascia la richiesta appesa fino al timeout, senza alcun messaggio d'errore. *(Dispensa: cap. 15, «I middleware»)*

Es. **343.** **(2 punti)** Un gestore di rotta è una funzione `async` che fa `await` di una Promise che viene rifiutata. Che cosa succede con Express 5?

- **A** Express invoca automaticamente `next(err)`: l'errore arriva al gestore degli errori.
- B Il rifiuto non viene gestito e Node.js termina l'intero processo.
- C La richiesta resta appesa finché il client non va in timeout.
- D Express risponde 200 con un corpo vuoto.
- E L'errore viene ignorato e si passa alla rotta successiva.
- F Express riavvia il server.

Risposta: A. Da Express 5 i gestori che restituiscono una Promise rifiutata invocano `next(err)` da soli. Con Express 4 la Promise veniva ignorata: il rifiuto non gestito faceva terminare il processo, e bisognava usare `try/catch` e chiamare `next(err)` a mano. *(Dispensa: cap. 15, «Le Promise in Express 5»)*

Es. **344.** **(2 punti)** Un'applicazione esegue `app.use("/static", express.static("public"))`. Con quale URL si ottiene il file `public/css/style.css`?

- **A** `/static/css/style.css`
- B `/public/css/style.css`
- C `/css/style.css`
- D `/static/public/css/style.css`
- E `/style.css`
- F `/static/style.css`

Risposta: A. `express.static` serve i file di una cartella, ricavando il `Content-Type` dall'estensione. Montato sul percorso virtuale `/static`, mappa `/static/<percorso>` sul file `public/<percorso>`; montato senza percorso, `public/css/style.css` sarebbe `/css/style.css`. *(Dispensa: cap. 15, «express.static»)*

Es. **345.** **(2 punti)** Quali delle seguenti affermazioni su `express-session` e `cookie-parser` sono **vere**?

- **A** Con `express-session` i dati stanno sul server e il cookie contiene solo l'identificatore della sessione.
- **B** L'identificatore della sessione è firmato con il `secret`: se il client lo modifica, il server lo ignora.
- **C** Con `cookie-parser` i valori stanno nei cookie, e il client può modificarli a piacere.
- **D** L'archivio predefinito di `express-session` tiene le sessioni in memoria: in produzione ne serve uno dedicato.
- E `cookie-parser` cifra i cookie, che il client non può leggere.
- F `express-session` salva i dati della sessione nel browser.
- G Il `secret` di `express-session` si può pubblicare senza rischi.
- H Una richiesta senza cookie di sessione riceve comunque la sessione dell'ultimo client.
- I `saveUninitialized: false` crea una sessione per ogni visitatore.

Risposte: A, B, C, D. `express-session` realizza le sessioni web del capitolo 13: dati sul server, identificatore firmato nel cookie `connect.sid` (il `secret` va tenuto segreto). `cookie-parser` analizza l'header `Cookie` in `req.cookies`: i valori viaggiano e restano sul client. `saveUninitialized: false` non crea sessioni per chi non vi ha scritto nulla. *(Dispensa: cap. 15, «express-session»)*

RACCOLTA

Es. **346.** **(2 punti)** Dopo aver cancellato un elemento, il server reindirizza l'utente alla lista e vuole mostrargli «Elemento cancellato». Perché servono i *messaggi flash*?

- **A** Perché il reindirizzamento avvia un ciclo richiesta-risposta nuovo: il messaggio va conservato (nella sessione) fino alla richiesta successiva, che lo mostra e lo cancella.
- B Perché `res.redirect` non può inviare testo al client.
- C Perché i messaggi flash vengono mostrati dal browser in una finestra di dialogo.
- D Perché i messaggi flash si salvano nel database e restano visibili per sempre.
- E Perché senza di essi il reindirizzamento non funziona.
- F Perché HTTP non permette di inviare due risposte alla stessa richiesta.

Risposta: A. È ancora l'assenza di stato di HTTP. `connect-flash` salva i messaggi nella sessione (va registrato dopo `express-session`); `req.flash(chiave)` li legge e li consuma, quindi ricaricando la pagina non compaiono più: è una notifica «usa e getta». *(Dispensa: cap. 15, «Messaggi flash»)*

Es. **347.** **(2 punti)** Un'applicazione scritta per Express 4 dichiara la rotta `app.get("/web(-)?tech*", ...)`. Che cosa succede eseguendola con Express 5?

- **A** Il programma si ferma già all'avvio, quando la rotta viene dichiarata, con un errore sulla sintassi del percorso.
- B Funziona come in Express 4: corrisponde a `/webtech`, `/web-tech`, `/webtechs`…
- C La rotta viene ignorata in silenzio e tutte le richieste ricevono 404.
- D La rotta corrisponde solo al percorso letterale `/web(-)?tech*`.
- E Express 5 la converte automaticamente in un'espressione regolare.
- F L'errore compare solo alla prima richiesta che corrisponde.

Risposta: A. Express 5 non ammette più nei percorsi i caratteri speciali `?`, `+`, `*` e le parentesi: la dichiarazione solleva un `PathError` all'avvio. Le parti facoltative si scrivono con le graffe (`/web{-}tech`), una sequenza qualsiasi con un jolly con nome (`{*resto}`), oppure si usa un'espressione regolare. *(Dispensa: cap. 15, «Percorsi delle rotte»)*

Es. **348.** **(2 punti)** In un gestore Express, a che cosa serve assegnare `res.locals.items = [...]` prima di `res.render("lista")`?

- **A** A rendere disponibile `items` a tutte le viste generate per quella risposta.
- B A salvare `items` nella sessione dell'utente.
- C A memorizzare `items` in un cookie.
- D A inviare `items` al client in formato JSON.
- E A rendere `items` disponibile a tutte le richieste successive.
- F A salvare `items` nel file `views/lista.pug`.

Risposta: A. `res.locals` vive per la sola risposta corrente: i suoi dati sono disponibili ai template generati con `res.render` (che cerca la vista nella cartella `views` con il motore impostato da `app.set("view engine", "pug")`). Un middleware può riempirlo per tutte le pagine. *(Dispensa: cap. 15, «Passare dati alla vista»)*

Es. **349.** **(5 punti)** Che differenza c'è fra una libreria e un framework? Si spieghi l'inversione del controllo e si descriva che cosa offre tipicamente un web framework, con riferimento a Express.

Traccia della risposta.

- Libreria: funzioni che il nostro codice chiama, e che restituiscono il controllo; il flusso lo decide il nostro codice.
- Framework: progetto astratto in cui il nostro codice viene innestato; è il framework a controllare il flusso e a chiamare il nostro codice (IoC, «non chiamarci, ti chiamiamo noi»).
- Un web framework offre routing, analisi delle richieste, cookie e sessioni, un motore di template, spesso un ORM; può essere più o meno *opinionated*.
- Express: minimalista e unopinionated; rotte (`app.get(percorso, gestore)`, parametri, router), middleware, `res.render` con un motore di template.

(Dispensa: cap. 15, «Che cos'è un framework»)

RACCOLTA

Es. **350.** **(5 punti)** Si descriva il ciclo richiesta-risposta di Express e il ruolo dei middleware, con un esempio. Che cosa succede se un middleware non chiama `next()`?

Traccia della risposta.

- Arrivata una richiesta, Express crea `req` e `res` e la fa passare per una sequenza di middleware e gestori, nell'ordine di dichiarazione, finché qualcuno non risponde.
- Un middleware è una funzione `(req, res, next)`: può modificare `req` e `res` (es. aggiungere `req.timestamp`), eseguire codice, rispondere, o passare al successivo con `next()`.
- Si montano con `app.use([percorso], fn)`; esempi: log, `express.static`, `express.urlencoded`, sessione, autenticazione.
- Senza `next()` e senza risposta la richiesta resta appesa fino al timeout del client, senza messaggi d'errore.

(Dispensa: cap. 15, «I middleware»)

Es. **351.** **(5 punti)** Si descriva la gestione degli errori in Express: errori nel codice sincrono e asincrono, gestori personalizzati, differenze fra Express 4 ed Express 5.

Traccia della risposta.

- Un errore sollevato nel codice sincrono di un gestore viene catturato da Express e passato al gestore degli errori (predefinito: `500`, traccia solo in sviluppo).
- Un errore nato in una callback asincrona va passato a mano con `next(err)`.
- Gestori personalizzati: middleware con quattro parametri `(err, req, res, next)`, dichiarati dopo le rotte; con tre parametri non vengono riconosciuti.
- Express 5: una Promise rifiutata da un gestore (es. `async` con `await`) invoca `next(err)` da sola; Express 4 la ignorava e il processo terminava.

(Dispensa: cap. 15, «Gestione degli errori»)

Es. **352.** **(5 punti)** Si confrontino `cookie-parser` ed `express-session`, e si spieghi a che cosa servono i messaggi flash e come funzionano.

Traccia della risposta.

- `cookie-parser` analizza l'header `Cookie` in `req.cookies`; con `res.cookie` si impostano cookie i cui valori restano sul client, che può modificarli.
- `express-session` tiene i dati sul server e mette nel cookie solo l'identificatore, firmato con il `secret` (manomissioni riconosciute); archivio in memoria solo per lo sviluppo.
- Messaggi flash: dopo un'azione si reindirizza, e il messaggio va conservato fino alla richiesta successiva, che è un nuovo ciclo.
- `connect-flash` li salva nella sessione (dopo `express-session`): `req.flash(k, msg)` li aggiunge, `req.flash(k)` li legge e li consuma.

(Dispensa: cap. 15, «Middleware utili» e «Messaggi flash»)

Capitolo 16 — Una web app con Express

RA

Es. **353.** **(2 punti)** Si consideri la seguente applicazione Express, in ascolto sulla porta 3000. Che cosa rispondono, nell'ordine, le richieste `GET /`, `GET /nonesiste`, `GET /todo` e `GET /todo` con l'header `Cookie: login=si`?

```
1  import express from "express";
2
3  const app = express();
4  const todoRouter = express.Router();
5
6  function richiediLogin(req, res, next) {
7    if (req.headers.cookie === "login=si") next();
8    else res.redirect("/auth");
9  }
10 todoRouter.use(richiediLogin);
11 todoRouter.get("/", (req, res) => res.send("La lista"));
12 app.get("/", (req, res) => res.send("Home"));
13 app.use("/todo", todoRouter);
14 app.get("/*splat", (req, res) => res.status(404).send("Non trovato"));
15
16 app.listen(3000);
```

- **A** `200 Home, 404 Non trovato, 302 -> /auth, 200 La lista.`
- B `200 Home, 404 Non trovato, 302 -> /auth, 302 -> /auth.`
- C `302 -> /auth, 302 -> /auth, 302 -> /auth, 200 La lista.`
- D `200 Home, 302 -> /auth, 302 -> /auth, 200 La lista.`
- E `200 Home, 404 Non trovato, 200 La lista, 200 La lista.`
- F `200 Home, 404 Non trovato, 404 Non trovato, 200 La lista.`

Risposta: A. Un router montato senza percorso (`app.use(todoRouter)`) riceve **tutte** le richieste che le rotte precedenti non hanno gestito, e `todoRouter.use(mw)` senza percorso applica il middleware a tutte quelle che entrano nel router: anche `/nonesiste` verrebbe intercettata e reindirizzata al login invece di arrivare al *catch-all*. Il problema sparisce dando un percorso al middleware, montando il router sotto un prefisso o applicando il middleware rotta per rotta. *(Dispensa: cap. 16, «Una trappola: il middleware di un router»)*

Es. **354.** **(2 punti)** Quali delle seguenti affermazioni sulle librerie ORM (*Object-Relational Mapping*) sono **vere**?

- **A** Mantengono allineate la rappresentazione a oggetti dei dati e quella relazionale del database.
- **B** Permettono di definire in modo dichiarativo come le tabelle sono mappate sugli oggetti.
- **C** Rendono più facile passare da un database supportato a un altro.
- **D** Possono introdurre un sovraccarico nelle prestazioni, soprattutto se usate male.
- E Eliminano del tutto la necessità di un database.
- F Funzionano solo con i database che non usano SQL.
- G Rendono più semplice qualunque operazione rispetto all'SQL scritto a mano.
- H Impediscono di eseguire interrogazioni con condizioni.
- I Rendono impossibile cambiare la libreria ORM usata.

Risposte: A, B, C, D. Un ORM evita il codice ripetitivo che tiene allineati oggetti e tabelle: modelli dichiarativi, schema generato, operazioni CRUD e interrogazioni come chiamate di metodo, portabilità fra database. In cambio ci sono una curva di apprendimento, operazioni a volte più contorte che in SQL, un possibile sovraccarico e il *vendor lock-in* verso l'ORM. *(Dispensa: cap. 16, «Object-Relational Mapping»)*

Es. **355.** **(2 punti)** In Sequelize, che cosa fa `User.sync({ force: true })`?

- **A** Elimina la tabella, se esiste, e la ricrea vuota secondo il modello.
- B Crea la tabella solo se non esiste, altrimenti non fa nulla.
- C Controlla colonne e tipi della tabella e la modifica se serve, senza perdere dati.
- D Forza la scrittura su disco delle modifiche in sospeso.
- E Sincronizza i dati della tabella con quelli di un altro database.
- F Ricrea il modello JavaScript a partire dalla tabella esistente.

Risposta: A. `sync()` crea la tabella se non esiste; `sync({ alter: true })` la adatta al modello; `sync({ force: true })` la elimina e la ricrea, cancellando tutti i dati: comodo in sviluppo, distruttivo in produzione. *(Dispensa: cap. 16, «Sincronizzazione dei modelli»)*

RACCOLTA

Es. **356.** **(2 punti)** Si considerino i modelli `User` (con chiave primaria `id`) e `Todo`, e le associazioni `User.hasMany(Todo)` e `Todo.belongsTo(User)`. Che cosa aggiunge Sequelize allo schema?

- **A** La colonna `UserId` nella tabella `Todos`, chiave esterna verso `Users`.
- B La colonna `TodoId` nella tabella `Users`.
- C Una tabella di collegamento `UserTodos` con le due chiavi.
- D Le colonne `UserId` in `Todos` e `TodoId` in `Users`.
- E Nulla: le associazioni esistono solo negli oggetti JavaScript.
- F La colonna `userId` nella tabella `Users`.

Risposta: A. In un'associazione uno-a-molti la chiave esterna sta nella tabella dei «molti», e prende il nome del modello riferito seguito da quello della sua chiave primaria (`User` + `Id`). Una tabella di collegamento serve solo per il molti-a-molti (`belongsToMany` con `through`). *(Dispensa: cap. 16, «Modelli come classi e associazioni»)*

Es. **357.** **(2 punti)** Con Sequelize, `Todo.findAll()` restituisce le cose da fare ma non i loro utenti. Come si ottengono le une con gli altri in un'unica interrogazione?

- **A** Con `Todo.findAll({ include: [User] })`, che produce una *join*.
- B Con `Todo.findAll({ where: User })`.
- C Non si può: bisogna chiamare `User.findByPk` per ogni cosa da fare.
- D Con `Todo.findAll().join(User)`.
- E Con `Todo.findAll({ force: true })`.
- F Le associazioni vengono sempre caricate, basta leggere `todo.User`.

Risposta: A. Per impostazione predefinita le associazioni non vengono caricate: si chiedono con `include`, e Sequelize risponde con una `LEFT OUTER JOIN` (per questo una cosa da fare senza utente ha `User` nullo). *(Dispensa: cap. 16, «Caricare le associazioni, cercare per chiave»)*

Es. **358.** **(2 punti)** Nei log di Sequelize gli `INSERT` contengono segnaposto come `VALUES ($1,$2,NULL,$3,$4)` invece dei valori. Perché?

- **A** Sono query parametriche: i dati viaggiano separati dall'SQL, e questo protegge dalla *SQL injection*.
- B Perché Sequelize nasconde i dati sensibili nei log.
- C Perché i valori non sono ancora stati inseriti nel database.
- D Perché SQLite non accetta valori scritti dentro l'SQL.
- E Perché i valori sono cifrati prima di essere salvati.
- F Perché l'`INSERT` viene eseguito più tardi, in un'unica transazione.

Risposta: A. Con le query parametriche il database riceve separatamente l'istruzione e i dati: un valore inserito dall'utente non può mai diventare parte del comando SQL. `NULL` è l'`id`, scelto dal database grazie ad `AUTOINCREMENT`. *(Dispensa: cap. 16, «Creare entità»)*

Es. **359.** **(2 punti)** Quali delle seguenti pratiche sulla configurazione di un'applicazione sono corrette?

- **A** Tenere credenziali e impostazioni dell'ambiente in un file `.env`, letto all'avvio (per esempio con `dotenv`, che le copia in `process.env`).
- **B** Aggiungere il file `.env` al `.gitignore`, e versionare al suo posto una versione fittizia da copiare e modificare.
- **C** Usare file diversi per ambienti diversi, per esempio sviluppo e produzione.
- D Scrivere le credenziali nel codice e rimuoverle prima di pubblicare il repository.
- E Versionare il file `.env` con le password, così il gruppo le condivide.
- F Lasciare che `dotenv` sovrascriva le variabili d'ambiente già definite dal sistema.
- G Duplicare l'applicazione in una versione di sviluppo e una di produzione.

Risposte: A, B, C. Le impostazioni sensibili o specifiche dell'ambiente vanno in un unico file di configurazione non versionato (una chiave finita in un commit resta nella cronologia anche se poi viene rimossa). `dotenv` non sovrascrive le variabili già definite: così in produzione il sistema può imporre le proprie. *(Dispensa: cap. 16, «Configurazione»)*

RACCOLTA

Es. **360.** **(2 punti)** Nell'organizzazione dell'applicazione Express adottata nel corso, che cosa contiene la cartella `controllers`?

- **A** Il codice che implementa la logica di business per gestire le richieste.
- B I template Pug delle pagine.
- C La definizione dei router, che indirizzano le richieste.
- D I file statici: CSS, immagini, script.
- E I middleware personalizzati, come quello di autenticazione.
- F I modelli dei dati e la connessione al database.

Risposta: A. La struttura è: `views` (template), `routes` (router che indirizzano le richieste verso il controller giusto), `controllers` (logica di business), `models` (modelli e database), `middleware` (middleware personalizzati), `public` (file statici), e `index.js` come punto d'ingresso. Express è unopinionated: è una scelta, non una regola. *(Dispensa: cap. 16, «Organizzare un'applicazione Express»)*

Es. **361.** **(2 punti)** Un'applicazione usa `express-session` e `connect-flash`, che salva i messaggi nella sessione. In che ordine vanno registrati i middleware?

- **A** Prima `app.use(session(...))`, poi `app.use(flash())`.
- B Prima `app.use(flash())`, poi `app.use(session(...))`.
- C L'ordine non conta: Express li ordina da sé.
- D `flash()` va registrato dopo tutte le rotte.
- E `session(...)` va registrato dopo il gestore degli errori.
- F Vanno registrati nello stesso `app.use`, in qualunque ordine.

Risposta: A. I middleware vengono eseguiti nell'ordine di dichiarazione: la sessione deve essere già stabilita quando i middleware che la usano (i messaggi flash, quelli che ne copiano lo stato in `res.locals`) vengono eseguiti. *(Dispensa: cap. 16, «Configurazione dei middleware»)*

Es. **362.** **(2 punti)** Come vanno memorizzate le password degli utenti nel database di un'applicazione reale?

- **A** Come hash calcolato con una funzione lenta e con *salt* (bcrypt, scrypt, Argon2); al login si confronta l'hash.
- B In chiaro, così si possono confrontare direttamente al login.
- C Cifrate con una chiave scritta nel codice dell'applicazione.
- D Codificate in Base64.
- E Nella sessione dell'utente, invece che nel database.
- F In un cookie firmato, sul client.

Risposta: A. Confrontare direttamente la password (come nell'esempio del corso) vuol dire memorizzarla in chiaro: chi ottiene il database ottiene tutte le password. Una funzione di hash lenta e con salt rende costoso risalire alle password originali. *(Dispensa: cap. 16, «Autenticazione»)*

Es. **363.** **(2 punti)** Nella lista di cose da fare con Express, segnare un elemento come fatto, modificarlo o eliminarlo avviene con un form `POST`, seguito da un reindirizzamento. Perché?

- **A** Perché una `GET` non deve avere effetti (browser, proxy e motori di ricerca la ripetono o la anticipano), e il reindirizzamento evita di reinviare il form ricaricando la pagina (*Post/Redirect/Get*).
- B Perché i link non possono contenere l'identificativo dell'elemento.
- C Perché Express non supporta le rotte con il metodo `GET`.
- D Perché una `POST` è più veloce di una `GET`.
- E Perché i messaggi flash funzionano solo con le `POST`.
- F Perché il reindirizzamento cifra i dati del form.

Risposta: A. Le azioni che modificano i dati usano metodi non sicuri. Dopo la `POST` il server risponde `302` verso la lista: il browser la chiede con una `GET` innocua, che si può ricaricare quante volte si vuole. *(Dispensa: cap. 16, «Configurazione dei middleware»)*

RACCOLTA

Es. **364.** **(5 punti)** Che cos'è un ORM e quali problemi risolve? Si descriva, con Sequelize, come si definiscono i modelli e le associazioni, e si discutano pro e contro di questo approccio.

Traccia della risposta.

- Gli stessi dati hanno due rappresentazioni (oggetti nello heap, righe nelle tabelle) da tenere allineate: un ORM lo fa con modelli dichiarativi, schema generato, CRUD e interrogazioni come metodi.
- Sequelize: `database.define(...)` o classi che estendono `Model` con `init`; tipi e vincoli (`allowNull`, `primaryKey`, `autoIncrement`); `sync()` allinea lo schema.
- Associazioni: `hasOne`, `belongsTo`, `hasMany`, `belongsToMany` (con `through`); nell'uno-a-molti la chiave esterna (es. `UserId`) sta nella tabella dei «molti».
- Pro: astrazione, produttività, portabilità, leggibilità. Contro: curva di apprendimento, operazioni complesse più contorte, *vendor lock-in*, prestazioni.

(Dispensa: cap. 16, «Object-Relational Mapping»)

Es. **365.** **(5 punti)** Si descriva come realizzare l'autenticazione in un'applicazione Express con le sessioni e un middleware, e si spieghi il problema che nasce applicando il middleware con `router.use` a un router montato senza percorso.

Traccia della risposta.

- Al login (`POST`) il controller verifica le credenziali (password come hash) e scrive nella sessione `isAuthenticated` e il nome utente; il logout distrugge la sessione.
- Un middleware controlla la sessione e reindirizza al login (con un messaggio flash) chi non è autenticato, altrimenti chiama `next()`: un solo punto invece di un controllo in ogni controller.
- Si applica a tutte le rotte di un router (`router.use(mw)`) o a una sola rotta (`router.post("/reset", mw, gestore)`).
- Trappola: con `app.use(router)` e `router.use(mw)` senza percorso il middleware intercetta tutte le richieste non gestite prima (anche quelle per percorsi inesistenti, che non arrivano mai al 404); rimedio: `router.use("/todo", mw)` o `app.use("/todo", router)`.

(Dispensa: cap. 16, «Autenticazione» e «Una trappola: il middleware di un router»)

Es. **366.** **(5 punti)** Dove vanno impostazioni come le credenziali del database o le chiavi API di un'applicazione Express? Si spieghi perché non vanno scritte nel codice e come si gestiscono con un file `.env` e il pacchetto `dotenv`.

Traccia della risposta.

- Sono dati sensibili (password, chiavi API) o legati all'ambiente (URL del database di sviluppo o di produzione, livello di log).
- Nel codice finirebbero nel repository (una chiave committata resta nella cronologia anche dopo la rimozione), cambiarle richiederebbe di modificare più file e servirebbero versioni diverse dell'app per ogni ambiente.
- Vanno in un file di configurazione, unico punto in cui si cambiano: il `.env` nella radice del progetto, **non** versionato (`.gitignore`); si versiona invece una copia fittizia (es. `.env.dummy`) da copiare e completare.
- `import "dotenv/config"` legge il file all'avvio e ne copia i valori in `process.env`; le variabili d'ambiente già definite hanno la precedenza, così in produzione il sistema può imporre le proprie.

(Dispensa: cap. 16, «Configurazione»)